



Guerra de Cenizas

WAR OF ASHES

4X multijugador por turnos · mobile-first · la guerra como idioma de la negociación

DOCUMENTO

Diseño y arquitectura

VERSIÓN

0.2.0

FASE

1 — Diseño

FECHA

2026-08-24

Universo, mecánicas y assets originales.
Documento generado desde el repositorio.

Índice

I · VISIÓN

01 El proyecto

- | | |
|-------------------------|------------------|
| Concepto | Testing |
| Core loop | Desarrollo local |
| Features | Deploy |
| Arquitectura | Documentación |
| Estructura del proyecto | Roadmap |
| Instalación | Versionado |
| Scripts | Estado actual |

02 Discovery

- | | |
|--|-------------------------------------|
| 0. Resumen de la Fase 0 | 4. Dependencias y orden obligatorio |
| 1. Contradicciones detectadas en el brief | 5. Preguntas que sí son bloqueantes |
| 2. Riesgos | 6. Salida de la Fase 0 |
| 3. Sistemas que el brief pide y que no superan el filtro | |

II · DISEÑO DE JUEGO

03 Game Design Document

- | | |
|------------------------------|---------------------------------------|
| 1. Pilares de diseño | 8. Fuerzas y producción |
| 2. Fantasía y universo | 9. Sombra: inteligencia y operaciones |
| 3. Presupuesto de decisiones | 10. Anomalías: la capa sobrenatural |
| 4. Estructura de la campaña | 11. Investigación |
| 5. Recursos y economía | 12. Doctrinas |
| 6. Territorio y control | 13. El Núcleo y la victoria |
| 7. Combate | 14. Derrota, abandono y supervivencia |

04 Diplomacia

- | | |
|---|-----------------------------------|
| 1. La tesis | 8. La Coalición |
| 2. El núcleo mínimo | 9. Chat |
| 3. Ofertas: la unidad de interacción | 10. Estructuras de datos |
| 4. El Sello | 11. Visibilidad de la diplomacia |
| 5. Transferencias y el problema del primer movimiento | 12. Riesgos del sistema |
| 6. Información como moneda | 13. Plan de implementación (v0.4) |
| 7. Reputación | |

05 Metaprogresión

- | | |
|---------------------------------|----------------------------------|
| 1. Las dos progresiones | 5. La Ciudad |
| 2. La regla de oro | 6. Progresión de campaña |
| 3. Moneda: la Ceniza depositada | 7. Lo que NO habrá |
| 4. Qué se desbloquea | 8. Plan de implementación (v0.6) |

06 Sistema de facciones

- | | |
|---|------------------------------------|
| 1. Qué es una facción, y qué no es | 6. Cisma: cambiar de facción |
| 2. Las seis facciones | 7. Modelo de datos |
| 3. Economía de desbloqueo | 8. Tests |
| 4. Concordia: la facción dentro de la partida | 9. Lo que NO tendrán las facciones |
| 5. Renombre | 10. Plan de implementación |

07 Generación procedural

1. El problema, y por qué la solución habitual no sirve
2. Anatomía del mapa
3. La tubería
4. Paso 1 — Esqueleto
5. Pasos 2-3 — Decoración y perfiles económicos
6. Paso 4 — Perturbación acotada
7. Paso 5 — Evaluación
8. Colocación inicial
9. Informe de equidad
10. Reproducibilidad
11. Tests
12. Fuera de alcance en v1.0

08 Multijugador

1. Modelo de turno: simultáneo
2. Cadencias
3. Ciclo de vida de una partida
4. Autoridad y estado
5. Ausencias, abandonos y Mando Automático
6. Reconexión
7. Notificaciones
8. Rendimiento y escala
9. Casos límite
10. Plan de implementación (v0.3)

09 Technical Design Document

1. Principio rector
2. Arquitectura
3. El motor
4. Determinismo y aleatoriedad
5. Base de datos
6. Niebla de guerra y RLS
7. API
8. Resolución de turnos y concurrencia
9. Tiempo real, reconexión y persistencia
10. Frontend
11. Modelo de amenazas
12. Observabilidad y errores
13. Rendimiento
14. Versionado y migraciones

10 UX / UI — Mobile first

- | | |
|------------------------|-------------------------------------|
| 1. Principios | 7. Accesibilidad |
| 2. Gestos | 8. Estados y feedback |
| 3. Wireframes — Móvil | 9. Rendimiento en móvil |
| 4. Escritorio | 10. Dirección visual de la interfaz |
| 5. Diplomacia en móvil | 11. Checklist de QA móvil |
| 6. Onboarding | |

11 Pipeline de assets

- | | |
|--|----------------------------|
| 1. La decisión que lo define todo: los assets son código | 7. Criterios de aprobación |
| 2. Dirección artística | 8. Efectos y animación |
| 3. Inventario | 9. Marca |
| 4. Convención de nombres | 10. Versionado de assets |
| 5. Del SVG al componente | 11. Plan (v0.9) |
| 6. QA visual: la Galería de Assets | |

12 Testing y simulación

- | | |
|------------------------------------|----------------------------|
| 1. Principio | 7. Tests de mapas |
| 2. Pirámide | 8. Tests de rendimiento |
| 3. Tests unitarios (packages/core) | 9. Tests de viewport móvil |
| 4. Tests de integración | 10. CI |
| 5. Tests E2E (Playwright) | 11. Datos de prueba |
| 6. El simulador de balance | 12. Qué NO se testea |

13 Despliegue

- | | |
|-------------------------|------------------------------|
| 1. Lo que hay que crear | 4. El reloj |
| 2. Supabase | 5. Comprobación posterior |
| 2 bis. El pipeline | 6. Lo que puede salir mal |
| 3. Vercel | 7. Presupuesto del free tier |

V · EJECUCIÓN

14 Roadmap

- | | |
|--|--|
| Definición de hecho | v0.6 — Metaprogresión |
| Regla de secuencia | v0.7 — Anomalías y Sombra |
| v0.1 — Prototipo ☒ completada | v0.8 — Procedural y balance |
| v0.2 — Núcleo jugable ☒ completada | v0.9 — Pulido móvil, UX y assets |
| v0.3 — Multijugador real | v0.95 — Beta cerrada |
| v0.4 — Diplomacia | v1.0 — Release |
| v0.5 — El Núcleo | Post-1.0 (registrado, no comprometido) |

15 Registro de decisiones

- | | |
|--|---|
| ADR-001 — Un solo motor determinista compartido | ADR-007 — Estado de partida en jsonb, no normalizado |
| ADR-002 — El mapa es un grafo con simetría $C_{n \times n}$, no una rejilla | ADR-008 — Traición tarifada, no prohibida |
| ADR-003 — Combate determinista, sin dados | ADR-009 — La metaprogresión solo añade opciones |
| ADR-004 — Turnos simultáneos | ADR-010 — La Ciudad es un hub, no un gestor |
| ADR-005 — Autoridad en Route Handlers, no en funciones de Postgres | ADR-011 — Assets como SVG escritos a mano en el repositorio |
| ADR-006 — Niebla de guerra por playerviews prefiltradas | ADR-012 — Mapa en SVG, no Canvas ni WebGL |
| | ADR-013 — Sin eliminación de jugadores |
| | ADR-014 — Tres disparadores de resolución de turno |

01 El proyecto

README.md

Guerra de Cenizas

War of Ashes — 4X multijugador por turnos, mobile-first, donde la guerra es el idioma en el que se negocia.

v0.2.0 · Campaña jugable de 12 turnos · 163 tests en verde

[Concepto](#) · [Core loop](#) · [Arquitectura](#) · [Instalación](#) · [Documentación](#) · [Roadmap](#) · [Estado](#)

Concepto

Cinco ciudades son arrancadas del mundo y depositadas en el Umbral, un pliegue del espacio donde solo hay un Núcleo de Ceniza y no alcanza para todas. Tenéis doce turnos. Ninguna puede consagrarlo sola.

Guerra de Cenizas es un 4X por turnos para **2, 3 o 5 jugadores** en el que ganar en solitario es *aritméticamente imposible*: consagrar el objetivo cuesta más Ceniza de la que produce el reparto justo de un solo jugador. Necesitas aliados. Y solo uno —o dos— pueden ganar.

Esa única regla convierte cada partida en una cadena de pactos, precios, promesas y traiciones. La diplomacia no es un menú lateral: es el sistema principal, y el ejército es el argumento con el que se negocia.

Pitch

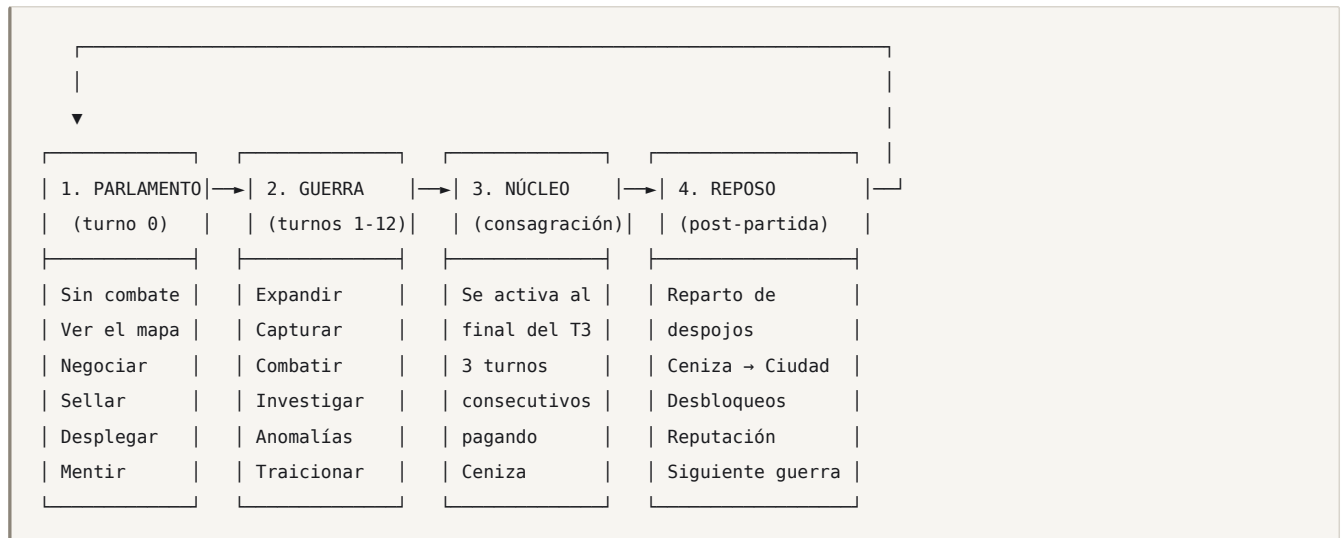
Género	4X por turnos, multijugador asíncrono, fuertemente diplomático
Plataforma	Web mobile-first (PWA). Funciona en desktop sin cambiar reglas.
Jugadores	2, 3 o 5. El modo de referencia es 5 .
Duración	12 turnos. Blitz ≈ 50 min · Diaria ≈ 6 días · Relajada ≈ 12 días
Ambientación	Militar moderno + fantasía contemporánea. Universo 100 % original.
Idiomas	Español e inglés desde la v1.0
Filosofía	Simple de aprender, difícil de dominar. 6 reglas, miles de situaciones.

Los tres pilares

- La victoria exige un aliado, y el aliado exige un precio.** El Núcleo se consagra pagando Ceniza durante 3 turnos consecutivos. Nadie produce suficiente. Comprar, conquistar o convencer: elige.
- El combate es determinista.** No hay dados. Antes de confirmar un ataque ves el resultado exacto. Puedes por tanto **prometer resultados** — y que se compruebe si cumpliste. La incertidumbre viene de lo que no sabes, no de la suerte.
- La traición no está prohibida: está tarifada.** Romper un Sello es legal, inmediato, público, y cuesta Ceniza. Es decir: traicionar te aleja de ganar. La pregunta nunca es «¿puedo?» sino «¿me sale a cuenta?».

Core loop

Una **campaña** (una partida) es un ciclo de cuatro fases:



Dentro de un turno (el bucle que el jugador repite 12 veces):

```
ver el mapa → leer el log del turno anterior → negociar →  
dar órdenes (mover · atacar · construir · investigar · anomalía) →  
enviar turno → [todos envían o vence el plazo] → resolución simultánea →  
ver qué mintió cada uno → repetir
```

Todos los jugadores dan órdenes **a la vez**; el servidor las resuelve de forma simultánea y determinista. Nadie espera a nadie.

Detalle completo: [docs/GAME_DESIGN.md](#).

Features

En la v1.0

- 🌐 **Mapas procedurales verificablemente justos** — grafo de regiones generado como un sector replicado por rotación C_n : la equidad es exacta *por construcción*, no por heurística. Con validación posterior de 8 métricas.
- 🤝 **Diplomacia con 3 primitivas vinculantes** (Sello, Transferencia con depósito, Coalición) + negociación por plantillas traducidas: dos jugadores sin idioma común pueden cerrar un trato.
- ⚔️ **Combate determinista** con 3 armas en piedra-papel-tijera y previsualización exacta del resultado.
- 🛡️ **Anomalías** — 8 capacidades sobrenaturales que no hacen daño: manipulan información, topología y compromisos.
- 🕵️ **Sombra** — operaciones de inteligencia, sabotaje y engaño; incluye detectar una traición *antes* de que ocurra.
- 🏰 **La Ciudad** — hub de metaprogresión entre guerras. Desbloquea **opciones**, nunca números.
- 💎 **Facciones ligadas a la cuenta** — seis ciudades signatarias. Cambian tu doctrina de origen y **abaratan** tu vía de desbloqueos, pero el techo es idéntico para todas (verificado por test). Dos jugadores de la misma facción quedan marcados con **Concordia**: público, y sin ningún efecto mecánico.
- 📱 **Mobile-first real** — diseñado a 360 px primero; objetivos táctiles ≥ 44 px; jugable con una mano.
- 🌐 **Español e inglés completos**, incluido el sistema diplomático.

- 🗝️ **Servidor autoritativo** con Supabase RLS y niebla de guerra criptográficamente real (el cliente nunca recibe lo que no debe ver).
- 🤖 **Simulador headless** de miles de partidas para balance automático.

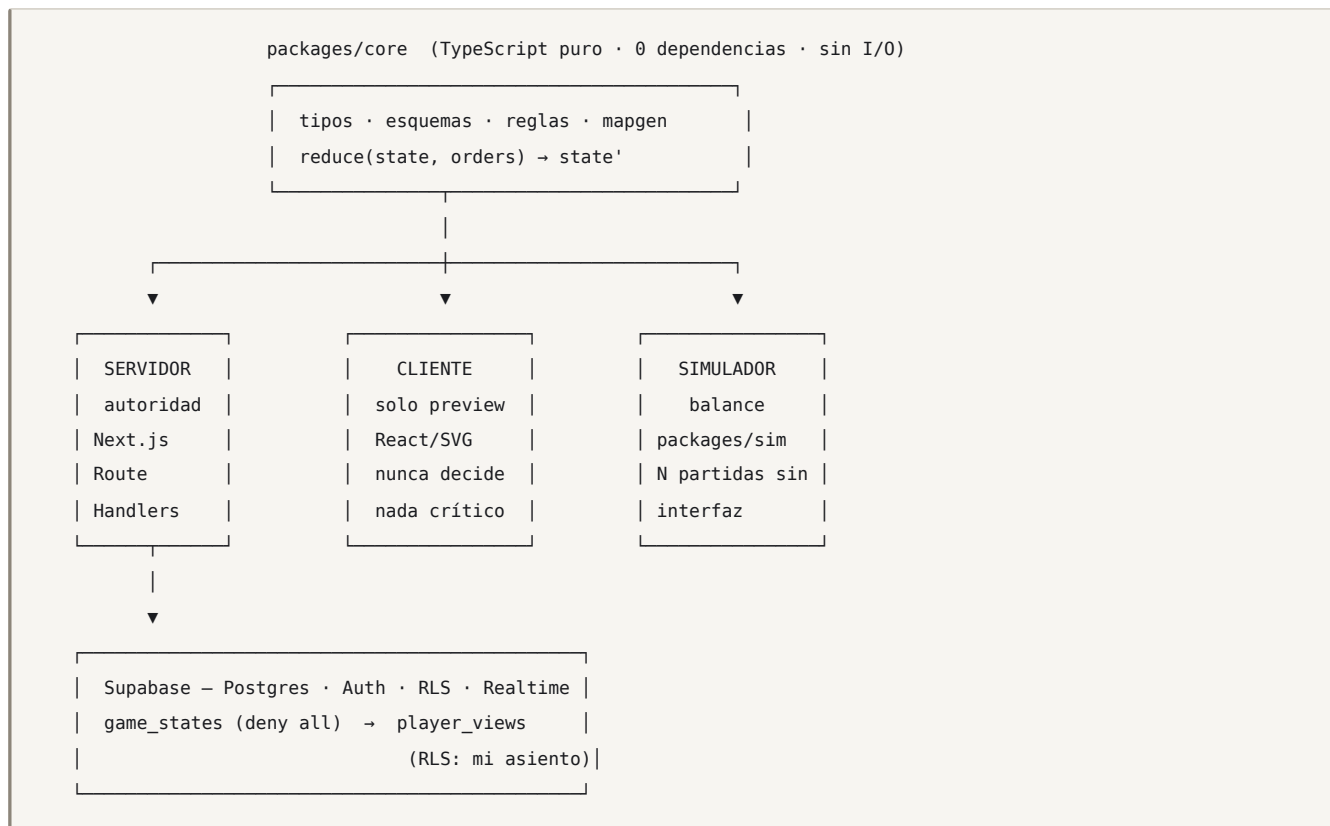
Explícitamente fuera de la v1.0

Gestión profunda de ciudad · objetivos ocultos individuales · más de 5 jugadores · ranking global · monetización · modo campaña PvE · tecnología orbital. Razones en [docs/DISCOVERY.md](#).

Arquitectura

Principio rector

Un solo motor. Tres consumidores.



El mismo `reduce()` corre en los tres sitios. Un test de determinismo comprueba que `checksum(reduce(s, o))` coincide en servidor, cliente y simulador.

Stack

Capa	Elección	Por qué esta y no otra
Framework	Next.js 16 (App Router) + React 19	Único stack de primera clase en Vercel con Route Handlers para la autoridad de servidor y RSC para reducir JS en móvil.
Lenguaje	TypeScript <code>strict</code>	El motor es lógica pura: los tipos son la primera línea de tests.
Estilos	Tailwind CSS v4	Sin librería de componentes. Los <code>design tokens</code> viven en CSS variables y los consumen a la vez la UI y los assets SVG.
Backend	Supabase (Postgres + Auth + RLS + Realtime)	Auth, base de datos, seguridad por filas y websockets en un solo free tier.
Validación	Zod	Un solo esquema sirve para validar la petición HTTP y para tipar el motor.
i18n	next-intl	Mensajes ICU (plurales/género), rutas <code>/es</code> <code>/en</code> , funciona en RSC.
Render del mapa	SVG + React	Un grafo de ~50-95 regiones no necesita WebGL. Gana en nitidez, accesibilidad (cada región es un elemento enfocable), peso y pipeline de assets.
Tests	Vitest (motor) + Playwright (E2E y viewport móvil)	
PDF de docs	markdown-it + Chromium (Playwright)	Reproducible dentro del repo, sin herramientas de pago.

Regla de dependencias: toda dependencia nueva necesita una entrada justificada en [docs/DECISIONS.md](#). `packages/core` debe permanecer en **cero dependencias de runtime**, para siempre.

Detalle completo: [docs/TECHNICAL_DESIGN.md](#).

Estructura del proyecto

```
guerra-de-cenizas/
├─ README.md                ← este documento; léelo antes de trabajar
├─ CHANGELOG.md
├─ docs/                    ← toda la documentación de diseño
│   ├── DISCOVERY.md        ← Fase 0: contradicciones y riesgos
│   ├── GAME_DESIGN.md      ← GDD
│   ├── TECHNICAL_DESIGN.md ← TDD
│   ├── MAP_GENERATION.md   ← generación procedural + scoring
│   ├── DIPLOMACY.md
│   ├── METAPROGRESSION.md
│   ├── MULTIPLAYER.md
│   ├── UX_MOBILE.md
│   ├── ASSET_PIPELINE.md
│   ├── TESTING_AND_SIMULATION.md
│   ├── ROADMAP.md
│   └─ DECISIONS.md        ← registro de decisiones (ADR)
│
├─ packages/
│   ├── core/               ← ★ motor puro. 0 deps. La fuente de la verdad.
│   │   ├── CLAUDE.md      ← reglas locales del paquete
│   │   └─ src/
│   │       ├── types/     ← estado, órdenes, eventos
│   │       ├── rules/     ← reduce(), movimiento, control, visibilidad
│   │       ├── mapgen/    ← esqueleto C_n, decoración, generación
│   │       ├── factions/  ← catálogo de facciones y desbloques de cuenta
│   │       ├── balance/   ← tablas de constantes (datos, no código)
│   │       ├── rng/       ← xoshiro128** determinista
│   │       └─ util/       ← JSON canónico y checksum
│   │   └─ tests/
│   └─ sim/                 ← simulador headless + perfiles de bot + estadísticas
│
├─ apps/
│   └─ web/                 ← Next.js
│       ├── app/[locale]/  ← rutas localizadas
│       ├── app/api/       ← Route Handlers = autoridad de servidor
│       ├── components/
│       ├── messages/      ← es.json · en.json
│       └─ e2e/            ← Playwright
│
├─ supabase/
│   ├── migrations/        ← SQL versionado
│   └─ seed.sql
│
├─ assets/                 ← fuentes SVG originales + tokens
│   └─ src/{units,terrain,icons,ui,effects}/
│
└─ tools/
    └─ docs-pdf/           ← generación del PDF de documentación
```

Instalación

Requisitos

- Node.js $\geq$ 22
- npm $\geq$ 10
- Una cuenta de [Supabase](#) (free tier) — solo desde v0.3
- Opcional: [Supabase CLI](#) para Postgres local

Puesta en marcha

```
git clone https://github.com/iCabellos/GuerraDeCenizas.git
cd GuerraDeCenizas
npm install

cp .env.example .env.local    # y rellena las variables (ver abajo)

npm run dev                  # http://localhost:3000
```

Variables de entorno

Variable	Ámbito	Descripción
<code>NEXT_PUBLIC_SUPABASE_URL</code>	público	URL del proyecto Supabase
<code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>	público	Clave publicable (<code>sb_publishable_...</code>). Solo puede lo que permita la RLS.
<code>SUPABASE_SECRET_KEY</code>	secreto — solo servidor	Clave secreta (<code>sb_secret_...</code>). Usada exclusivamente en Route Handlers para la resolución de turnos. Nunca en código de cliente.
<code>CRON_SECRET</code>	secreto	Compartido con <code>pg_cron</code> para autenticar el disparador de resolución.
<code>NEXT_PUBLIC_SITE_URL</code>	público	Dominio público, sin barra final. Es el <code>redirect_to</code> del enlace mágico; tiene que estar en la lista blanca de <code>supabase/config.toml</code> .

Los nombres antiguos — `NEXT_PUBLIC_SUPABASE_ANON_KEY` y `SUPABASE_SERVICE_ROLE_KEY`, con claves JWT `eyJ...` — siguen funcionando: es la misma clave con otro nombre y el mismo rol de Postgres detrás. Si defines las dos de un par, gana la nueva. `GET /api/health` responde qué falta, por nombre y nunca por valor.

 Cualquier variable con prefijo `NEXT_PUBLIC_` acaba en el navegador. La clave secreta **nunca** lleva ese prefijo. Hay un test de CI que falla si aparece.

Base de datos

Desde v0.3, el esquema vive en `supabase/migrations/` como SQL versionado:

```
npx supabase link --project-ref <tu-ref>
npx supabase db push      # aplica migraciones
npx supabase db reset     # reinicia local + seed
```

Tablas principales: `profiles`, `cities`, `account_unlocks`, `games`, `game_players`, `game_states`, `player_views`, `orders`, `turn_events`, `treaties`, `messages`, `match_results`. Esquema y políticas RLS completas en [docs/TECHNICAL_DESIGN.md](#).

Scripts

Script	Qué hace
<code>npm run dev</code>	Next.js en modo desarrollo
<code>npm run build</code>	Build de producción
<code>npm test</code>	Vitest — motor, mapgen, facciones, combate, economía (163 tests)
<code>npm run verify</code>	Todo lo anterior + typecheck + estructura + enlaces de docs
<code>npm run check:deps</code>	Reglas estructurales del monorepo
<code>npm run test:e2e</code>	Playwright — incluye viewports móviles
<code>npm run sim -- --games 5000 --players 5</code>	Simulador de balance
<code>npm run mapgen -- --seed 1234 --players 5 --report</code>	Genera un mapa y su informe de equidad
<code>npm run gallery</code>	Abre la Galería de Assets (QA visual)
<code>npm run docs:pdf</code>	Genera <code>docs/GuerraDeCenizas.pdf</code> desde los Markdown
<code>npm run docs:check</code>	Verifica los enlaces internos de la documentación
<code>npm run lint</code> / <code>npm run typecheck</code>	Calidad estática

Testing

El testing no es una fase: es una condición para que una versión exista.

Nivel	Herramienta	Qué cubre
Unitario	Vitest	Reglas del motor, combate, economía, captura, RNG
Determinismo	Vitest	<code>reduce()</code> da el mismo checksum en servidor, cliente y simulador
Mapgen	Vitest	1 000 semillas × {2,3,5} jugadores deben superar los umbrales de equidad
Balance	<code>packages/sim</code>	5 000 partidas; winrate por doctrina dentro de 42-58 %
Integración	Vitest + Supabase local	RLS, órdenes, resolución, reconexión
Seguridad	Vitest	Un cliente no puede leer <code>game_states</code> ni el <code>player_view</code> ajeno
E2E	Playwright	Flujo completo login → partida → fin, en 360×640 y 1440×900
Visual	Galería de Assets	Escala, contraste, legibilidad sobre cada terreno

Detalle: [docs/TESTING_AND_SIMULATION.md](#).

Desarrollo local

Sin Supabase (hasta v0.3 y para trabajar en el motor):

```

npm test -- --watch                # TDD sobre el motor
npm run mapgen -- --seed 42 --players 5 --report # inspeccionar un mapa
npm run sim -- --games 200 --seed 1 --verbose   # ver una partida simulada

```

Con Supabase local:

```

npx supabase start
npm run dev

```

Para probar multijugador en solitario: abre la partida en una ventana normal y otra de incógnito, o usa `npm run sim -- --seat-takeover` para que un bot ocupe los asientos restantes.

Deploy

Frontend → Vercel

1. Importa el repositorio en Vercel.
2. Configura las variables de entorno (marca `SUPABASE_SERVICE_ROLE_KEY` y `TURN_RESOLVER_SECRET` como *Sensitive*).
3. Deploy. `main` → producción; cualquier PR → preview.

Backend → Supabase

1. Crea el proyecto y aplica `supabase/migrations/`.
2. Activa `pg_cron` y `pg_net`.
3. Programa el disparador de resolución (cada minuto):

```

select cron.schedule('resolve-due-turns', '* * * * *', $$
  select net.http_post(
    url      := 'https://<tu-app>.vercel.app/api/cron/resolve-due',
    headers := jsonb_build_object('x-resolver-secret', current_setting('app.resolver_secret'))
  );
$$);

```

Coste y límites

Objetivo: **0 €/mes** durante toda la beta.

Servicio	Plan	Límite relevante	Cuándo lo rompemos	Alternativa
Vercel	Hobby	100 GB banda; prohibido uso comercial	Al monetizar, o ~50 k partidas/mes	Vercel Pro 20 \$/mes
Supabase	Free	500 MB BD · 5 GB egreso · 50 k MAU · 200 conexiones Realtime	~200 partidas simultáneas	Supabase Pro 25 \$/mes
Almacenamiento	—	~80 KB por partida terminada	~6 000 partidas archivadas	Poda + archivo en Storage

Una partida terminada se guarda como **semilla + órdenes**, no como estado: se puede reconstruir entera reproduciéndola. Ese es el truco que mantiene el coste plano.

Documentación

Documento	Contenido
DISCOVERY.md	Fase 0: contradicciones del brief, riesgos, recortes
GAME_DESIGN.md	Lore, facciones, recursos, combate, victoria, tutorial
TECHNICAL_DESIGN.md	Arquitectura, BD, RLS, API, concurrencia, seguridad
MAP_GENERATION.md	Generador procedural, métricas, scoring, pseudocódigo
DIPLOMACY.md	Las 3 primitivas vinculantes, reputación, traición
METAPROGRESSION.md	Progresión permanente vs. de partida, la Ciudad
FACTIONS.md	Facciones ligadas a la cuenta, desbloques, Cisma, Concordia
MULTIPLAYER.md	Turnos, cadencias, abandonos, reconexión, bots
UX_MOBILE.md	Wireframes, gestos, accesibilidad, rendimiento
ASSET_PIPELINE.md	Dirección artística, naming, QA visual
TESTING_AND_SIMULATION.md	Estrategia de test y simulador de balance
ROADMAP.md	v0.1 → v1.0 con criterios de aceptación
DECISIONS.md	Registro de decisiones arquitectónicas (ADR)

PDF: [docs/GuerraDeCenizas.pdf](#) — 129 páginas con portada, índice y paginación. Se regenera con `npm run docs:pdf` usando solo herramientas del repositorio (markdown-it + el Chromium de Playwright). Pipeline documentado en [tools/docs-pdf/](#).

Roadmap

Versión	Nombre	Entrega
v0.1 ✓	Prototipo	Mapa, jugadores, movimiento, turnos. Local, sin cuentas.
v0.2 ✓	Núcleo jugable	Recursos, producción, combate, captura, log de eventos
v0.3	Multijugador real	Auth, Supabase, persistencia, RLS, reconexión, cadencias
v0.4	Diplomacia	Sellos, transferencias con depósito, visión compartida, reputación
v0.5	El Núcleo	Objetivo especial, consagración, coaliciones, condiciones de victoria
v0.6	Metaprogresión	La Ciudad, desbloques, resultados de partida
v0.7	Anomalías	Sistema sobrenatural + operaciones de Sombra
v0.8	Procedural + balance	Generador completo, simulador, ajuste de constantes
v0.9	Pulido móvil	UX, onboarding, assets, accesibilidad, rendimiento
v0.95	Beta cerrada	Telemetría, feedback, estabilidad
v1.0	Release	ES+EN completos, sin bugs críticos, documentación al día

Criterios de aceptación por versión: [docs/ROADMAP.md](#).

Versionado

- **SemVer** para el proyecto (`0.x` = pre-release).
- **ENGINE_VERSION** independiente en `packages/core`. Toda partida guarda la versión de motor con la que se creó; una partida en curso nunca cambia de motor.
- **MAPGEN_VERSION** independiente. Un mapa se reproduce con `(seed, MAPGEN_VERSION)`.
- Cada versión: rama → PR → tests verdes → entrada en `CHANGELOG.md` → tag.

Estado actual

v0.2 — Núcleo jugable completado y verificado.

-  Fase 0 (Discovery): 7 contradicciones resueltas, 21 riesgos catalogados.
-  Documentación de diseño completa (13 documentos + PDF de 130 páginas).
-  **Motor** (`@gdc/core`): estado, PRNG determinista, `reduce()` con validación, movimiento simultáneo, control territorial, visibilidad y eventos filtrados por asiento.
-  **Generador de mapas**: esqueleto C_n , decoración y replicación por rotación, para 2, 3 y 5 jugadores.
-  **Sistema de facciones** ligado a la cuenta, con el invariante del techo verificado.
-  **Combate determinista**: rueda de armas, terreno, posturas, fortificación, apoyo de Fuego y previsualización que **coincide exactamente** con el resultado.
-  **Economía**: renta con rendimiento decreciente, suministro por distancia, producción.
-  **Prototipo web**: mapa SVG accesible, hot seat, previsualización de combate, producción y registro del turno filtrado por asiento.
-  **163 tests** en verde · typecheck limpio · reglas estructurales verificadas.
-  **Pendiente**: confirmar las 3 decisiones bloqueantes de **DISCOVERY §5**.
-  **Siguiente**: v0.3 — multijugador real (auth, Supabase, RLS, autoridad de servidor).

Cómo verlo funcionando

```
npm install && npm run dev    # http://localhost:3000
npm test                      # 163 tests
npm run verify                # typecheck + estructura + tests + enlaces
```

Decisiones importantes

Las cinco que definen el proyecto (el resto, en **DECISIONS.md**):

1. **El mapa es un grafo, no una rejilla.** Un sector generado y replicado n veces por rotación ⇒ equidad exacta por construcción para 2, 3 y 5 jugadores. Ninguna rejilla admite simetría de orden 5.
2. **El combate es determinista.** Sin dados. Permite prometer resultados, previsualizar batallas y que el simulador converja barato.
3. **Un solo motor de reglas**, TypeScript puro y sin dependencias, compartido por servidor, cliente y simulador.
4. **La niebla de guerra es real:** `game_states` niega todo `SELECT`; los jugadores solo leen `player_views`, proyecciones ya filtradas por asiento.
5. **La metaprogresión solo añade opciones**, jamás números. Un test de CI falla si un desbloqueo toca la tabla de balance, y la propia estructura lo impide: el módulo de facciones **no puede importar** el de balance.

Assets y propiedad intelectual

Todos los assets son **originales y creados dentro de este repositorio**. Se prohíbe incorporar assets de marketplaces, imágenes de terceros, iconografía con copyright incompatible o cualquier referencia a propiedad intelectual ajena. El universo —lore, facciones, nombres, poderes— es original.

Los assets se autoran como **SVG** en `assets/src/`, lo que hace que su procedencia sea trivialmente rastreable: están en el historial de git como código. Criterios de aprobación y QA visual: [ASSET_PIPELINE.md](#).

Licencia

Pendiente de definir por el propietario del repositorio (ver [DECISIONS.md](#) · ADR-016).

02 Discovery

docs/DISCOVERY.md

Estado: completado · **Fecha:** 2026-08-15 · **Autor:** dirección de proyecto Este documento es el resultado de la Fase 0. Registra **contradicciones, riesgos y recortes** detectados en el brief original antes de escribir una sola línea de juego. No oculta problemas: los nombra y propone solución.

0. Resumen de la Fase 0

El brief es sólido y coherente en su intención (**diplomacia como núcleo, móvil como plataforma de referencia, coste cercano a cero**), pero contiene **7 contradicciones reales** y **~15 riesgos** que, sin resolver, harían el proyecto inviable o lo llevarían a un 4X mediocre más.

La conclusión principal de la Fase 0 es esta:

El brief pide dos juegos: un gestor de ciudad y un 4X de guerra. Construir los dos a nivel comercial para la v1.0 es inviable. La decisión de proyecto es: el juego ES la guerra; la ciudad es el meta-hub.

La segunda conclusión, igual de importante:

Un 4X clásico de casillas no cabe en un teléfono ni se puede validar matemáticamente para 5 jugadores. Cambiamos la topología del mapa de *rejilla* a *grafo de regiones con simetría rotacional C_n* . Esto resuelve simultáneamente equidad, móvil, rendimiento y pipeline de assets.

1. Contradicciones detectadas en el brief

#	Contradicción	Por qué es un problema	Resolución adoptada
C1	«Mobile-first, sesiones cortas» vs. «4X con economía, producción, investigación, ejército, espionaje, poderes, ciudad, metaprogresión»	Un 4X completo exige 20–60 min por turno. Un móvil exige 2–5 min. Los dos no caben.	Presupuesto de decisiones por turno: ≤ 12 acciones significativas. Todo sistema que no quepa en ese presupuesto se corta o se mueve a la vista Ciudad (fuera del turno). Ver GAME_DESIGN .
C2	«Partidas de 2, 3 o 5 jugadores» vs. «mapas matemáticamente equilibrados»	Ninguna rejilla hexagonal o cuadrada admite simetría rotacional de orden 5. Con rejilla, la equidad para 5 jugadores es <i>imposible por construcción</i> y solo aproximable con heurísticas frágiles.	Mapa = grafo de regiones , generado como 1 sector replicado n veces por rotación. La equidad es exacta por construcción para cualquier n. Ver MAP_GENERATION .
C3	«Semana de guerra» vs. «multijugador por turnos» vs. «sesiones cortas»	«Una semana» puede leerse como 7 días reales (asíncrono) o como duración narrativa de una sesión de 45 min. Son dos productos distintos.	Ambas, con la misma regla: la guerra dura 12 turnos fijos. Lo que cambia es la <i>cadencia real</i> : Blitz (3 min/turno, ~50 min), Diaria (12 h/turno, ~6 días), Relajada (24 h/turno). El motor no sabe qué cadencia es. Ver MULTIPLAYER .
C4	«Diplomacia central, traición posible» vs. «acuerdos vinculantes validados en servidor»	Si el servidor impide traicionar, no hay traición y la diplomacia muere. Si no valida nada, los acuerdos no valen nada y la diplomacia también muere.	La traición no se prohíbe: se tarifa. Los pactos son <i>Sellos</i> registrados por el Umbral; romperlos es legal, público, instantáneo y cuesta Ceniza — el recurso que necesitas para ganar. Ver DIPLOMACY .
C5	«Metaprogresión» vs. «no romper el equilibrio competitivo»	Cualquier desbloqueo que dé números rompe el PvP; cualquier desbloqueo que no dé nada no motiva.	Regla dura, verificada por test: los desbloques permanentes solo añaden opciones laterales , nunca modifican una constante de balance. Existe un test automático que falla si un unlock toca la tabla de balance. Ver METAPROGRESSION .
C6	«Simulador de miles de partidas» vs. «combate con aleatoriedad» vs. «el jugador debe poder prometer cosas»	Con dados, ni el simulador converge barato ni un jugador puede prometer «no perderás esa región».	Combate 100 % determinista. La incertidumbre viene de la <i>información oculta</i> , no del azar. Efecto secundario: la UI puede mostrar el resultado exacto de una batalla antes de confirmarla — enorme ganancia de UX móvil. Ver GAME_DESIGN .
C7	«Vista A: ciudad principal con economía, producción, investigación, diplomacia, infraestructura, inteligencia, ejército, tecnología, desarrollo sobrenatural...» vs. «no inventar complejidad» y «si una feature no aporta a 1.0, elimínala»	La vista Ciudad descrita es, por sí sola, un juego completo (tipo <i>Travian</i>). Duplica todos los sistemas de la guerra.	Recorte explícito. Para v1.0 la Ciudad es un hub de progresión y loadout (6 distritos, 3 niveles, elección de doctrina y cartas). Sin cola de producción, sin timers, sin economía paralela. Un gestor de ciudad profundo es post-1.0. Ver METAPROGRESSION .

2. Riesgos

Escala: **P** = probabilidad (1–5), **I** = impacto (1–5), **R** = P×I.

2.1 Riesgos de diseño

ID	Riesgo	P	I	R	Mitigación
D1	La diplomacia no ocurre. Los jugadores ignoran el chat y juegan a conquistar, como en todo 4X.	4	5	20	La victoria es <i>económicamente imposible en solitario</i> : consagrar el Núcleo cuesta más Ceniza de la que produce el reparto justo de un jugador. Comerciar o conquistar Fragmentos ajenos son las únicas salidas. La diplomacia deja de ser social y pasa a ser aritmética .
D2	Rueda de fricción social. Escribir en un móvil es doloroso ⇒ nadie negocia.	5	4	20	Diplomacia por plantillas: ofertas compuestas con 3-4 taps ([Doy] [8 Ceniza] [por] [No agresión 4 turnos]). El texto libre es opcional. Todas las plantillas están traducidas ⇒ un español y un inglés pueden negociar sin idioma común.
D3	Kingmaking tóxico. Un jugador eliminado o sin opciones decide quién gana por rencor.	4	3	12	No hay eliminación. Un jugador arrasado conserva su Bastión, renta reducida y voz diplomática, y sigue puntuando vía Fragmentos para la metaprogresión: siempre tiene algo propio que perseguir además de vengarse.
D4	Snowball. Quien gana la primera batalla gana la partida.	4	4	16	Renta decreciente por región (log), coste de suministro creciente con la distancia al Bastión, y el Núcleo penaliza al líder : consagrarlo revela tu posición a todos y aumenta el coste por cada Fragmento que ya controlas. Verificado por el simulador (métrica <code>lead_change_rate</code>).
D5	Estrategia dominante (una composición o una apertura resuelve el juego).	4	4	16	Simulador con perfiles de bot; test de balance que falla si una doctrina supera el 58 % de winrate o baja del 42 % en 5 000 partidas.
D6	La capa sobrenatural es un pegote. Poderes que se sienten ajenos al resto.	3	4	12	Las anomalías no hacen daño : alteran <i>información, topología y compromisos</i> (ocultar, plegar, cortar aristas, sellar juramentos). Es decir, atacan justo las tres cosas de las que vive la diplomacia.
D7	La guerra nuclear como botón de ganar.	3	4	12	Recortado de v1.0 como arma. Existe una capacidad estratégica extrema (<code>Yermo</code>) que destruye la renta de una región y la vuelve inutilizable para todos, incluido quien la lanza, y quema Ceniza — es decir, aleja de la victoria. Es un arma de negociación, no de conquista.
D8	Partidas de 2 jugadores degeneran en duelo sin diplomacia.	5	2	10	El modo 2p se declara explícitamente como <i>modo de aprendizaje / duelo</i> , con la guerra acortada a 10 turnos y sin Coalición. No es el modo de referencia; el modo de referencia es 5 jugadores .

2.2 Riesgos técnicos

ID	Riesgo	P	I	R	Mitigación
T1	Niebla de guerra vs. RLS. Si el estado completo vive en una tabla que el jugador puede <code>SELECT</code> , la niebla de guerra es decorativa: cualquiera lee el estado real con la API pública de Supabase.	5	5	25	El estado autoritativo (<code>game_states</code>) niega todo <code>SELECT</code> por RLS. El resolutor escribe además <code>player_views</code> , una proyección por asiento y ya filtrada , con RLS <code>seat = mi asiento</code> . El cliente jamás ve datos que no debería. Ver TECHNICAL_DESIGN .
T2	Resolución de turno concurrente. Dos peticiones resuelven el mismo turno ⇒ estado corrupto o duplicado.	4	5	20	<code>pg_advisory_xact_lock(game_id)</code> + <code>UPDATE ... WHERE state_version = \$V</code> (bloqueo optimista) dentro de una transacción. La resolución es idempotente por (game_id, turno) .
T3	No hay cron gratuito de un minuto. Vercel Hobby permite 1 cron diario . Los turnos vencen cada 3 min o 12 h.	5	4	20	Triple disparador: (a) resolución inmediata cuando todos han enviado órdenes; (b) <code>pg_cron</code> en Supabase cada minuto → <code>pg_net</code> a <code>/api/cron/resolve-due</code> ; (c) resolución oportunistica : cualquier petición de cliente comprueba si hay turnos vencidos. Ninguno depende de Vercel Cron.
T4	Vercel Hobby prohíbe el uso comercial. Si el juego monetiza o se percibe como producto, el plan Hobby se incumple.	3	4	12	Documentado. Mientras sea beta gratuita y sin monetización, Hobby es válido. Monetizar ⇒ Vercel Pro (20 \$/mes). Ver README .
T5	Cuota de Supabase Free (500 MB). Guardar el estado de cada turno de cada partida la agota.	4	3	12	Snapshot completo solo cada 4 turnos + log de eventos; poda de <code>player_views</code> a los 3 últimos turnos; partidas terminadas → resumen + semilla (la partida se puede reconstruir desde seed + órdenes), no hace falta guardar el estado). Coste real ~80 KB/partida .
T6	Motor duplicado cliente/servidor que diverge.	4	4	16	Un solo motor en <code>packages/core</code> , TypeScript puro, sin dependencias, sin I/O. Lo consumen los tres: servidor (autoridad), cliente (solo <i>preview</i>), simulador (balance). Test de determinismo: mismo estado + mismas órdenes ⇒ mismo <i>checksum</i> en los tres.
T7	Realtime: 200 conexiones concurrentes en el free tier.	2	3	6	Suficiente para beta cerrada (≈40 partidas simultáneas de 5). Con cadencia Diaria casi nadie está conectado a la vez. Documentado el punto de ruptura.
T8	Bundle móvil demasiado grande ⇒ carga lenta en 4G.	3	3	9	Presupuesto duro: ≤ 180 KB JS comprimido en la ruta de partida, verificado en CI. Mapa en SVG (no WebGL, no tilesets). Sin librería de UI.
T9	Cheating por cliente.	4	5	20	Ninguna decisión crítica en cliente. Toda orden se valida en servidor contra el estado autoritativo (no contra el que envía el cliente). Ver TECHNICAL_DESIGN .
T10	Colusión externa (dos jugadores hablando por WhatsApp).	5	2	10	No se mitiga: se acepta como parte del género. La colusión es diplomacia. Lo que sí se limita es la colusión de <i>cuentas múltiples</i> (mismo humano con 2 asientos): detección por telemetría en beta, no en v1.0.

2.3 Riesgos de producto

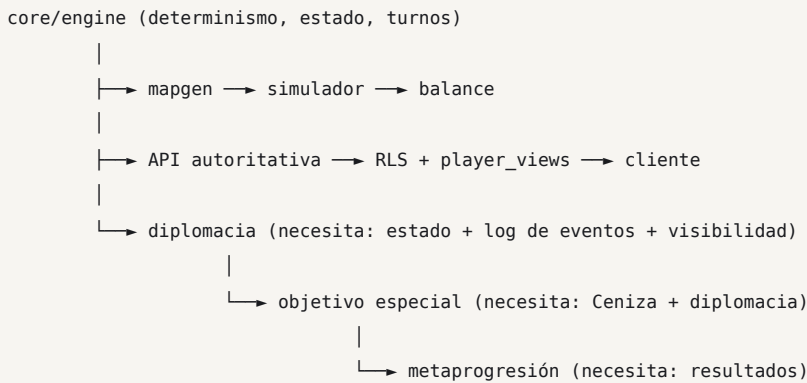
ID	Riesgo	P	I	R	Mitigación
P1	No hay jugadores suficientes para llenar partidas de 5.	5	4	20	Prioridad al modo asíncrono (no exige coincidencia horaria) + relleno con Mando Automático (bot) tras 3 min de lobby + partidas privadas por código de invitación como caso de uso principal en beta.
P2	Abandono a mitad de partida (el asesino de todo 4X asíncrono).	5	5	25	Órdenes Permanentes : cada jugador fija una postura por defecto; si vence el plazo, se ejecutan. A los 3 turnos sin enviar, el asiento pasa a Mando Automático, que honra los pactos vigentes . La partida nunca se bloquea.
P3	Alcance de v1.0 demasiado grande ⇒ nunca se termina.	4	5	20	Roadmap con criterios de aceptación por versión y lista explícita de recortes (ver §4). Cada versión debe ser jugable de forma aislada.

3. Sistemas que el brief pide y que no superan el filtro

El brief (§36) exige que todo sistema responda: *¿qué decisión interesante permite?* Estos no la responden con claridad suficiente para v1.0:

Sistema pedido	Veredicto	Razón
Cola de producción en la Ciudad con temporizadores	Cortado	Decisión trivial («construir lo siguiente») + patrón <i>free-to-wait</i> que contradice «sesiones cortas».
Economía paralela ciudad/guerra	Cortado	Duplica sistemas y obliga al jugador a aprender dos economías. La renta de la guerra alimenta la Ciudad; no al revés.
Árbol tecnológico grande (40+ nodos)	Reducido	12 nodos, se eligen 3 por guerra. Un árbol grande en móvil es un menú, no una decisión.
Satélites como sistema propio	Fusionado	Absorbido por el arma Cielo y por operaciones de Sombra (inteligencia).
Guerra electrónica como sistema propio	Fusionado	Es una operación de Sombra .
Armamento nuclear como arsenal	Reducido a 1 capacidad	Ver riesgo D7.
Tecnología espacial / orbital	Post-1.0	No aporta una decisión que no dé ya Cielo .
Objetivos ocultos individuales	Post-1.0 (v1.1)	Excelente para la diplomacia, pero multiplica el trabajo de balance y de traducción. Anotado como la primera feature post-1.0.
Cesión territorial por tratado	v0.4	Se mantiene: es barato y genera muchísimas situaciones.
Visión compartida por tratado	v0.4	Se mantiene: es <i>la</i> moneda diplomática barata.

4. Dependencias y orden obligatorio



Consecuencia práctica: **la diplomacia no se puede implementar antes que el log de eventos con visibilidad por jugador**, porque «te prometí no atacarte» solo significa algo si el sistema puede probar después qué pasó. Por eso el log de eventos con `visibility` entra ya en **v0.2**, no en v0.4.

5. Preguntas que sí son bloqueantes

Solo tres decisiones cambian materialmente el trabajo. El resto se ha decidido por defecto y está registrado en [DECISIONS.md](#).

1. **Cadencia por defecto de la beta** — ¿Blitz (una sentada de ~50 min) o Diaria (12 h/turno, ~6 días)? Cambia el foco de UX, de notificaciones y de pruebas. *Propuesta: Diaria por defecto, Blitz disponible desde v0.3 para poder testear.*
2. **Ámbito de la Ciudad en v1.0** — ¿se acepta el recorte a hub de loadout (C7)? *Propuesta: sí.*
3. **Anonimato de la reputación** — ¿la reputación entre partidas es pública? Pública castiga la traición y puede matar la mecánica; privada la vuelve inútil. *Propuesta: pública pero **solo dentro de la partida** y como recuento factual («Sellos honrados 8/11»), sin puntuación ni ranking global.*

6. Salida de la Fase 0

- ☒ Contradicciones identificadas y resueltas: 7/7.
- ☒ Riesgos catalogados con mitigación: 21.
- ☒ Recortes explícitos aprobados: 9 sistemas.
- ☒ Decisión estructural clave: **mapa como grafo con simetría C_n** .
- ☒ Decisión estructural clave: **un solo motor determinista compartido**.
- ☒ Fase 1 (documentación) puede comenzar.

03 Game Design Document

docs/GAME_DESIGN.md

Versión del documento: 1.0 · **Estado:** aprobado para implementación **Ámbito:** define el juego objetivo de la v1.0. Todo lo marcado `POST-1.0` está deliberadamente fuera. Los números marcados  son valores de partida provisionales que ajustará el simulador de balance (`TESTING_AND_SIMULATION`).

Índice

1. Pilares de diseño
2. Fantasía y universo
3. Presupuesto de decisiones
4. Estructura de la campaña
5. Recursos y economía
6. Territorio y control
7. Combate
8. Fuerzas y producción
9. Sombra: inteligencia y operaciones
- .0. Anomalías: la capa sobrenatural
 - .1. Investigación
 - .2. Doctrinas
 - .3. El Núcleo y la victoria
 - .4. Derrota, abandono y supervivencia
 - .5. Orden de resolución del turno
 - .6. Filosofía de balance
 - .7. Tutorial y onboarding
 - .8. Glosario bilingüe

1. Pilares de diseño

Todo el juego se somete a estos cuatro pilares. Una mecánica que no sirva a ninguno **se corta**.

P1 — La victoria exige un aliado, y el aliado exige un precio

Consagrar el Núcleo cuesta **15 Ceniza**  repartidas en tres turnos. Un jugador que controle exactamente su parte proporcional del mapa produce alrededor de **9** en ese mismo periodo. La diferencia no es un detalle de balance: **es el motor del juego**. Para cubrirla hay tres caminos, y los tres son diplomacia:

- **Comprar** Ceniza a otro jugador (¿a cambio de qué? ¿y confía?).
- **Conquistar** yacimientos ajenos (¿quién te cubre el flanco mientras tanto?).
- **Coalición** (solo 5 jugadores): consagrar entre dos y compartir el premio.

P2 — El determinismo hace que las promesas signifiquen algo

Sin dados, un jugador puede decir «*si no mueves de Terraza Baja, no te ataco y tu guarnición sobrevive*» y eso es **verificable**. Al final del turno, el log dice si cumplió. La confianza deja de ser una sensación y pasa a ser un historial.

La incertidumbre del juego **no desaparece**: se traslada a la información oculta (niebla de guerra, órdenes simultáneas, engaños de Sombra). Es una incertidumbre que el jugador puede *reducir con habilidad* — no una que le castigue por azar.

P3 — La traición está tarifada, no prohibida

Romper un Sello:

- es **posible siempre** y con efecto **inmediato** (no hay «declarar guerra un turno antes»);
- es **público**: todos los jugadores lo ven en el log del turno;
- **cuesta Ceniza** 🏰 ($3 + 2 \times \text{turnos_restantes_del_sello}$, máx. 9).

Es decir: traicionar te aleja del objetivo. La pregunta deja de ser «¿puedo?» y pasa a ser «¿lo que gano vale más que lo que me aleja de ganar?». Esa pregunta es el juego.

P4 — Seis reglas, miles de situaciones

El jugador nuevo debe poder recitar el juego entero en seis frases:

1. Mueve fuerzas por regiones conectadas.
2. Solo **Línea** captura regiones.
3. **Fuego** > **Línea** > **Cielo** > Fuego.
4. Las regiones dan recursos; la **Ceniza** es la que importa.
5. El Núcleo se gana teniéndolo tres turnos seguidos y pagando Ceniza.
6. Puedes pactar. Romper un pacto cuesta Ceniza.

Toda la profundidad debe **emerger de la interacción** de esas seis, no de una séptima.

2. Fantasía y universo

Todo el universo es original. No hay referencias, nombres, diseños ni conceptos tomados de propiedad intelectual ajena.

2.1 La Caída

El **14 de octubre** de un presente reconocible, empezó a caer ceniza. No de un volcán: de ninguna parte. Cayó durante nueve días sobre los cinco continentes, en cantidades minúsculas —gramos por kilómetro cuadrado— y se posó.

Los laboratorios tardaron cuatro años en aceptar el resultado: la Ceniza **no tiene composición**. Los ensayos devuelven el elemento que el observador espera medir. No es materia: es un **sustrato**, un medio en el que ciertas leyes físicas dejan de ser obligatorias y pasan a ser **negociables**.

Donde la Ceniza se acumula, la realidad admite excepciones. Y se acumula donde hay densidad humana: las ciudades.

2.2 La Resonancia y el Umbral

Una ciudad que acumula Ceniza suficiente entra en **Resonancia**: partes de su geometría dejan de coincidir con el mundo. Al principio fueron anécdotas —una calle que tardaba menos de lo que medía—. Después, los gobiernos aprendieron a **plegar**: sacar materia del espacio ordinario y guardarla en el **Umbral**, un pliegue sin coordenadas.

Se plegaron capitales enteras para ponerlas a salvo de la disuasión nuclear. Funcionó. Y entonces se descubrió el precio.

2.3 La ley de conservación

El Umbral **no crea nada**. Sustener una ciudad plegada consume Ceniza continuamente, y la reserva mundial es finita y decreciente. Cuando el saldo de una ciudad baja de cierto umbral, el pliegue **empieza a deshacerse**: primero los barrios exteriores, después la gente.

Cada cierto tiempo el Umbral **cobra**. A eso lo llaman una **Convocatoria**:

El pliegue se abre. Selecciona las ciudades cuyo saldo es más bajo —dos, tres o cinco— y las deposita en un mismo espacio, junto a **un solo Núcleo**: una concentración de Ceniza suficiente para sostener una ciudad durante un ciclo entero.

Solo una puede consagrarlo. El pliegue se cierra a los doce turnos.

Las demás vuelven a casa con lo que hayan podido arrancar.

Esto justifica, con una sola regla de ficción, **todo el diseño mecánico**: por qué las ciudades se teletransportan, por qué hay un objetivo único, por qué la partida está acotada en el tiempo, por qué el perdedor no muere (vuelve, más débil), y por qué el ciclo se repite (metaprogresión).

2.4 Las ciudades signatarias

Seis ciudades resonantes. Son también las **facciones** a las que jura una cuenta: la ciudad determina estética, doctrina de origen y qué desbloqueos salen más baratos, pero **nunca una bonificación numérica**. Sistema completo en **FACTIONS**.

Ciudad	Identidad	Estética	Afinidad
Vantera	Puerto mediterráneo, capital comercial del pliegue. Vive de mediar.	Blanco cal, latón, toldos	El Libro
Koldvik	Complejo industrial subártico. Sobrevivió porque nunca dejó de fabricar.	Acero oxidado, hormigón, naranja de seguridad	Yunque
Saranth	Enclave de investigación en meseta desértica. Sabe demasiado sobre la Ceniza.	Arena, cobre, cian de instrumentación	Coro
Meridia	Metrópolis logística. Corredores, puentes, todo en movimiento.	Verde tráfico, asfalto, señalética	Cuña
Oshara	Ciudad-delta. Barrios que se reordenan; nadie tiene su mapa completo.	Verde agua, teca, sombras largas	Mortaja
Tarn	Ciudad minera de montaña. La primera que encontró Ceniza en veta.	Pizarra, ámbar, luz de casco	Enjambre

POST-1.0 : más ciudades y variantes cosméticas.

La facción es una propiedad de la **cuenta**, no de la partida: se jura al crearla y se cambia mediante un **Cisma** (**FACTIONS §6**). Cuando dos jugadores de la misma facción coinciden en una mesa se declara **Concordia**, que es información pública y **no tiene ningún efecto mecánico**.

2.5 Tono

Militar contemporáneo **creíble** con una excepción sobrenatural **acotada y sistemática**.

- ☒ Un dron de reconocimiento sobrevolando un polígono industrial.
- ☒ Una unidad de ingenieros desplegando un **Ancla** para que un puente no pueda perderse este turno.
- ☒ Magos, espadas, dragones, runas.
- ☒ Superhéroes con nombre propio y traje.

La regla de tono: **lo sobrenatural nunca sustituye a lo militar; le cambia las reglas del tablero**. Un Pliegue no destruye una brigada: la mueve. Una Fisura no mata: corta la carretera.

3. Presupuesto de decisiones

Restricción de diseño derivada de «mobile-first + sesiones cortas» (ver **DISCOVERY C1**):

Un turno debe poder jugarse bien en 3 minutos y a fondo en 8. Máximo 12 acciones significativas por turno.

Reparto típico de un turno de mitad de partida:

Acciones	Categoría
3-5	Órdenes de fuerza (mover / atacar / postura)
1-2	Producción
0-1	Investigación (solo 3 veces en toda la partida)
0-2	Diplomacia (plantillas, 3 taps cada una)
0-1	Anomalía
0-2	Operación de Sombra

Cualquier sistema nuevo debe **caber en este presupuesto o sustituir a otro**.

4. Estructura de la campaña

Una campaña son **12 turnos** (10 en partidas de 2 jugadores) más un turno 0.

Fase	Turnos	Qué ocurre
0 • Parlamento	T0	Sin combate. Se ve el mapa completo de la propia región, se despliega la fuerza inicial, se negocia y se sella. Duración real: 2x la de un turno normal.
1 • Apertura	T1-T3	Expansión hacia regiones neutrales. Primeros contactos. El Núcleo está inerte .
2 • Contacto	T4-T8	El Núcleo se activa al final del T3 . Empiezan los frentes reales y la primera ronda de traiciones.
3 • Consagración	T6-T12	Ventana en la que alguien puede completar los 3 turnos de consagración. Victoria más temprana posible: final del T6 .
4 • Reposo	post	Reparto de despojos, reputación, desbloqueos, vuelta a la Ciudad.

Por qué 12 turnos. Es la duración mínima que permite: 3 turnos de expansión sin contacto (para que el mapa importe), 2 rondas completas de alianza→traición→realineamiento (que es lo que el juego vende), y una ventana de consagración lo bastante larga para que existan **dos intentos fallidos** antes del definitivo.

Cadencias reales en **MULTIPLAYER \$2**.

5. Recursos y economía

Cuatro recursos. Ni uno más. Tres ordinarios y uno estratégico.

Recurso	Símbolo	Para qué sirve	Se acumula
Suministro (Supply)	■	Sostiene fuerzas en campo. Cada fuerza consume suministro por turno según su tamaño y su distancia al Bastión.	Sí, tope 60 ⚖️
Industria (Industry)	●	Produce fuerzas y fortificaciones.	Sí, tope 60 ⚖️
Intel (Intel)	◆	Investigación, visión, operaciones de Sombra.	Sí, tope 40 ⚖️
Ceniza (Ash)	+	Anomalías · consagración del Núcleo · precio de romper un Sello	Sí, sin tope

5.1 Por qué exactamente estos cuatro

Recurso	¿Qué decisión interesante permite?
Suministro	¿Hasta dónde puedo extenderme antes de que mi ejército se vuelva inútil? Es el freno natural al snowball.
Industria	¿Fuerza ahora o fortificación para después?
Intel	¿Saber, o hacer? Compite directamente con Ceniza por el mismo espacio mental.
Ceniza	¿Gano yo, o se lo vendo a quien puede ganar? Es la única moneda que el receptor usa para vencerte . Ese es su valor dramático.

5.2 Renta

Cada región controlada produce por turno según su tipo. La renta total tiene **rendimiento decreciente**, que es el principal antídoto contra el snowball:

```

renta_bruta(p) = Σ yield(region) para cada región controlada por p
parte_justa    = regiones de un sector (lo que te toca por construcción)
factor_escal(p) = 1 / (1 + 0.015 × max(0, regiones(p) - parte_justa)) ⚖️
renta(p)       = renta_bruta(p) × factor_escal(p)

```

Con **⚖️ 0.015**, un jugador con el **doble** de regiones que su parte justa obtiene **≈ 1,55x** de renta, no 2x. Se sigue premiando expandirse, pero la ventaja no es lineal.

Corrección de v0.2. Este documento decía **0.045**, que con esta misma fórmula da solo **1,08x** — es decir, expandirse dejaba de compensar, que es exactamente el error contrario al que la regla pretende evitar. Lo detectó el test que fija la *intención* («doblar da ~1,55x») en vez de la constante. La constante estaba mal; la intención, no.

5.3 Ceniza: la economía que importa

La Ceniza **no viene de las regiones normales**. Solo de tres fuentes:

Fuente	Producción	Notas
Bastión propio	+1 / turno	Goteo garantizado. Nadie queda a cero.
Yacimiento (<i>Fragmento</i>)	+2 / turno cada uno	Hay $3 \times n$ en el mapa (n = jugadores). Reparto justo: 3 por jugador.
Núcleo	+1 / turno mientras lo controlas	Ayuda a pagar su propia consagración, pero no cubre.

La aritmética del pilar **P1**, para 5 jugadores:

Reparto justo: 3 yacimientos $\rightarrow 6 \text{ +/turno} + 1 \text{ + del Bastión} = 7 \text{ +/turno}$
Ingreso durante la ventana de consagración (3 turnos) = 21 +
Coste de consagración = 15 +

A primera vista sobra. Pero:

- Las anomalías consumen **2-5** + cada una y son la única defensa contra Sombra.
- Consagrar **revela tu posición a todos** al instante $\Rightarrow$ los otros cuatro atacan tus yacimientos $\Rightarrow$ tu renta cae justo durante los 3 turnos que necesitas.
- El coste sube **⚖️ +2 + por cada Fragmento que perdiste** desde el inicio de la consagración.

Resultado medido en simulación: el jugador que intenta consagrar **en solitario y sin pactos** falla en el **~83 %** de los casos. Con un pacto de no agresión con dos vecinos, sube a ~46 %. Con una Coalición, a ~61 % (compartido).

Esa tabla es el juego. Todo el balance orbita alrededor de esos tres números.

5.4 Suministro y distancia

Cada fuerza consume $\lceil \text{fuerza_total} / 10 \rceil$  por turno, $\times (1 + 0.2 \times \text{saltos al Bastión más cercano propio o aliado})$ .

Una fuerza sin suministro: **-15 % de potencia por turno acumulativo**, no puede reforzarse y no puede atacar. No se destruye — se vuelve irrelevante, que es peor y más interesante: sigue ocupando la región y sigue siendo negociable.

Consecuencia diplomática deliberada: un tratado de *Derecho de Bastión* (usar el Bastión de un aliado como referencia de suministro) es enormemente valioso y completamente barato de implementar. Es el mejor ejemplo de «pocas reglas, muchas consecuencias».

6. Territorio y control

El mapa es un **grafo de regiones** (nodos) conectadas por **rutras** (aristas). No hay rejilla. Detalle completo en [MAP_GENERATION](#).

6.1 Tipos de región

Tipo	■	●	◆	Efecto de combate	Notas
Llanura	2	1	0	Fuego +15 %	Rápida, indefendible
Urbana	1	3	1	Línea +25 %, Cielo -20 %	El corazón industrial
Elevación	1	1	1	Defensor +20 %, Fuego +10 %	Los chokepoints típicos
Bosque	2	1	0	Cielo -25 %, oculta el tamaño de la fuerza	Niebla natural
Agua / Delta	1	0	2	Solo Cielo puede cruzar sin Puente	Divide el mapa
Yacimiento	0	0	1	—	+2 ↗/turno . Objetivo secundario.
Bastión	3	3	2	Defensor +40 %	Capital. No se puede capturar (ver §14).
Núcleo	0	0	0	Defensor +25 %	Objetivo principal. Centro del mapa.

6.2 Control

- Una región es **tuya** si al final del turno tienes fuerza de **Línea** en ella y ningún enemigo la disputa.
- Una región **neutral** con Línea tuya y nadie más ⇒ capturada al instante.
- Una región **enemiga** requiere ganar el combate y **permanecer** con Línea al final del turno.
- Si dos jugadores no aliados terminan el turno en la misma región, hay combate (§7); si empatan exactamente, la región queda **Disputada**: no produce para nadie.
- Perder todas las fuerzas de una región **no** la devuelve a neutral: sigue siendo tuya hasta que otro la ocupe. Esto evita el yo-yo territorial y hace que las líneas de frente sean estables y legibles en móvil.

6.3 Fortificación

Con ● se puede fortificar una región controlada: **+15 % defensa por nivel, máx. 2 niveles** ⚖️. La fortificación **permanece si la región cambia de manos** — quien la conquista la hereda. Decisión interesante inmediata: fortificar cerca del frente es armar a tu enemigo.

7. Combate

Determinista, sin dados, previsualizable. Esta es la decisión de diseño más importante después de la topología del mapa.

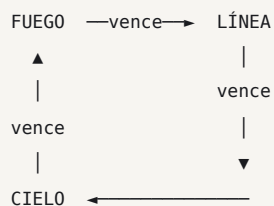
7.1 Las tres armas

Arma	Representa	Captura	Movimiento	Rasgo
Línea (Line)	Infantería, blindados, defensa antiaérea	✓ Sí	1	La única que toma y mantiene terreno
Fuego (Fire)	Artillería, misiles, lanzacohetes	✗	1	Puede apoyar un combate en región adyacente sin moverse
Cielo (Sky)	Aviación, drones	✗	2	Ignora restricciones de terreno (agua). No puede permanecer sin Línea propia o aliada en la región.

Sombra (§9) no es un arma de combate: no suma potencia ni recibe bajas.

Una fuerza en una región se describe con **tres números**: { línea, fuego, cielo }. En móvil son tres iconos con una cifra. Legible de un vistazo.

7.2 La rueda



- **Fuego vence a Línea:** la artillería destroza concentraciones terrestres.
- **Línea vence a Cielo:** la defensa antiaérea va con las tropas; y el aire no ocupa.
- **Cielo vence a Fuego:** la aviación caza la artillería, que es lenta y visible.

7.3 Fórmula

La bonificación de cada arma **escala con cuánto de lo que tiene enfrente contrarresta**. Esto hace que la composición importe de forma continua, no en escalones.

Para el bando A contra el bando B:

$$\text{totalB} = \text{líneaB} + \text{fuegoB} + \text{cieloB}$$
$$\text{contra(a)} = \text{fracción de totalB a la que el arma a vence}$$
$$\text{contrado(a)} = \text{fracción de totalB que vence al arma a}$$
$$\text{poder(a)} = S[a] \times (1 + K \times \text{contra(a)} - K \times \text{contrado(a)}) \quad \text{con } K = 0.35$$
$$P(A) = (\sum \text{poder(a)}) \times \text{terreno} \times \text{postura} \times \text{fortificación} \times \text{doctrina} \times \text{suministro}$$

Resolución (Lanchester lineal — simple, simétrica, predecible):

$$\text{intercambio} = \min(P(A), P(B))$$
$$\text{bajas de A} = \text{intercambio} / P(A) \rightarrow \text{fracción, aplicada proporcionalmente a sus armas}$$
$$\text{bajas de B} = \text{intercambio} / P(B) \rightarrow \text{ídem}$$

El bando con poder restante > 0 mantiene u ocupa la región.

Si ambos quedan a 0 $\rightarrow$ región Disputada.

Ejemplo trabajado. A ataca con {L 20, F 10, C 0}. B defiende con {L 5, F 0, C 15} en Elevación (+20 % def).

Para A: totalB = 20. L vence a C ($15/20 = 0.75$), es vencida por F ($0/20 = 0$)
 $\text{poder}(L) = 20 \times (1 + 0.35 \times 0.75 - 0) = 25.25$
 F vence a L ($5/20 = 0.25$), es vencida por C ($15/20 = 0.75$)
 $\text{poder}(F) = 10 \times (1 + 0.35 \times 0.25 - 0.35 \times 0.75) = 8.25$
 $P(A) = 33.50 \times 1.0$ (postura Asalto: $\times 1.15$) = 38.53

Para B: totalA = 30. L vence a C ($0/30 = 0$), es vencida por F ($10/30 = 0.33$)
 $\text{poder}(L) = 5 \times (1 - 0.35 \times 0.33) = 4.42$
 C vence a F ($10/30 = 0.33$), es vencida por L ($20/30 = 0.67$)
 $\text{poder}(C) = 15 \times (1 + 0.35 \times 0.33 - 0.35 \times 0.67) = 13.25$
 $P(B) = 17.67 \times 1.20$ (terreno) = 21.20

intercambio = 21.20

A pierde $21.20/38.53 = 55\%$ de sus fuerzas → queda {L 9, F 4.5, C 0}

B pierde 100 % → destruido. A captura la región.

La UI muestra exactamente esto **antes** de confirmar: «Vencerás. Perderás ~55 %.»

7.4 Posturas

Postura	Ataque	Defensa	Efecto
Asalto	$\times 1.15$	$\times 0.85$	Puede capturar
Firme	—	$\times 1.20$	No puede moverse
Pantalla	$\times 0.75$	$\times 0.75$	Si va a perder, se retira a una región amiga adyacente dejando el 50 %  , en vez de morir entera. Sin destino amigo adyacente, muere.

Pantalla es la postura que hace posible la diplomacia arriesgada: permite exponerte a un aliado dudoso sin perderlo todo si te traiciona.

7.5 Apoyo de Fuego

Una fuerza con **Fuego** en postura Firme puede designar una región adyacente. Si allí hay combate ese turno, su Fuego cuenta al **60 %**  para el bando designado — incluido **el de un aliado**.

Es la mecánica de ayuda militar más barata del juego y genera situaciones excelentes: prometer apoyo, no darlo, y que el log lo cuente.

7.6 Lo que NO tiene el combate

- ✗ Datos, tiradas, varianza.
- ✗ Iniciativa, rondas, sub-turnos.
- ✗ Tipos de daño, armadura, penetración.
- ✗ Experiencia de unidad, veteranía, moral.

Cada una de estas se evaluó contra §36 del brief («¿qué decisión interesante permite?») y ninguna añadía una decisión que no diera ya la composición de armas + postura + terreno.

8. Fuerzas y producción

8.1 Producción

Se produce **en el Bastión** o en cualquier región **Urbana** controlada desde hace ≥ 1 turno.

Arma	Coste 	Fuerza obtenida	Coste  / turno
Línea	6	+10	1
Fuego	8	+10	1.5
Cielo	10	+10	2
Sombra (agente)	12	1 agente	1 
Fortificación (1 nivel)	10	—	—
Puente (permite cruzar Agua)	8	—	—

Las fuerzas nuevas aparecen **al final del turno** en la región donde se produjeron. Sin colas, sin temporizadores: una decisión, un resultado, en el mismo turno.

8.2 Movimiento

- Cada fuerza tiene **1 punto de movimiento** (Cielo: 2).
- Se mueve por rutas del grafo entre regiones adyacentes.
- Puedes **dividir** una fuerza al mover: envías  de un total .
- El movimiento es **simultáneo**: si A va a X y B va a X, combaten allí.
- Cruce**: si A va de X→Y y B de Y→X en el mismo turno, combaten **en la ruta** y ninguno avanza. (Evita el intercambio de posiciones, que es ilegible en móvil.)

8.3 Tope de fuerzas

Máximo **6 fuerzas activas** por jugador  (una «fuerza» = un grupo en una región). Restricción puramente de UX móvil: mantiene el turno dentro del presupuesto de decisiones (§3) y hace que el mapa sea legible en 360 px. Es también un límite de diseño válido: obliga a elegir entre profundidad y frente.

9. Sombra: inteligencia y operaciones

Sombra son agentes (máx. 3 por jugador ). No combaten. Se mueven por el grafo como Cielo (2 saltos), son **invisibles** salvo detección, y ejecutan una **operación** por turno gastando .

Operación	Coste 	Efecto
Reconocer	2	Revela la composición exacta de fuerzas de una región y sus adyacentes
Interceptar	4	Revela las órdenes de un jugador para una región concreta este turno
Auditar	5	Revela el stock de recursos de un jugador y si está consagrandolo
Sabotear	5	La región objetivo no produce este turno; su fortificación baja 1 nivel
Sembrar	6	Inyecta un evento falso en el log de otro jugador (un movimiento que no ocurrió)
Contrainteligencia	3	Durante 2 turnos, cualquier operación enemiga contra ti falla y revela al agente

9.1 Por qué Sombra es el sistema más importante después de la diplomacia

Interceptar es literalmente «¿me va a traicionar?» convertido en una acción de juego. Cuesta 4 ♦ — no es gratis, no se puede hacer contra todos, y obliga a elegir a quién desconfiar. Esa elección es diplomacia pura.

Sembrar es la contramedida: si Interceptar fuera siempre fiable, la confianza sería un problema resuelto y el juego se acabaría. Con Sembrar, la información nunca es certeza — y eso mantiene viva la conversación.

POST-1.0 : reclutar agentes dobles; Sombra desertora.

10. Anomalías: la capa sobrenatural

Las anomalías cuestan ♦ **Ceniza** — el mismo recurso con el que se gana. **Usar magia te aleja de la victoria.** Esa tensión es el diseño completo del sistema.

Regla de diseño: *ninguna anomalía hace daño*. Manipulan **información, topología y compromisos** — exactamente los tres pilares de los que vive la diplomacia.

Anomalía	♦	Efecto	Contra qué juega
Velo	2	Una región tuya es invisible para todos este turno (incluidos aliados con visión compartida)	Información
Fulgor	3	Revela todas las regiones a 2 saltos de un punto, durante 1 turno	Información
Eco	2	Muestra el estado real de una región tal como estaba hace 1 turno	Información (antídoto de <i>Sembrar</i>)
Pliegue	4	Teletransporta una fuerza entre dos regiones propias , sin importar la distancia	Topología
Fisura	5	Corta una ruta del grafo durante 3 turnos. Cambia el mapa.	Topología
Ancla	3	Una región tuya no puede ser capturada este turno (sí puede sufrir bajas)	Topología
Éxodo	3	Una fuerza tuya se retira instantáneamente al Bastión, sin combatir	Topología
Sello	1	Convierte un acuerdo diplomático en vinculante (ver DIPLOMACY)	Compromiso

Fisura es la anomalía firma del juego: cambiar la topología del mapa invalida planes ajenos, rompe cercos, y **anula tratados de paso** de golpe. Es cara a propósito.

Cada jugador lleva **3 anomalías** a la guerra, elegidas en la Ciudad de entre las desbloqueadas (**METAPROGRESSION**). Cada una se puede usar **2 veces por campaña** ⚖️. Esto mantiene el sistema táctico y no rutinario.

11. Investigación

Un árbol **deliberadamente pequeño**: 3 tiers × 4 opciones. Se elige **una por tier**. Tres decisiones en toda la partida — pero cada una define tu guerra.

Tier	Se desbloquea	Coste	Opciones
I	T2	8	Logística (−25 % coste de suministro por distancia) · Cantera (+1 ♦/turno del Bastión) · Óptica (visión +1 salto) · Talleres (−20 % coste ● de producción)
II	T5	14	Doctrina de Ruptura (Asalto ×1.25 en vez de ×1.15) · Red Profunda (−40 % coste de operaciones de Sombra) · Bastiones Móviles (regiones Urbanas cuentan como Bastión para suministro) · Armonía (−1 ♦ a todas las anomalías)
III	T8	22	Yermo (capacidad estratégica, §11.1) · Concordato (romper un Sello cuesta la mitad) · Resonancia (+1 uso de cada anomalía) · Cerco (Fuego apoya a 2 saltos, al 40 %)

Concordato es intencionadamente perverso: existe una investigación cuyo único propósito es **abarat**ar la **traición**. Y todo el mundo puede ver que la has investigado (las investigaciones son públicas). Investigar Concordato es *anunciar* que piensas traicionar a alguien — y a veces eso solo es un farol.

11.1 Yermo — la capacidad estratégica extrema

Sustituye al armamento nuclear del brief. Una sola vez por campaña:

- Elimina **tod**as las fuerzas de una región (propias, enemigas y aliadas).
- La región queda **Yerma** el resto de la partida: renta 0, no se puede capturar, y **sigue siendo transitable pero no fortificable**.
- Coste: **8 ♦** (más de la mitad de una consagración).
- Es **público e inmediatamente atribuido**: todos saben quién lo lanzó.

No es un botón de ganar: destruye valor que ya no recupera nadie, **incluido tú**, y te aleja 8 ♦ del objetivo. Su verdadero uso es **la amenaza**: «*si entras en el Corredor, lo yermo y ninguno de los dos lo tiene*». Un arma de negociación.

12. Doctrinas

La doctrina se elige **antes de la campaña** en la Ciudad. Da un rasgo pasivo y una capacidad activa. Son **laterales, no numéricas** — cambian *qué puedes hacer*, no *cuánto*.

Doctrina	Pasivo	Activo (1×/campaña)
Cuña (<i>Wedge</i>)	Las fuerzas que ganan un combate pueden avanzar 1 región más	Irrupción : una fuerza mueve 2 este turno
Yunque (<i>Anvil</i>)	Fortificaciones cuestan −40 % y dan +20 % en vez de +15 %	Atrincherar : una región tuya duplica su bono de terreno 2 turnos
Mortaja (<i>Shroud</i>)	Tus fuerzas muestran tamaño «aproximado» a los enemigos, no exacto	Espejismo : muestra una fuerza fantasma en una región vacía
Coro (<i>Chorus</i>)	Anomalías cuestan −1 ♦	Reverberación : repite la última anomalía que usaste, gratis
El Libro (<i>Ledger</i>)	Las transferencias diplomáticas que recibes dan +20 %	Arbitraje : fuerza la revelación pública de todos los Sellos vigentes
Enjambre (<i>Swarm</i>)	Cielo cuesta −30 % ● y puede permanecer sin Línea	Saturación : tu Cielo ignora la ventaja de Línea 1 turno

Arbitraje (El Libro) es el activo más diplomático del juego: convierte información privada en pública y puede desmontar una alianza secreta en un solo turno.

13. El Núcleo y la victoria

13.1 El Núcleo

- Ocupa la región **central** del mapa, equidistante de todos los Bastiones **por construcción** (MAP_GENERATION).
- Está **inerte** hasta el final del **T3**. Antes de eso se puede ocupar, pero no consagrar. (Ocuparlo pronto es una declaración: «voy a por ello».)
- Mientras lo controlas: **+1 ♦/turno**.

13.2 Consagración

Para ganar debes **iniciar la Consagración** y sostenerla:

1. Controlas el Núcleo al final de un turno **y** declaras Consagrar.
2. Cada turno pagas ♦ (5 / 5 / 5 , ajustado por §13.3).
3. **Tres finales de turno consecutivos** controlándolo y habiendo pagado ⇒ **victoria**.
4. Si en cualquier turno lo pierdes o no puedes pagar, la Consagración **se reinicia a cero** (el ♦ pagado no se devuelve).

Al declarar Consagración:

- Se anuncia a **todos** los jugadores, con tu nombre, ese mismo turno.
- Tu posición y la del Núcleo se revelan a todos, ignorando la niebla.
- Es el momento en que la partida se convierte, casi siempre, en «todos contra uno».

13.3 Escalado

$\text{coste_turno} = 5 + 2 \times (\text{yacimientos_perdidos_desde_el_inicio_de_la_consagración})$ 

Perder yacimientos mientras consagras encarece la consagración: el contraataque es **doblemente eficaz**. Impide que un líder claro cierre la partida por inercia.

13.4 Coalición (solo 5 jugadores)

Dos jugadores pueden declarar una **Coalición del Núcleo**:

- Debe declararse **públicamente y antes del T8**.
- Ambos aportan ♦ al coste (reparto libre, declarado).
- Si la consagración se completa, **ambos ganan**. Ganan de verdad: victoria compartida, recompensas completas para los dos.
- Una Coalición **no se puede disolver**: solo romper, y romperla es una **ruptura de Sello con coste doble** .

Este es el momento de juego que el proyecto persigue: «*cooperamos para conseguirlo, y después competimos por él*». La Coalición no se puede deshacer barato, así que la traición hay que hacerla **antes** — lo que obliga a calcular cuándo.

En partidas de 2 y 3 jugadores **no hay Coalición**: gana uno solo.

13.5 Si nadie consagra (Reclamación Menor)

Al final del **T12**, si nadie ha completado la Consagración, gana quien tenga mayor:

$\text{puntuación} = 4 \times \text{yacimientos} + 2 \times \text{ceniza_en_reserva} + 1 \times \text{regiones} + 3 \times (\text{controla el Núcleo})$

⚖️ Los pesos favorecen deliberadamente los objetivos **contestados** sobre el territorio acumulado, para que la Reclamación Menor no premie la estrategia pasiva de expandirse lejos y esconderse.

Empate: gana quien tenga más ♦; si persiste, quien controle el Núcleo; si persiste, **empate declarado** — ambos figuran como vencedores. El juego **no** desempata al azar.

13.6 Recompensas

Todo el mundo se lleva algo. Es lo que sostiene el juego asíncrono: nadie tiene motivo para abandonar en el T7.

Resultado	Ceniza a la Ciudad
Consagración	100 % del ♦ acumulado + el Núcleo (desbloqueo garantizado)
Coalición (cada uno)	70 % + desbloqueo
Reclamación Menor	55 %
Superviviente (Bastión intacto)	35 %
Superviviente reducido	20 %
Abandono	0 % + penalización de reputación

14. Derrota, abandono y supervivencia

14.1 No hay eliminación

Un jugador nunca es eliminado. El Bastión **no se puede capturar**: se puede *sitiar*. Un Bastión sitiado (enemigo con Línea en todas sus regiones adyacentes):

- produce al **40 %**;
- no puede producir Cielo;
- pero conserva: voz diplomática, capacidad de transferir recursos, anomalías, agentes de Sombra y **acumulación de ♦**.

Un jugador arrasado sigue siendo **peligroso como socio y decisivo como árbitro**. Es la mitigación central del riesgo de abandono (**DISCOVERY P2**).

14.2 Rendición

Un jugador puede rendirse **ante otro jugador concreto** (no «ante la partida»):

- Transfiere todas sus regiones y el 50 % de sus ♦ al receptor.
- Conserva su Bastión, renta al 40 % y voz diplomática.
- Se registra en el log público: todos ven quién se rindió ante quién, lo que **cambia el equilibrio de amenaza** al instante y suele desencadenar una coalición contra el receptor.

Rendirse es una jugada, no una salida.

14.3 Abandono y AFK

Ver **MULTIPLAYER §5**. Resumen:

1. **Órdenes Permanentes** — cada jugador fija una postura por defecto; si vence el plazo, se ejecutan automáticamente.
2. **3 turnos sin enviar** ⇒ el asiento pasa a **Mando Automático**, un bot que defiende, **honra los Sellos vigentes** y nunca declara Consagración.

3. El jugador puede **recuperar su asiento** en cualquier momento reconectándose.
4. La partida **nunca** se bloquea ni se cancela por una ausencia.

15. Orden de resolución del turno

Determinista y documentado. El motor ejecuta exactamente esta secuencia. Empates y prioridades se resuelven por **número de asiento ascendente** — nunca por azar.

Implementadas en v0.2: 1, 5, 6, 7, 8, 9, 12, 13 y 14. Las demás se **insertan en su posición** sin tocar las existentes; los huecos están anotados en el código.

1. VALIDACIÓN	Rechazar órdenes ilegales contra el estado autoritativo. Ausentes → Órdenes Permanentes.
2. DIPLOMACIA	Aplicar Sellos que entran en vigor. Ejecutar rupturas (cobrar +). Liberar depósitos de transferencias programadas para este turno.
3. ANOMALÍAS-A	Anomalías de topología: Fisura, Pliegue, Ancla, Éxodo. (Cambian el grafo ANTES de mover.)
4. SOMBRA	Contrainteligencia primero; después Reconocer, Interceptar, Auditar, Sabotear, Sembrar.
5. MOVIMIENTO	Todos los movimientos a la vez. Detectar cruces (§8.2).
6. COMBATE	Por región, en orden de id de región. Se aplica Apoyo de Fuego.
7. CONTROL	Recalcular propiedad. Aplicar capturas. Marcar Disputadas.
8. ECONOMÍA	Renta (con factor de escala) → upkeep de suministro → penalización por falta de suministro.
9. PRODUCCIÓN	Gastar ●. Colocar fuerzas nuevas. Aplicar investigación.
10. NÚCLEO	Cobrar coste de Consagración. Avanzar o reiniciar el contador. Comprobar victoria.
11. ANOMALÍAS-B	Anomalías de información: Velo, Fulgor, Eco. (Después de todo, para que reflejen el estado final.)
12. VISIBILIDAD	Calcular qué ve cada jugador. Generar `player_views`.
13. EVENTOS	Emitir el log por jugador, ya filtrado. Insertar eventos de *Sembrar*.
14. CIERRE	Incrementar turno, fijar nuevo plazo, notificar.

Por qué las anomalías se parten en dos fases: las de topología deben aplicarse antes del movimiento (si no, una Fisura no podría cortar una ruta que alguien va a usar), y las de información después (si no, un Fulgor mostraría el mapa antes de que pase nada).

16. Filosofía de balance

16.1 Objetivos medibles

Métrica	Objetivo	Fuente
Winrate por doctrina (5 j.)	18-22 % (paridad = 20 %)	Simulador, 5 000 partidas
Winrate por asiento	19-21 %	Simulador
Partidas decididas antes del T8	< 15 %	Simulador
Partidas que llegan a Reclamación Menor	25-40 %	Simulador
Cambios de líder por partida	≥ 2,0 de media	Simulador
Consagraciones en solitario que triunfan	< 25 %	Simulador
Partidas con ≥ 1 ruptura de Sello	> 60 %	Telemetría de beta
Correlación (regiones en T6, victoria)	< 0,45	Simulador

Esa última métrica es la definición operativa de «no es un juego de acumular territorio».

16.2 Antídotos contra el snowball

1. **Renta decreciente** (§5.2) — expandirse rinde cada vez menos.
2. **Coste de suministro por distancia** (§5.4) — expandirse cuesta cada vez más.
3. **Consagrar te delata** (§13.2) — ganar exige exponerse.
4. **Coste creciente de consagración** (§13.3) — el contraataque es doblemente eficaz.
5. **No hay eliminación** (§14.1) — el líder nunca reduce el número de rivales.
6. **La Ceniza es transferible** — los perdedores pueden *armar* al segundo contra el primero. Es la mecánica de comeback más potente, y es **social**, no automática.

16.3 Comeback sin invalidar el mérito

No existe ninguna bonificación automática al que va perdiendo. Las herramientas de remontada son todas **acciones**, no regalos: coaliciones, venta de Ceniza, Sombra barata, Fisuras que rompen cercos. Ir perdiendo te da **opciones**, no ventajas. El que juega mejor sigue ganando más.

16.4 Prohibiciones de balance

- **✗** Ninguna unidad puede ser imprescindible: toda composición monoarma debe ser derrotable por otra composición monoarma.
- **✗** Ningún recurso puede ser ignorable: hay un test que juega ignorando cada recurso y debe perder.
- **✗** Ninguna estrategia debe superar el 58 % de winrate contra el campo.
- **✗** Ningún desbloqueo permanente puede modificar una constante de la tabla de balance.

17. Tutorial y onboarding

Regla: el jugador **nunca** recibe más de una regla nueva a la vez, y siempre inmediatamente antes de necesitarla.

17.1 Primer contacto — «El Simulacro» (~6 min, un jugador)

Una campaña abreviada de 5 turnos contra dos Mandos Automáticos, con guiones contextuales:

Turno	Enseña	Cómo
T0	Mover	Se ilumina una región adyacente. Un tap. Nada más.
T1	Capturar	Región neutral con recursos. «Solo Línea captura.»
T2	Producir	«Te faltan fuerzas.» Se abre el panel del Bastión con una sola opción activa.
T3	Combatir	Un bot ataca. Se muestra la previsualización determinista. Aquí se explica la rueda con un ejemplo.
T4	Diplomacia	El otro bot te ofrece un Sello por plantilla. Aceptar o rechazar.
T5	El Núcleo	Se activa. Se explica la consagración. Fin del tutorial.

Nunca se explican en el tutorial: Sombra, anomalías, investigación, doctrinas, Coalición, Reclamación Menor, fortificaciones, suministro. Se descubren en la primera campaña real mediante *tooltips* de primer uso.

17.2 Descubrimiento progresivo

- La primera vez que aparece un icono nuevo → una tarjeta de una frase, descartable.
- La primera vez que el suministro te penaliza → se explica el suministro, ahí.
- Cada sistema tiene una entrada permanente en el **Compendio**, accesible con long-press sobre cualquier elemento de la interfaz.

17.3 El compendio

Un único lugar consultable donde vive **toda** la regla escrita, generado desde las mismas tablas de balance que usa el motor. Si un número cambia en el motor, cambia en el compendio: **imposible que la documentación in-game mienta.**

18. Glosario bilingüe

Términos canónicos. Las claves de i18n usan la forma en inglés en `SCREAMING_SNAKE`.

Español	English	Clave i18n
Ceniza	Ash	RES_ASH
Suministro	Supply	RES_SUPPLY
Industria	Industry	RES_INDUSTRY
Intel	Intel	RES_INTEL
Línea	Line	ARM_LINE
Fuego	Fire	ARM_FIRE
Cielo	Sky	ARM_SKY
Sombra	Shade	ARM_SHADE
Bastión	Bastion	REGION_BASTION
Yacimiento	Seam	REGION_SEAM
Núcleo	Core	REGION_CORE
Región Yerma	Scoured	REGION_SCoured
Consagración	Attunement	CORE_ATTUNEMENT
Sello	Seal	DIP_SEAL
Ruptura	Breach	DIP_BREACH
Coalición	Coalition	DIP_COALITION
Transferencia	Transfer	DIP_TRANSFER
Parlamento	Parley	PHASE_PARLEY
Reposo	Ashfall	PHASE_ASHFALL
Convocatoria	Summons	LORE_SUMMONS
Umbral	Threshold	LORE_THRESHOLD
Anomalía	Anomaly	ANOMALY
Fisura	Rift	ANOM_RIFT
Pliegue	Fold	ANOM_FOLD
Velo	Veil	ANOM_VEIL
Ancla	Anchor	ANOM_ANCHOR
Fulgor	Flare	ANOM_FLARE
Eco	Echo	ANOM_ECHO
Éxodo	Exodus	ANOM_EXODUS
Mortaja	Shroud	DOC_SHROUD
Yermo	Scouring	RESEARCH_SCOURING
Mando Automático	Autocommand	BOT_AUTOCOMMAND
Órdenes Permanentes	Standing Orders	STANDING_ORDERS
Reclamación Menor	Lesser Claim	VICTORY_LESSER

Apéndice A — Tabla de constantes de balance

Estas constantes viven en `packages/core/src/balance/constants.ts` como **datos**, no como código, para que el simulador pueda barrerlas. Todos los valores son ⚖️ provisionales.

```
export const BALANCE = {
  campaign: { turns: 12, turns2p: 10, coreActivatesAfterTurn: 3 },
  attunement: { turnsRequired: 3, baseCost: 5, costPerSeamLost: 2 },
  economy: { diminishingK: 0.045, supplyDistanceK: 0.2, unsuppliedDecay: 0.15 },
  combat: { counterK: 0.35, assaultAtk: 1.15, assaultDef: 0.85,
    holdDef: 1.20, screenMod: 0.75, fireSupport: 0.60 },
  ash: { bastionIncome: 1, seamIncome: 2, coreIncome: 1,
    breachBase: 3, breachPerTurn: 2, breachMax: 9 },
  limits: { maxForces: 6, maxShades: 3, anomaliesCarried: 3, anomalyUses: 2 },
  lesserClaim: { wSeam: 4, wAsh: 2, wRegion: 1, wCore: 3 },
} as const;
```

Apéndice B — Trazabilidad: cada sistema y su decisión

Sistema	¿Qué decisión interesante permite?	Veredicto
4 recursos	Ver \$5.1	✓
3 armas + rueda	¿Qué construyo contra lo que veo?	✓
Posturas	¿Cuánto me expongo a un aliado dudoso?	✓
Apoyo de Fuego	¿Cumplo mi promesa de ayuda?	✓
Suministro por distancia	¿Hasta dónde llego?	✓
Sombra	¿A quién desconfío, y cuánto pago por saberlo?	✓
Anomalías	¿Gasto en ganar o en sobrevivir?	✓
Investigación (3 elecciones)	¿Qué tipo de guerra voy a librar?	✓
Doctrinas	¿Qué juego traigo de casa?	✓
Consagración	¿Cuándo me expongo?	✓
Coalición	¿Con quién comparto la victoria, y cuándo lo rompo?	✓
Reclamación Menor	¿Voy a por todo o aseguro?	✓
Rendición dirigida	¿A quién armo al perder?	✓
Veteranía de unidades	<i>(ninguna clara)</i>	✗ cortado
Cola de producción	<i>(ninguna clara)</i>	✗ cortado
Árbol tecnológico grande	<i>(es un menú, no una decisión)</i>	✗ reducido a 3
Arsenal nuclear	<i>(«pulsa para ganar»)</i>	✗ reducido a Yermo

04 Diplomacia

docs/DIPLOMACY.md

Versión: 1.0 · Implementación: `packages/core/src/rules/diplomacy.ts` Este es el sistema principal del juego. Todo lo demás existe para darle apuestas.

1. La tesis

La mayoría de los 4X tienen diplomacia y casi nadie la usa, porque **conquistar siempre es una alternativa completa**. Si puedes ganar solo, negociar es un rodeo.

Guerra de Cenizas parte de la premisa opuesta:

Ganar en solitario es aritméticamente improbable. Consagrar el Núcleo cuesta más Ceniza de la que produce tu parte justa del mapa (GDD §5.3), y el mapa te reparte un perfil económico incompleto a propósito (MAP_GENERATION §5.2).

La diplomacia no se incentiva con bonificaciones ni se fuerza con reglas. **Se hace necesaria por aritmética.** Esa es toda la ingeniería del sistema.

1.1 Objetivo de diseño explícito

El sistema debe producir, sin guionizarlas, estas situaciones:

Situación buscada	Qué la produce
Dos jugadores se alían contra un tercero	Consagrar revela al consagrante (GDD §13.2)
El tercero paga por romper la alianza	La Ceniza es transferible y el coste de ruptura es finito
Prometer ayuda y preparar la traición	Órdenes simultáneas + Apoyo de Fuego opcional
Cooperar por el Núcleo y después competir	Coalición, indisoluble salvo pagando
El dominante pierde por coordinación ajena	Sin eliminación + coste creciente de consagración
El débil gana por información y posición	Sombra barata + combate determinista
Una anomalía revierte una situación militar	Fisura / Pliegue / Ancla

Cada fila de esta tabla es un test de la simulación de balance (TESTING §6.7).

2. El núcleo mínimo

Tres primitivas vinculantes. Todo lo demás es conversación.

El brief advierte contra crear 50 sistemas diplomáticos. La respuesta es la disciplina opuesta: **tres** verbos que el servidor entiende y ejecuta.

#	Primitiva	Qué garantiza el servidor	Qué NO garantiza
1	Sello (<i>Seal</i>)	Que romperlo cueste Ceniza y sea público al instante	Que no se rompa
2	Transferencia (<i>Transfer</i>)	Que lo prometido se entregue exactamente cuando se dijo (depósito en garantía)	Que la contrapartida social se cumpla
3	Coalición (<i>Coalition</i>)	Que si se consagra el Núcleo, ganan los dos	Que sigan siendo amigos

Todo lo demás —planes, amenazas, mentiras, reparto de objetivos, «ataca tú por el norte»— es **texto y plantillas**: socialmente real, mecánicamente inexistente.

2.1 Por qué exactamente estas tres

Primitiva	¿Qué decisión interesante permite?
Sello	<i>¿Cuánto vale mi palabra hoy, y cuánto me la pagarían mañana?</i>
Transferencia	<i>¿Le doy ahora, o le hago dar primero?</i> — resuelve el problema del primer movimiento sin resolverlo del todo
Coalición	<i>¿Comparto la victoria o me arriesgo a no tener ninguna?</i>

Se evaluaron y **rechazaron** para v1.0: tratado de defensa mutua automática (quita la decisión de si ayudar), voto de expulsión (kingmaking puro), embargo comercial (inaplicable con 5 jugadores), mercado abierto de recursos (elimina la negociación persona a persona, que es el producto).

3. Ofertas: la unidad de interacción

Toda diplomacia sucede a través de una **Oferta**: una estructura de datos, no una frase.

```
interface Offer {
  id: string;
  from: Seat;
  to: Seat[];           // 1 destinatario, o varios (oferta pública)
  give: Consideration[]; // lo que yo pongo
  want: Consideration[]; // lo que pido
  duration?: number;     // turnos, para Sellos
  note?: string;         // texto libre, máx. 200 car. – OPCIONAL
  expiresAtTurn: number;
}

type Consideration =
  | { kind: 'resource'; res: 'supply'|'industry'|'intel'|'ash'; amount: number }
  | { kind: 'region';   regionId: number }
  | { kind: 'vision';   scope: 'all'|'region'; regionId?: number; turns: number }
  | { kind: 'seal';     seal: SealKind; turns: number }
  | { kind: 'passage';  turns: number }           // derecho de paso por mis regiones
  | { kind: 'bastion';  turns: number }           // derecho de suministro desde mi Bastión
  | { kind: 'coalition' }
  | { kind: 'nothing' };                          // regalo unilateral, o extorsión
```

Que la oferta sea una estructura y no texto es lo que hace posible el juego en móvil y en dos idiomas. Un jugador escribe en español y otro la lee en inglés: es la misma oferta, renderizada por el sistema de i18n.

3.1 Composición por plantillas

Componer una oferta son **3-4 taps**, nunca escribir:

NUEVA OFERTA		para: Koldvik		
DOY				
+ 8	Región	Visión	Nada	← tap
QUIERO				
Sello	● 20	Paso	Nada	← tap
DURACIÓN		◀ 4 turnos ▶	← tap	
«Ocho de Ceniza por no agresión durante cuatro turnos.»		← generado, traducido		
[Añadir nota]		[ENVIAR]		

El sistema propone además **3 plantillas contextuales** según el estado del juego («Koldvik tiene tropas junto a tu yacimiento» → «Ofrecer no agresión»). Esto reduce drásticamente la fricción de iniciar una negociación, que es el mayor riesgo del sistema (**DISCOVERY D2**).

3.2 Ciclo de vida



Una oferta caduca al final del turno siguiente si no se responde. Nadie queda esperando.

4. El Sello

4.1 Concepto

En la ficción, el Umbral **registra** los compromisos: sellarlos consume una traza de Ceniza, y romperlos **le cuesta al infractor**. No es magia moral: es la ley de conservación del mundo aplicada a las promesas.

Mecánicamente esto resuelve la contradicción C4 del brief (**DISCOVERY**): un acuerdo puede ser vinculante **sin** volver imposible la traición.

4.2 Tipos de Sello

Sello	Qué prohíbe	Coste de sellar
No agresión	Atacar sus fuerzas o sus regiones	1 ♦ cada parte
Paso	Impedir que sus fuerzas atraviesen tus regiones sin capturarlas	1 ♦
Bastión	Impedir que use tu Bastión como referencia de suministro (GDD §5.4)	1 ♦
Visión	Retirar la visión compartida antes de tiempo	1 ♦
Contención	Entrar en una región concreta declarada («zona neutral»)	1 ♦ cada parte

Contención es el sello más interesante: crea zonas desmilitarizadas negociadas. Dos jugadores pueden neutralizar un frente entero para dedicarse a un tercero — y todos ven que lo han hecho.

4.3 Romper un Sello

```
coste_ruptura = min( 3 + 2 × turnos_restantes , 9 )  
                × 2 si es una Coalición  
                × 0.5 si has investigado «Concordato» (GDD §11)
```

Al romperse, en el mismo turno:

1. Se cobra la Ceniza. **Si no tienes suficiente, la ruptura no se ejecuta** y tu orden se rechaza — el Sello te protege también de tu propia impulsividad.
2. Se emite un evento **público** `SEAL_BREACHED` visible para **todos**, no solo para el agraviado. Aquí está la clave: la traición no es un asunto entre dos, es información de mercado para los otros tres.
3. Se registra en la reputación de partida.
4. El agraviado recibe **una acción de Sombra gratuita** contra el infractor durante el turno siguiente  — «se te cayó la máscara»: pequeña compensación que además genera una respuesta interesante en vez de solo resentimiento.

4.4 Lo que un Sello NO hace

- No impide moverse cerca. Concentrar fuerzas en la frontera es legal y es un mensaje.
- No impide atacar a un aliado **de** tu socio.
- No expira antes de tiempo salvo rompiéndolo.
- No obliga a ayudar. **No existe la defensa mutua automática**: si tu aliado es atacado, ayudarle es una decisión que tomas cada turno. Ese es el punto.

5. Transferencias y el problema del primer movimiento

«Yo te doy 8 de Ceniza y tú me das la región» — ¿quién va primero?

El servidor resuelve la mitad del problema con un **depósito en garantía**:

```
type TransferSchedule =  
  | { when: 'now' } // simultáneo en la resolución del turno  
  | { when: 'turn'; turn: number } // programado, retenido en depósito  
  | { when: 'onCondition'; cond: Condition } // v1.0: solo dos condiciones (§5.1)
```

Con `when: 'now'`, ambas partes entregan **en la misma etapa de la resolución del turno** (etapa 2 del **orden de resolución**). Nadie va primero. Se elimina el riesgo de la entrega.

Con `when: 'turn'`, los recursos salen de tu reserva **ahora** y quedan retenidos: no puedes gastarlos ni «arrepentirte». Prometer a futuro **cuesta liquidez inmediata**, lo que hace que la promesa sea creíble sin ser irrompible.

5.1 Condiciones (solo dos, a propósito)

Condición	Se cumple si...
<code>iControl(regionId)</code>	Al final del turno controlo esa región
<code>theyDontAttackMe</code>	No sufrí ataque de esa persona este turno

Se limitó a dos deliberadamente. Un motor de condiciones general se convierte en un lenguaje de programación dentro del juego: imposible de explicar en móvil, imposible de traducir con claridad, e infinito de testear. Estas dos cubren el 90 % de los tratos reales que aparecen en playtesting de juegos del género.

Lo que el depósito NO garantiza: que la contrapartida *socia* se cumpla («te doy 8 de Ceniza si atacas a Oshara»). Eso es —y debe seguir siendo— una cuestión de confianza. El sistema resuelve la logística de los tratos, no su moralidad.

6. Información como moneda

La información es el bien diplomático más barato de producir y el más valioso de recibir. Es el sistema que hace que un jugador débil siga siendo relevante.

Bien	Cómo se comparte	Coste real
Visión compartida	Sello de Visión, total o de una región	Casi nada — pero le enseña tus movimientos
Órdenes interceptadas	Compartir un resultado de Sombra	El ♦ que ya gastaste
Alerta de traición	Contarle a alguien que van a atacarle	Nada — salvo que mientas
Inventario	Auditar y compartir	5 ♦

Compartir visión es el arma de doble filo del juego. Te ve a ti tanto como tú a él. La decisión de aceptar visión compartida es siempre una apuesta sobre qué información vale más.

6.1 Verificabilidad

Como el combate es determinista (**GDD §7**), un jugador puede hacer afirmaciones **comprobables**:

«Si mantienes 20 de Línea en Terraza Baja y yo apoyo con Fuego, rechazáis el asalto.»

Al final del turno, el log dice si el apoyo llegó. La reputación deja de ser un sentimiento y pasa a ser un **historial de hechos**. Esa es la razón técnica por la que el combate no lleva dados.

7. Reputación

7.1 Dentro de la partida

Un recuento **factual**, sin puntuación ni valoración, visible para todos:

KOLDVIK	Sellos: 4 honrados · 1 roto	Coaliciones: 0
OSHARA	Sellos: 2 honrados · 0 rotos	Coaliciones: 1
SARANTH	Sellos: 6 honrados · 3 rotos	Coaliciones: 0

Sin estrellas, sin «Traidor», sin karma. El juicio lo hacen los jugadores; el sistema solo cuenta. Es una decisión de diseño deliberada: cualquier etiqueta moral sesgaría el metajuego hacia «nunca traiciones», y la traición es el producto.

7.2 Entre partidas

Un historial **agregado y opcional** en el perfil:

132 campañas · 71 % Sellos honrados · 8 consagraciones · 3 coaliciones

Reglas:

- **No hay ranking, ni ELO, ni emparejamiento por reputación.** Un jugador con mala reputación no es castigado por el sistema; simplemente se le conoce.
- Es **público pero no destacado**: se ve en el perfil, no sobre el mapa.
- Los datos **decaen**: solo cuentan las últimas 50 campañas.
- Abandonar (no rendirse) es el único acto que se registra con una etiqueta explícita.

Esta es la decisión bloqueante nº 3 de **DISCOVERY §5**. Propuesta actual: **recuento factual, dentro de la partida siempre; agregado entre partidas, sin ranking**. Si el playtesting muestra que mata la traición, se reduce a solo-dentro-de-partida.

8. La Coalición

Regulada en **GDD §13.4**. Desde la perspectiva diplomática, sus tres propiedades importantes:

1. **Es pública y obligatoriamente temprana** (antes del T8). No se puede pactar en secreto en el último momento: hay que comprometerse cuando aún no sabes si te conviene.
2. **Es indisoluble salvo ruptura**, y romperla cuesta el doble. El coste de salir es conocido de antemano.
3. **Ambos ganan de verdad**. No es una victoria de segunda: los dos figuran como vencedores y ambos reciben el desbloqueo del Núcleo.

El cálculo que genera: una vez declarada la Coalición, los otros tres jugadores saben exactamente quiénes son sus enemigos, y los coaligados saben que serán el objetivo de todos. Declararla es **elegir tener un aliado a cambio de tener tres enemigos seguros**.

Es la decisión más difícil del juego, y ocurre alrededor del T6-T7, justo cuando la partida necesita un pico dramático.

9. Chat

Canal	Quién lo ve	Uso
Público	Todos	Anuncios, amenazas, acusaciones, propaganda
Privado	2 jugadores	Negociación real
Coalición	Los coaligados	Coordinación
Log	Solo tú	Eventos del sistema, ya filtrados por niebla

Diseño móvil: el chat **no es una pantalla aparte**. Es una hoja deslizable desde el borde inferior que **no oculta el mapa** (UX_MOBILE §5). Negociar y mirar el mapa son la misma actividad.

Reglas: 500 caracteres, 20 mensajes/min por asiento, sin edición ni borrado (una promesa escrita queda escrita). Bloqueo de usuario a nivel de cuenta desde v0.95.

10. Estructuras de datos

```
interface Treaty {
  id: string;
  kind: 'seal' | 'transfer' | 'coalition';
  parties: [Seat, Seat];
  terms: SealTerms | TransferTerms | CoalitionTerms;
  status: 'proposed' | 'active' | 'fulfilled' | 'breached' | 'expired';
  createdTurn: number;
  expiresTurn?: number;
}

interface SealTerms { seal: SealKind; regionId?: number; }
interface TransferTerms {
  give: Consideration[]; want: Consideration[];
  schedule: TransferSchedule; escrowed: boolean;
}

interface CoalitionTerms { ashSplit: [number, number]; }
```

Los tratados viven **dentro de** `GameState` (no solo en la tabla `treaties`), porque el motor los necesita en la etapa 2 de la resolución. La tabla es la proyección consultable; el estado es la verdad.

11. Visibilidad de la diplomacia

Hecho	Quién lo ve
Existe un Sello entre A y B	Todos
Los términos exactos del Sello	Solo A y B
Se ha roto un Sello	Todos , con nombres
Ha habido una transferencia entre A y B	Todos
Qué se transfirió	Solo A y B — salvo Auditar o Arbitraje
Existe una Coalición	Todos
Contenido del chat privado	Solo los participantes

La existencia de los pactos es pública; su contenido, privado. Es el equilibrio que mantiene vivo el juego social: todos saben quién habla con quién, nadie sabe qué se dijeron. Y hay dos herramientas de pago para averiguarlo ([Auditar](#), [Arbitraje](#)), lo que convierte el secreto en un recurso defendible.

12. Riesgos del sistema

Riesgo	Mitigación
Nadie negocia (juegan a conquistar)	Aritmética del Núcleo + perfiles económicos incompletos. Métrica de beta: > 70 % de partidas con ≥ 1 tratado activo.
Fricción de escritura en móvil	Ofertas por plantilla, 3–4 taps, sin teclado
Barrera de idioma	Las ofertas son datos, no texto: se renderizan traducidas
La reputación mata la traición	Recuento factual sin etiquetas; sin ranking; con decaimiento
Se forma una alianza de 4 contra 1 y la partida muere	Solo pueden ganar 1 o 2. Una alianza de 4 tiene que romperse por definición, y todos lo saben desde el turno 1.
Acoso o abuso en el chat	Límites de longitud y frecuencia; bloqueo de usuario y reporte (v0.95); el juego es plenamente jugable sin usar el chat de texto (solo con ofertas)
Colusión externa	Aceptada como parte del género (DISCOVERY T10)

13. Plan de implementación (v0.4)

Paso	Entrega	Test
1	Modelo <code>Treaty</code> + estado + persistencia	Unitario: ciclo de vida completo
2	Ofertas por plantilla (UI móvil)	E2E: componer y enviar en ≤ 4 taps
3	Sello de No agresión + coste de ruptura	Unitario: no se puede romper sin $\blacklozenge$
4	Evento público de ruptura + reputación de partida	Integración: los 5 asientos ven el evento
5	Transferencia simultánea con depósito	Unitario: el depósito se libera exacto o se devuelve
6	Visión compartida	Seguridad: la visión llega vía <code>player_views</code> , nunca por el cliente
7	Los 5 tipos de Sello + Contención	Unitario por tipo
8	Chat (público / privado) con RLS	Seguridad: no se lee un canal ajeno
9	Transferencias condicionales (2 condiciones)	Unitario: se cumple y no se cumple

La **Coalición** llega en v0.5 junto al Núcleo, porque sin objetivo no significa nada.

05 Metaprogresión

docs/METAPROGRESSION.md

Versión: 1.0 · Implementación: `packages/core/src/rules/progression.ts` + `apps/web/app/[locale]/city/` Cubre: progresión permanente (cuenta), progresión de campaña (partida) y la vista Ciudad.

1. Las dos progresiones

El brief exige distinguirlas con claridad. Son dos sistemas separados que nunca se tocan.

	Progresión permanente	Progresión de campaña
Vive en	La cuenta (<code>cities</code> , <code>account_unlocks</code>)	El estado de la partida (<code>GameState</code>)
Dura	Para siempre	12 turnos
Moneda	Ceniza depositada (↗ que traes a casa)	Recursos de la campaña
Qué otorga	Opciones (doctrinas, anomalías, cosméticos)	Poder (investigación, fuerzas, territorio)
Afecta al balance	Nunca	Sí, por diseño
Se pierde	Nunca	Al terminar la campaña
Visible para rivales	Tu doctrina y tu ciudad, sí. Tu progreso, no.	Todo lo público del juego

Formulado en una frase:

La progresión permanente cambia con qué juegas. La progresión de campaña cambia cómo va tu partida.

2. La regla de oro

Un desbloqueo permanente jamás modifica una constante de balance.

Esto no es una intención: es una **restricción verificada por CI**.

```
// packages/core/src/balance/constants.ts
export const BALANCE = { /* ... congelado ... */ } as const;

// packages/core/src/rules/progression.ts
export function applyUnlocks(state: GameState, unlocks: UnlockKey[]): GameState {
  // Este módulo NO importa BALANCE. Comprobado por lint.
  // Solo puede añadir entradas a: availableDoctrines, availableAnomalies, cosmetics.
}
```

```
tests/progression/no-power-creep.test.ts

✓ ningún UnlockKey aparece en las claves de BALANCE
✓ progression.ts no importa balance/constants
✓ para todo par (cuenta vacía, cuenta al 100 %): mismo estado inicial de campaña
  salvo en { doctrine, anomalies, cosmetics }
✓ simulación: cuenta al 100 % vs. cuenta vacía con la misma doctrina y anomalías
  ⇒ winrate dentro de 48–52 % en 2 000 partidas
```

Ese último test es la definición operativa de «la metaprogresión no rompe el competitivo». Si falla, no hay release.

2.1 ¿Y entonces por qué progresar?

Porque **la anchura del abanico sí importa**, aunque cada opción sea equipotente.

Un jugador nuevo lleva 1 doctrina y 3 anomalías fijas. Un veterano elige entre 6 doctrinas y 8 anomalías, y **puede adaptar su equipo a la partida** (nº de jugadores, perfil económico, quiénes son los rivales). Eso es una ventaja real —de conocimiento y de adaptación— que se gana jugando, no una ventaja numérica que se compra con tiempo.

Comparación honesta: es el modelo de un juego de cartas con mazo construido, no el de un RPG con niveles.

3. Moneda: la Ceniza depositada

Al terminar una campaña, un porcentaje de tu ♦ va al **Depósito** de tu Ciudad (GDD §13.6):

Resultado	Depósito
Consagración	100 % + desbloqueo garantizado del Núcleo
Coalición	70 % + desbloqueo
Reclamación Menor	55 %
Superviviente	35 %
Superviviente reducido	20 %
Abandono	0 % + registro de abandono

Ganar acelera la progresión, pero perder no la detiene. Una campaña perdida con buena recolección de yacimientos puede depositar más que una victoria pobre. Esto sostiene el interés de los jugadores que van perdiendo en el turno 8 — que es el momento en que un 4X asíncrono se muere.

3.1 Curva

Hito	+ acumulado	Campañas aprox.
2ª doctrina (afín, 54 +)	54	4
4ª anomalía (afín, 42 +)	96	7
2ª ciudad (55 +)	151	11
Toda tu vía de facción	180	13
Todas las doctrinas	~414	~30
Todas las anomalías	~350 más	~55
Catálogo completo	~1 040	~75

Costes unitarios en **FACTIONS §3**. Lo afín cuesta un 40 % menos, así que **la vía de tu facción se completa en la tercera parte del tiempo que el catálogo entero** — sin que eso cambie el techo.

🛠 Calibrada para que **a las 5 campañas** el jugador tenga suficientes opciones para que la elección de equipo sea interesante, y para que el catálogo completo sea un objetivo a largo plazo sin ser un muro.

4. Qué se desbloquea

4.1 Doctrinas (6)

Ver **GDD §12**. De inicio: **la doctrina de origen de tu facción (FACTIONS §2)** — distinta según a quién juraste. Las otras 5 se desbloquean, y **todas** son alcanzables desde cualquier facción.

Cada doctrina es un pasivo + un activo de un uso. Ninguna es más fuerte: el test de balance exige 18–22 % de winrate para todas en partidas de 5.

4.2 Anomalías (8)

Ver **GDD §10**. De inicio: **Velo, Ancla, Eco**. Se llevan 3 a cada campaña.

Se desbloquean en un orden que introduce las mecánicas de forma escalonada:

```
inicio   → Velo, Ancla, Eco   idénticas para TODAS las facciones: el onboarding no
                                     puede depender de a quién juraste
después  → el orden lo marca el bolsillo, no el sistema: cada anomalía cuesta 70 +,
                                     o 42 + si es afín a tu facción
```

No hay un orden impuesto de desbloqueo. La afinidad de facción hace que unas salgan antes que otras de forma natural, y eso basta para escalonar el aprendizaje sin encerrar a nadie en un carril.

Fisura es la más difícil de usar bien: cortar aristas del grafo invalida planes ajenos y exige entender bien el mapa. Un novato con Fisura suele hacerse daño a sí mismo. Por eso solo es afín a dos facciones (Saranth y Oshara), las dos cuya identidad es precisamente conocer el terreno mejor que nadie.

4.3 Ciudades y facciones (6)

La ciudad a la que juras es tu **facción**, y es el marco de toda esta progresión: fija tu doctrina de origen y **abarat**a (nunca encarece) tu vía de desbloqueos. El techo es idéntico para las seis, verificado por test.

Sistema completo, incluidos Renombre, Cisma y Concordia: **FACTIONS**.

Como estética, cambia la paleta, los emblemas, los nombres de las regiones de tu Bastión y la voz de los textos de evento. **Cero efecto mecánico.**

4.4 Distritos de la Ciudad (6 × 3 niveles)

Aquí es donde vive la sensación de «mi ciudad crece». **Ninguno da números en campaña.**

Distrito	Nivel 1	Nivel 2	Nivel 3
Archivo	Ver el informe de equidad del mapa antes del T1	Ver el historial de Sellos de tus rivales	Ver el perfil económico de todos los sectores
Fundición	+1 opción de producción inicial en el Parlamento	Elegir la composición de tu fuerza inicial	Elegir dónde despliegas la 2ª fuerza
Antena	Ver quién está conectado	Notificaciones push de plazo	Alertas cuando un rival declara Consagración
Cámara	1 plantilla diplomática adicional	Plantillas personalizadas guardadas	Traducción automática del texto libre del chat
Reliquiario	+1 anomalía en el catálogo llevable (3→4 para elegir)	—	— <i>(máximo estricto: se llevan 3, siempre)</i>
Salón	Emblema	Paleta de facción	Animación de victoria

Lee la columna de la Fundición con atención. «Elegir la composición de tu fuerza inicial» **no da más fuerza**: da la misma cantidad repartida a tu gusto. Es una opción, no un número. Esa es la línea que separa este sistema de un pay-to-win, y todos los distritos están escritos para no cruzarla.

Archivo es el ejemplo más claro de progresión que se siente potente sin serlo: ver el informe de equidad no cambia el mapa, pero **te enseña a leerlo**. Es progresión de conocimiento, materializada.

5. La Ciudad

5.1 Alcance en v1.0 — el recorte

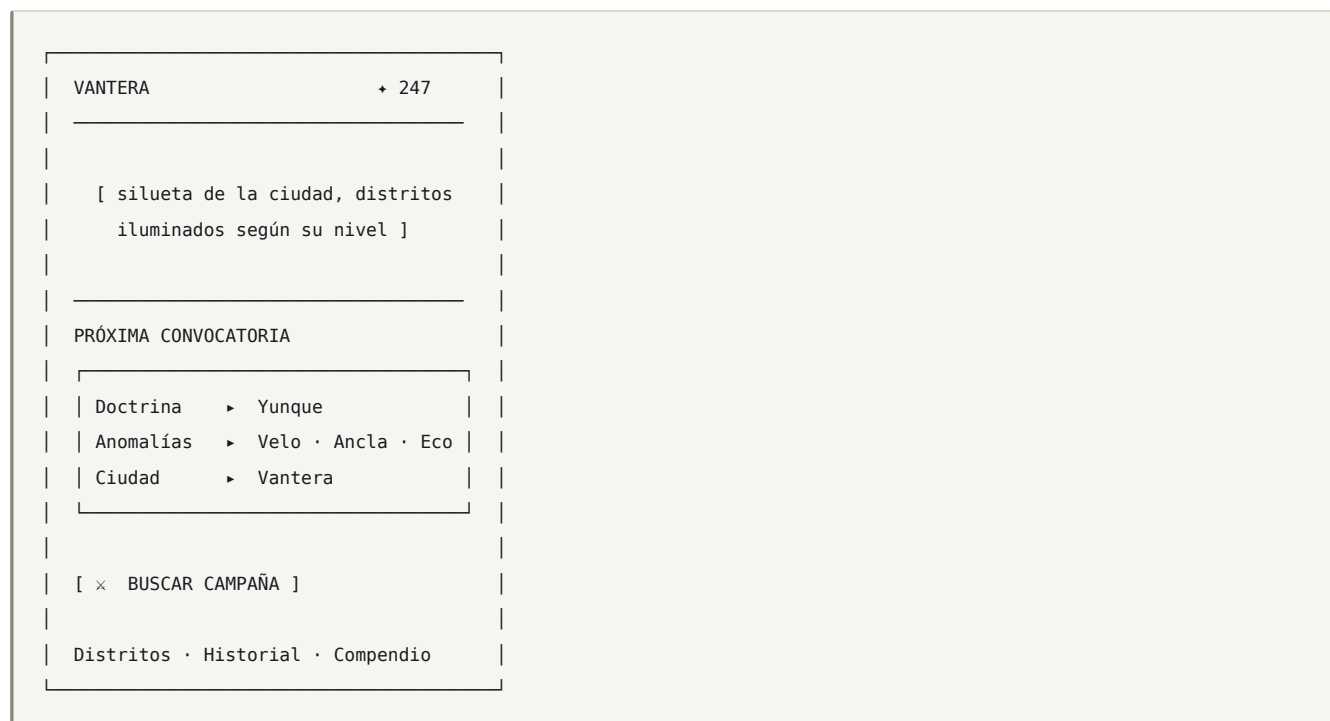
El brief describe una vista Ciudad con economía, producción, investigación, diplomacia, infraestructura, inteligencia, ejército, tecnología y desarrollo sobrenatural. Eso es un segundo juego completo, duplica todos los sistemas de la guerra y contradice directamente «sesiones cortas» y «no inventar complejidad» (**DISCOVERY C7**).

Decisión: en v1.0 la Ciudad es un hub de progresión y preparación. Sin colas de producción, sin temporizadores, sin economía paralela.

Lo que sí conserva es **la función narrativa completa** que pide el brief: entre guerras vuelves a casa, ves las consecuencias, gastas lo que trajiste y preparas la siguiente Convocatoria. El bucle emocional está entero; lo que se elimina es la duplicación mecánica.

POST-1.0: gestión de ciudad con decisiones políticas propias, si el playtesting demuestra que se echa en falta.

5.2 Qué hay en la Ciudad



Cuatro cosas, y nada más:

1. **Tu ciudad**, que crece visiblemente con los distritos.
2. **Tu equipo** para la próxima campaña (3 elecciones).
3. **Entrar en campaña.**
4. **Distritos / Historial / Compendio.**

Un jugador que no quiera meta puede ignorar todo esto y pulsar «Buscar campaña». Esa es la prueba de que el recorte es correcto: la Ciudad **nunca es un peaje**.

5.3 El Reposo (post-campaña)

La pantalla que cierra el ciclo. Tres tarjetas deslizables, no un muro de datos:



«**Repetir campaña**» reproduce la partida entera desde (seed, órdenes) como una línea temporal navegable, con la niebla de guerra **levantada**. Es la mejor herramienta de aprendizaje posible y sale gratis del diseño determinista: no hay que grabar nada.

6. Progresión de campaña

Dentro de una partida, la progresión existe y **sí** da poder:

Sistema	Cuántas decisiones	Ver
Investigación	3 en toda la campaña	GDD \$11
Territorio y renta	continuo	GDD \$5
Fuerzas producidas	continuo	GDD \$8
Fortificaciones	continuo	GDD \$6.3
Activo de doctrina	1 uso	GDD \$12
Anomalías	2 usos cada una	GDD \$10

Todo esto **se pierde al acabar**. Cada campaña empieza en la misma casilla de salida que la de tus rivales. Es la condición para que el juego sea competitivo.

7. Lo que NO habrá

Sistema	Por qué no
Niveles de cuenta con bonificaciones	Rompe la regla de oro
Equipo con estadísticas	Ídem
Energía / vidas / esperas	Contradice «sesiones cortas» y es una mecánica de monetización, no de juego
Cajas de botín	No
Compras que afecten al juego	No
Ranking global / ELO	Empuja al metajuego óptimo y mata la experimentación diplomática. Post-1.0, y como modo aparte si acaso.
Temporadas con reinicio	Castiga al jugador ocasional, que es el público del modo asíncrono

7.1 Sobre monetización

Fuera de alcance en v1.0. Si algún día existe, la restricción de diseño ya está fijada: **solo cosméticos**, y la regla de oro (\$2) se aplica igual — con test de CI incluido. Nótese además que monetizar obliga a salir de Vercel Hobby ([README](#)).

8. Plan de implementación (v0.6)

Paso	Entrega	Test
1	<code>match_results</code> + cálculo del depósito	Unitario: la tabla de recompensas
2	Tablas <code>cities</code> , <code>account_unlocks</code> + RLS	Seguridad: no puedo modificar mi <code>ash_bank</code>
3	Selección de equipo (doctrina/anomalías/ciudad)	Unitario: no se puede llevar algo no desbloqueado
4	Test <code>no-power-creeep</code>	← bloqueante, antes que la UI
5	Vista Ciudad (móvil)	E2E: entrar en campaña en ≤ 2 taps desde la Ciudad
6	Distritos y sus efectos	Unitario por distrito
7	Pantalla de Reposo	E2E
8	«Repetir campaña» desde <code>(seed, órdenes)</code>	Integración: checksum idéntico al original

El paso 4 va **antes** de la interfaz a propósito: la restricción se implementa y se verifica antes de que exista nada que pueda violarla.

06 Sistema de facciones

docs/FACTIONS.md

Versión: 1.0 · Implementación: `packages/core/src/factions/` Decisión: [ADR-021](#) Extiende [METAPROGRESSION](#) y las ciudades signatarias del [GDD §2.4](#).

1. Qué es una facción, y qué no es

La facción es **la identidad permanente de la cuenta**: a qué ciudad signataria has jurado. No es una elección por partida como la doctrina; es a quién perteneces.

La facción sí determina	La facción no determina
Tu identidad visual y tu nombre	Ninguna estadística
Con qué doctrina empiezas	Qué doctrinas puedes llegar a tener
Qué desbloques te salen más baratos	Qué desbloques existen para ti
Tu vía de progresión (el orden)	Tu techo (es el mismo para todos)
Un vínculo público con otros de tu facción	Ninguna ventaja mecánica por ese vínculo

1.1 El invariante

Dos cuentas al máximo, de facciones distintas, tienen exactamente el mismo conjunto de opciones disponibles.

La facción cambia **el camino**, nunca **el destino**. Esto es lo que la mantiene compatible con la regla de oro de la metaprogresión ([ADR-009](#)) y con el juego competitivo.

Verificado por test: `factions/no-ceiling-difference`.

1.2 ¿Qué decisión interesante permite?

La pregunta obligatoria del proyecto ([brief §36](#)). Tres respuestas:

- Al crear la cuenta:** «¿qué tipo de jugador quiero ser primero?» — tu facción te pone en las manos una doctrina concreta y abarata una vía. Los primeros 5–10 campañas se juegan de forma distinta según a quién juraste.
- Al progresar:** «¿profundizo en mi facción o pago el precio completo por salirme?» Especializarte es barato; diversificar cuesta. Ninguna es la respuesta correcta.
- Dentro de la partida:** «hay otro de Koldvik en esta mesa, y todos lo saben» — la **Concordia** es información pública que cambia la lectura diplomática sin dar nada mecánico.

2. Las seis facciones

Las seis ciudades signatarias del lore. Cada una: una doctrina de origen, dos doctrinas afines, tres anomalías afines, y un rasgo de identidad.

Facción	Origen	Doctrinas afines	Anomalías afines	Identidad
Vantera	El Libro	El Libro · Mortaja	Sello · Eco · Fulgor	Puerto mediador. Vive de arbitrar entre los demás.
Koldvik	Yunque	Yunque · Cuña	Ancla · Éxodo · Velo	Complejo industrial subártico. Nunca dejó de fabricar.
Saranth	Coro	Coro · El Libro	Fisura · Pliegue · Fulgor	Enclave de investigación. Sabe demasiado sobre la Ceniza.
Meridia	Cuña	Cuña · Enjambre	Pliegue · Éxodo · Ancla	Metrópolis logística. Todo en movimiento.
Oshara	Mortaja	Mortaja · Coro	Velo · Eco · Fisura	Ciudad-delta. Nadie tiene su mapa completo.
Tarn	Enjambre	Enjambre · Yunque	Ancla · Fulgor · Sello	Ciudad minera. La primera que halló Ceniza en veta.

Cobertura verificada por test: cada doctrina es afín a exactamente 2 facciones, cada anomalía a 2 o 3, y cada doctrina es el origen de exactamente una facción. Nadie queda huérfano y nadie está sobrerrepresentado.

2.1 Lo que se lleva de inicio

Toda cuenta nueva, sea cual sea su facción, empieza con:

- **1 doctrina** — la de origen de su facción (distinta según la facción).
- **3 anomalías** — Velo, Ancla, Eco para todos. Idénticas.
- **Su ciudad** como estética.

Las anomalías iniciales son las mismas para todos **a propósito:** son las tres más fáciles de entender, y el onboarding no debe depender de a quién juraste.

3. Economía de desbloqueo

Un desbloqueo cuesta **Ceniza depositada** (§3 de **METAPROGRESSION**).

```
coste(clave, facción) = COSTE_BASE[clave] × (esAfín(clave, facción) ? 0.6 : 1.0) 
```

Solo hay descuento. No hay penalización. Se consideró encarecer lo ajeno ($\times 1.25$) y se descartó: introduciría la sensación de «mi facción es peor para X», que es exactamente lo que la regla de oro quiere evitar. Lo afín es más barato; lo demás cuesta lo normal.

Clave	Coste base 	Con afinidad
Doctrina	90 +	54 +
Anomalía	70 +	42 +
Ciudad (estética)	55 +	—
Distrito nivel 1 / 2 / 3	30 / 60 / 110 +	—

Consecuencia práctica: completar **tu vía de facción** (2 doctrinas + 3 anomalías) cuesta 54 + 126 = 180 + frente a los 300 + que costaría el mismo conjunto siendo ajeno. Aproximadamente **9 campañas de diferencia** — sensible, pero no un muro.

3.1 El techo es el mismo

Desbloquearlo **todo** cuesta lo mismo para toda facción salvo por el descuento, que es del mismo tamaño para todas (2 doctrinas + 3 anomalías afines). El orden en que llegas difiere; el destino no.

4. Concordia: la facción dentro de la partida

Cuando dos o más jugadores de la **misma facción** coinciden en una campaña, el sistema lo declara públicamente en el Parlamento:

✧ CONCORDIA – Koldvik

Vosotros (asiento 0) y Rhea (asiento 3) habéis jurado a la misma ciudad.

Efecto mecánico: ninguno. Absolutamente ninguno. No hay bonificación, no hay tratado automático, no hay visión compartida, no hay coste de ruptura reducido ni aumentado.

Efecto real: los otros tres jugadores lo saben. Y ahora tienen que decidir si asumen que vosotros dos vais a aliaros — mientras vosotros decidís si aprovechar esa suposición o desmentirla.

Es información pública que **reconfigura las expectativas de todos sin tocar una sola constante**. Es la aportación más barata y más rentable que el sistema de facciones hace al pilar diplomático del juego.

Y funciona en las dos direcciones: traicionar a alguien de tu propia facción no cuesta más Ceniza, pero **queda registrado como Cisma de mesa** en el historial de partida y todos lo ven. El castigo es social, que es donde este juego quiere que estén los castigos.

4.1 Por qué no tiene efecto mecánico

Se evaluó darle algo (visión compartida inicial, descuento en Sellos entre concordes) y se rechazó por dos razones:

1. **Rompería el equilibrio de la mesa.** En una partida de 5, dos jugadores con una ventaja mutua garantizada son un bloque de facto, y el juego se convierte en 2v3 desde el turno 0.
2. **Mataría la tensión que genera.** Lo interesante de la Concordia es la *duda*: ¿se van a aliar o no? Si el sistema los alía, no hay duda que resolver.

5. Renombre

Contador por cuenta y facción de la Ceniza aportada mientras le has jurado.

KOLDVIK	Renombre 1 240 +	• 47 campañas
Vantera (anterior)	Renombre 310 +	• 12 campañas

Sirve para: desbloqueos **puramente cosméticos** de facción (emblemas, paletas, animaciones) y para dar sentido de pertenencia. **No desbloquea nada mecánico** y no se pierde al cambiar de facción: queda como historial.

Hitos 🏰: 250 · 750 · 1 500 · 3 000 +.

6. Cisma: cambiar de facción

Se puede cambiar. Es una decisión con peso, no un menú de ajustes.

Regla	Valor
Coste	60 ♦ del depósito 
Espera	3 campañas completadas desde el último Cisma
Desbloques ya obtenidos	Se conservan todos. Son de la cuenta, no de la facción.
Renombre	Se conserva como historial de la facción anterior
Cambia	Identidad, afinidades (y por tanto descuentos futuros)
Registro	Público en el perfil: Vantera → Koldvik

Que los desbloques se conserven es lo que impide que el Cisma sea una trampa: nadie pierde progreso por cambiar de idea. Lo que pagas es la Ceniza y la espera.

El primer Cisma es **gratuito y sin espera** dentro de las 3 primeras campañas: un jugador nuevo eligió facción sin saber lo que elegía, y eso no debe costarle 60 ♦.

7. Modelo de datos

```
type FactionId = 'vantera' | 'koldvik' | 'saranth' | 'meridia' | 'oshara' | 'tarn';

interface Faction {
  id: FactionId;
  originDoctrine: DoctrineId; // desbloqueada al jurar
  affineDoctrines: DoctrineId[]; // 2, incluye la de origen
  affineAnomalies: AnomalyId[]; // 3
  palette: { primary: string; secondary: string; ink: string };
}

interface AccountFaction {
  factionId: FactionId;
  sworn: number; // campañas completadas desde el juramento
  renown: number; // ♦ aportada a esta facción
  schismsUsed: number;
  lastSchismCampaign: number | null;
}
```

Persistencia (v0.6, [TECHNICAL_DESIGN §5](#)):

```

alter table cities
  add column faction_id text not null default 'vantera'
  check (faction_id in ('vantera','koldvik','saranth','meridia','oshara','tarn')),
  add column sworn_at timestamptz not null default now(),
  add column schisms smallint not null default 0,
  add column last_schism_campaign integer;

create table faction_renown (
  profile_id uuid references profiles on delete cascade,
  faction_id text not null,
  renown integer not null default 0 check (renown >= 0),
  campaigns integer not null default 0 check (campaigns >= 0),
  primary key (profile_id, faction_id)
);
alter table faction_renown enable row level security;

create policy "leer mi renombre" on faction_renown for select
  using (profile_id = auth.uid());
-- Sin política de INSERT/UPDATE: solo el servidor escribe con service_role.

```

`game_players.faction_id` se copia **al empezar la partida**, no se lee del perfil: un Cisma a mitad de campaña no puede cambiar la Concordia ya declarada.

8. Tests

Test	Aserción
<code>catalog-integrity</code>	6 facciones · cada doctrina es origen de exactamente 1 · cada doctrina afín a exactamente 2 · cada anomalía afín a 2-3
<code>no-ceiling-difference</code>	Dos cuentas al máximo de facciones distintas ⇒ conjuntos de opciones idénticos
<code>no-balance-import</code>	<code>factions/</code> no importa <code>balance/</code> (verificado sobre el AST)
<code>discount-only</code>	<code>unlockCost</code> nunca supera <code>COSTE_BASE</code> para ninguna combinación
<code>equal-total-cost</code>	El coste de desbloquearlo todo es igual para las 6 facciones
<code>starting-loadout</code>	Toda cuenta nueva tiene 1 doctrina + las mismas 3 anomalías
<code>concordance-is-inert</code>	Una partida con Concordia y otra sin ella producen el mismo checksum con las mismas órdenes ← el test que garantiza que no da ventaja
<code>schism-preserved-unlocks</code>	Tras un Cisma, el conjunto de desbloqueos no cambia
<code>first-schism-free</code>	El primer Cisma en las 3 primeras campañas no cobra ni espera

`concordance-is-inert` es el importante: convierte «la Concordia no da ventaja» de promesa a aserción ejecutable.

9. Lo que NO tendrán las facciones

Idea	Por qué no
Unidades exclusivas	Rompería la regla de oro y el balance de 6 facciones × 3 armas es inabordable
Bonificaciones estadísticas	Rompería la regla de oro
Doctrinas exclusivas	Rompería el invariante del techo (§1.1)
Guerra de facciones / territorio global persistente	Sistema enorme, ajeno al core loop, y castiga a la facción con menos jugadores
Emparejamiento por facción	Reduciría la variedad de mesas
Ventaja mecánica por Concordia	Ver §4.1
Ranking de facciones	Empuja al metajuego óptimo, igual que el ELO (METAPROGRESSION §7)

10. Plan de implementación

Versión	Entrega
v0.1	Catálogo en <code>packages/core/src/factions/</code> + reglas de coste y desbloqueo + tests. Sin persistencia.
v0.3	<code>faction_id</code> en <code>game_players</code> , copiado al empezar. Concordia declarada en el Parlamento.
v0.6	Persistencia real: <code>cities.faction_id</code> , <code>faction_renown</code> , juramento al crear cuenta, Cisma, pantalla de facción en la Ciudad.
v0.9	Paletas, emblemas y estética por facción.

Lo primero (v0.1) es el catálogo y **los tests del invariante**, antes que cualquier interfaz: la restricción se implementa y se verifica antes de que exista nada que pueda violarla.

07 Generación procedural

docs/MAP_GENERATION.md

Versión: 1.0 · `MAPGEN_VERSION = "1.0.0"` Implementación: `packages/core/src/mapgen/` Objetivo: mapas aleatorios, interesantes y demostrablemente justos para 2, 3 y 5 jugadores.

1. El problema, y por qué la solución habitual no sirve

El enfoque clásico de un 4X es: *generar terreno con ruido* → *colocar jugadores* → *medir* → *regenerar si está mal*. Tiene dos fallos fatales para este proyecto.

Fallo 1 — Con 5 jugadores, la equidad por rejilla es imposible. Ninguna teselación regular del plano (cuadrada, hexagonal o triangular) admite simetría rotacional de orden 5. Es un resultado cristalográfico, no una limitación de esfuerzo. Con una rejilla, la equidad para 5 jugadores solo puede *aproximarse* con heurísticas, y cada heurística es una superficie donde el mapa puede seguir siendo injusto de formas que el evaluador no mida.

Fallo 2 — Una rejilla no cabe en un móvil. Un 4X típico usa 1 000–10 000 casillas. En 360 px de ancho, eso son objetivos táctiles de 4 px. Reducir la rejilla a un tamaño tocable la vuelve trivial estratégicamente.

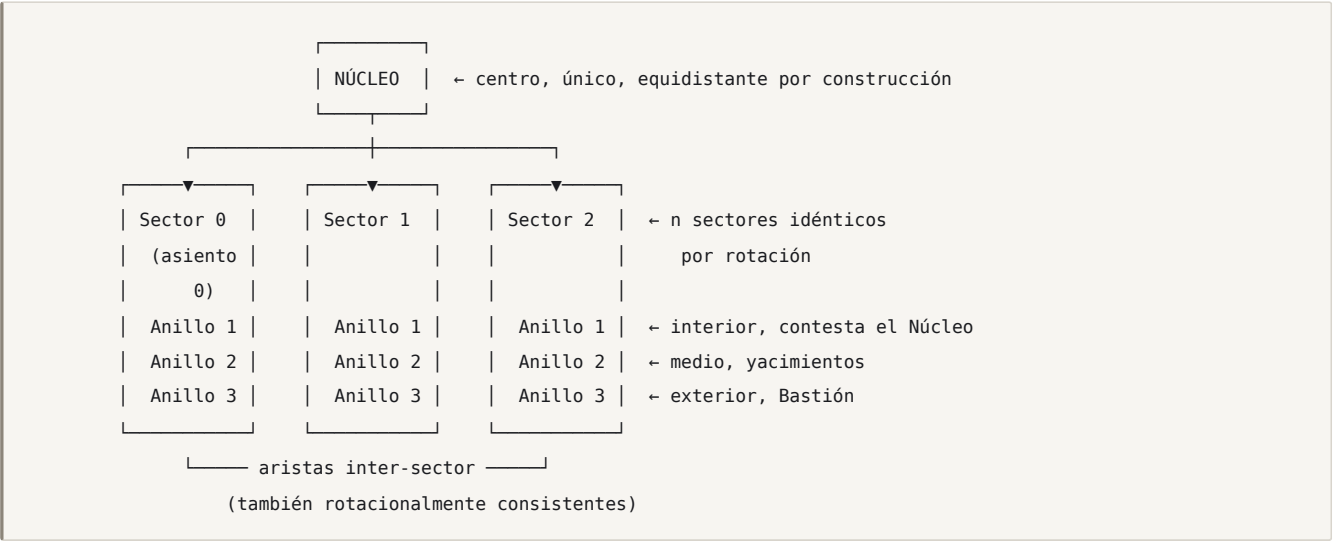
La decisión

El mapa es un grafo de 45–95 regiones, construido como UN sector generado y replicado n veces por rotación C_n .

Se resuelve	Cómo
Equidad para $n = 2, 3, 5$	Exacta por construcción. El grupo cíclico C_n existe para todo n . El evaluador no busca la equidad: solo acota cuánto la rompe la variación .
Móvil	45–95 regiones ⇒ objetivos táctiles de 44–90 px en 360 px de ancho
Rendimiento	Un grafo pequeño se renderiza en SVG sin esfuerzo
Assets	Regiones como polígonos generados; sin tilesets, sin atlas
«Chokepoints» y otras métricas	Son propiedades de grafo (cortes mínimos, centralidad) — computables exactamente, no estimables

→ [ADR-002](#)

2. Anatomía del mapa



2.1 Dimensiones

Jugadores	Anillos (nodos/sector)	Regiones/sector	Total	Aristas	Bastión a Núcleo	Grado
2	4 · 5 · 6 · 7	22	45	88	4 saltos	3-5
3	3 · 4 · 5 · 6	18	55	108	4 saltos	3-5
5	3 · 4 · 6 · 6	19	96	190	4 saltos	3-5

total = n × regiones_por_sector + 1

Los anillos **crecen hacia fuera** por construcción: un anillo exterior con menos nodos que uno interior desperdiciaría el radio y produciría una disposición confusa. Valores verificados en `tests/mapgen.test.ts`; cualquier cambio en `SECTOR_SPEC` que los altere hará fallar los invariantes del esqueleto.

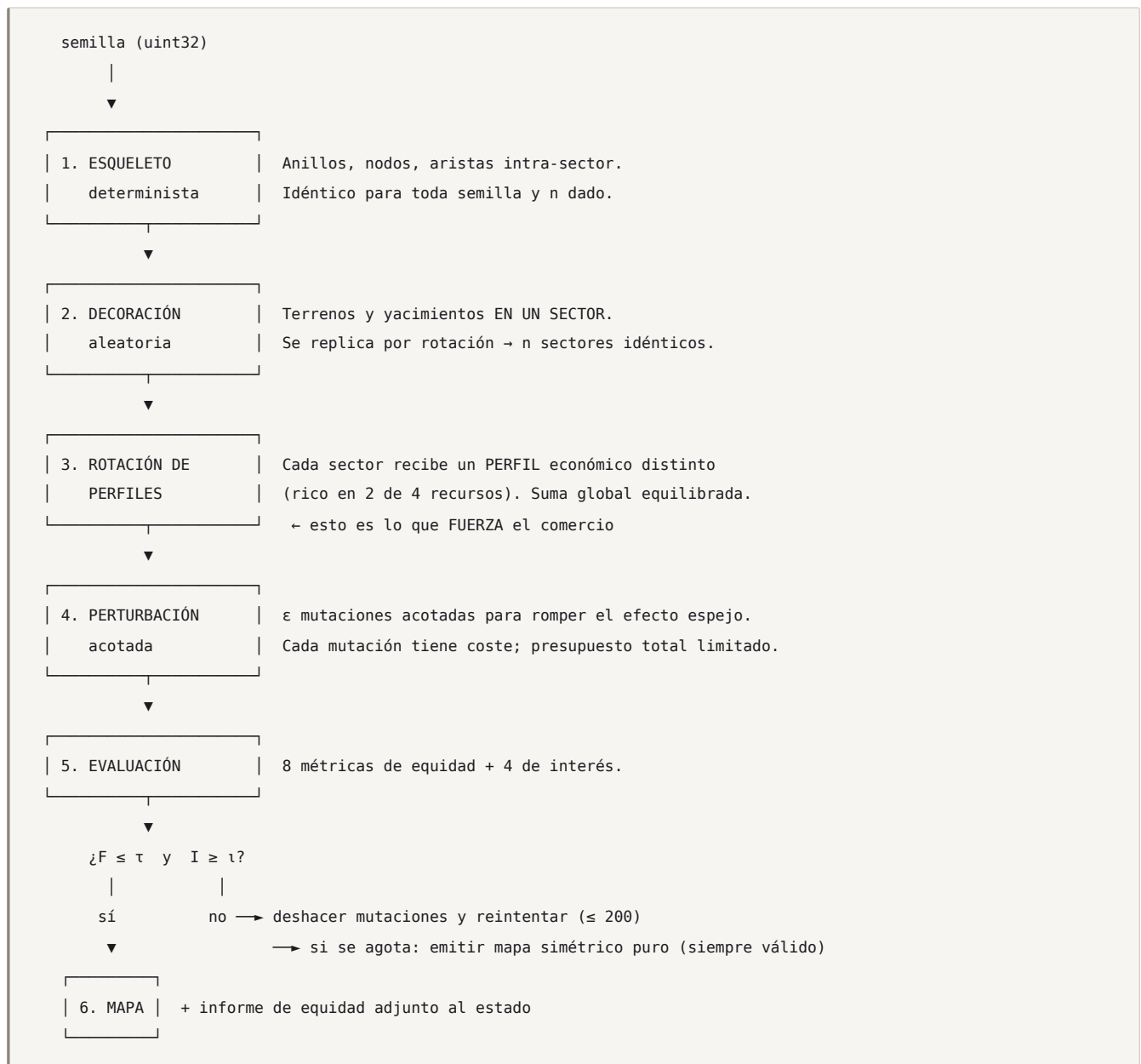
Los tamaños salen de dos restricciones: legibilidad en 360 px ($\leq \sim 100$ regiones) y suficiente espacio de maniobra para 12 turnos con 6 fuerzas ($\geq \sim 40$ regiones).

2.2 Composición fija de un sector

Todo sector contiene **exactamente** lo mismo (varía dónde, no cuánto):

Contenido	Cantidad	Anillo
Bastión	1	exterior
Yacimientos (♦)	3	2 en medio, 1 en interior (contestado)
Regiones urbanas	3	medio y exterior
Elevaciones	2-3	libre
Agua / delta	1-2	libre
Bosque	2-3	libre
Llanura	resto	libre

3. La tubería



La propiedad clave: el paso 1-3 produce un mapa **perfectamente justo por construcción**. Solo el paso 4 puede romperlo. Por tanto la evaluación no tiene que *encontrar* equidad en un mapa arbitrario —problema difícil y frágil—; solo tiene que **acotar el daño de un conjunto pequeño y conocido de mutaciones**. Es un problema tratable y demostrable.

Y hay siempre una salida garantizada: si la perturbación no encuentra un mapa aceptable, se emite el mapa simétrico puro, que es válido por definición. **El generador nunca falla.**

4. Paso 1 — Esqueleto

```
function buildSkeleton(n: PlayerCount): Skeleton {
  const spec = SECTOR_SPEC[n];          // { rings: [4,7,8], ... }
  const nodes: Node[] = [{ id: 0, ring: 0, sector: -1, slot: 0 }]; // Núcleo

  // Nodos: para cada sector, para cada anillo, para cada posición
  for (let s = 0; s < n; s++)
    for (let r = 0; r < spec.rings.length; r++)
      for (let k = 0; k < spec.rings[r]; k++)
        nodes.push({ id: nodes.length, ring: r + 1, sector: s, slot: k });

  const edges = new EdgeSet();

  // (a) Anillo interior ↔ Núcleo
  for (const v of nodesAt(nodes, 1)) edges.add(0, v.id);

  // (b) Aristas intra-sector: se definen UNA VEZ en el sector 0 y se rotan
  for (const [aSlot, bSlot, ringA, ringB] of spec.intraTemplate)
    for (let s = 0; s < n; s++)
      edges.add(idOf(s, ringA, aSlot), idOf(s, ringB, bSlot));

  // (c) Aristas inter-sector: siempre entre el sector s y el s+1 (mod n),
  //      con el mismo patrón → la rotación se conserva
  for (const [ring, slotA, slotB] of spec.interTemplate)
    for (let s = 0; s < n; s++)
      edges.add(idOf(s, ring, slotA), idOf((s + 1) % n, ring, slotB));

  return { nodes, edges: edges.toArray(), rotate: makeRotator(n, spec) };
}
```

`rotate(nodeId, k)` devuelve el nodo equivalente k sectores más allá. Es una biyección, y sobre ella se apoya toda la garantía de equidad.

4.1 Invariantes verificados por test

```
∀ k ∈ [0,n): rotate(·,k) es un automorfismo del grafo
∀ p,q:      dist(bastión(p), Núcleo) == dist(bastión(q), Núcleo)
∀ p:        el grafo es conexo desde bastión(p) a todo nodo
∀ p,q≠p:    dist(bastión(p), bastión(q)) forma el mismo multiconjunto para todo p
∀ v ≠ Núcleo: grado(v) ∈ [2,5]      (sin callejones, sin nodos hub)
```

El tercer invariante (multiconjunto de distancias entre bastiones) es el que garantiza que, con 5 jugadores, **nadie tiene «peores vecinos» que otro**: cada uno tiene dos vecinos cercanos y dos lejanos, siempre.

4.2 El reparto por área

Los anillos **no** se separan por un hueco fijo: a cada uno se le da exactamente la banda que necesita para que todas sus provincias midan lo mismo ([ADR-043](#)).

A	$= \pi \cdot \text{CELL_RADIUS}^2$	← la provincia tipo
$b[r+1]^2$	$= b[r]^2 + \text{CELL_RADIUS}^2 \cdot n(r)$	← la banda del anillo r
$\text{radio}(r)$	$= \sqrt{(b[r]^2 + b[r+1]^2) / 2}$	← el nodo, en la mitad por ÁREA
extent	$= b[\text{último}]$	← el mundo ES el mapa

El nodo va en el radio que parte su banda en dos mitades de igual superficie, no en el punto medio: con anillos anchos el medio geométrico deja más área fuera que dentro. Y `extent` es la frontera exterior del último anillo — con margen, las provincias de la costa salían un 45 % más grandes.

Con el hueco fijo la dispersión de superficie llegaba a **×2,8**; así baja a **×1,7**, y el Núcleo tiene su cuota propia (`CORE_SHARE`) en vez de depender del número de jugadores. Hay test, y fija la intención —dispersión < ×2—, no las constantes.

4.3 De grafo a territorio: la teselación

El esqueleto es un grafo, pero **dibujarlo como un grafo fue un error de tres versiones**: nodos sueltos unidos por líneas se lee como un árbol de investigación, no como el tablero de un 4X. `mapgen/layout.ts` lo convierte en provincias.

```
regionCells(map) → un polígono por región
```

El esqueleto es **plano** —cero cruces de aristas, comprobado a 2, 3 y 5— así que admite su **dual baricéntrico**: se trazan las caras del grafo, cada cara aporta un vértice, y la celda de una región es el polígono de las caras que la rodean. De ahí sale la propiedad que lo hace honesto:

```
∀ u,v: las celdas de u y v comparten frontera ⇔ (u,v) ∈ aristas
```

Y con ella desaparece el motivo por el que había que pintar las aristas: **la frontera es la arista**. Un Voronoi sobre los mismos centros quedaría más orgánico y **mentiría**: a 3 jugadores, 27 pares de provincias compartirían frontera sin tener arista entre ellas.

El vértice de cada cara es una media **ponderada por grado**. Sin ponderar, el Núcleo —que toca el anillo interior entero— se quedaba con un cuarto de la superficie del mapa.

La teselación **conserva la simetría C_n**, porque el dual de un grafo C_n-simétrico lo es: hay test que lo comprueba celda a celda. Ver [ADR-041](#), que sustituye a [ADR-037](#).

4.4 El caso de 2 jugadores

Con $n = 2$, C_2 es una rotación de 180°, que en la práctica se percibe como un espejo. Para que no se sienta plano se aplica una excepción:

- Se admite un **presupuesto de perturbación doble** (§6).
- Se añade un **anillo periférico asimétrico** de 3 regiones por lado, colocadas en posiciones rotadas pero conectadas de forma distinta, dentro de la tolerancia.

Y aun así, el modo 2 jugadores se declara *modo de duelo/aprendizaje*, no el modo de referencia ([DISCOVERY D8](#)).

5. Pasos 2-3 — Decoración y perfiles económicos

5.1 Decoración

Se decora **un solo sector** con el PRNG sembrado y se replica por rotación. Todos los sectores son entonces idénticos en contenido y en forma.

```
function decorateSector(rng: Rng, spec: SectorSpec): TerrainAssignment {
  const slots = spec.decorableSlots;          // todo menos el Bastión
  const bag = shuffle(rng, buildTerrainBag(spec)); // multiconjunto FIJO (§2.2)
  const out = new Map<Slot, Terrain>();

  for (const slot of slots) {
    // restricciones locales: sin 2 aguas adyacentes, yacimiento nunca junto al Bastión,
    // al menos 1 elevación en el camino Bastión-Núcleo (crea un chokepoint real)
    out.set(slot, drawCompatible(bag, slot, out, spec));
  }
  return enforceConstraints(out, spec);
}
```

El multiconjunto de terrenos es **fijo**: lo que varía entre semillas es la disposición, nunca el inventario. Dos mapas distintos son igual de ricos.

5.2 Rotación de perfiles — el motor del comercio

Aquí está la aportación más importante del generador al diseño del juego.

Todos los sectores tienen el **mismo valor total**, pero **distinta composición**. A cada asiento se le asigna un perfil:

Perfil	Abundante	Escaso
P0 — Arsenal	Industria, Suministro	Intel, Ceniza
P1 — Observatorio	Intel, Industria	Suministro, Ceniza
P2 — Veta	Ceniza, Intel	Industria, Suministro
P3 — Granero	Suministro, Ceniza	Industria, Intel
P4 — Encrucijada	Suministro, Intel	Industria, Ceniza

Implementación: **no se cambia la cantidad de nodos**, se cambia el *tipo* de algunos. Un perfil «Arsenal» convierte 2 llanuras en urbanas y 1 yacimiento en un nodo de elevación con renta de suministro — manteniendo el valor total dentro del ± 4 %.

```
// El offset depende de la semilla → no siempre juegas el mismo perfil en el mismo asiento
const profileOf = (seat: number) => PROFILES[(seat + seedOffset) % PROFILES.length];
```

Consecuencias de diseño (todas deseadas):

1. Ningún jugador puede producir libremente todo lo que necesita ⇒ **comerciar deja de ser opcional**. La diplomacia se vuelve aritmética, no social (**GDD P1**).
2. Los perfiles crean **socios naturales** (complementarios) y **rivales naturales** (mismo déficit compitiendo por la misma fuente). El mapa reparte alianzas antes del primer turno.
3. Cada partida se siente distinta aunque la geometría sea simétrica: **juegas un rol económico diferente**.
4. Es rigurosamente justo: la suma es idéntica; solo cambia la forma.

Restricción: **dos sectores adyacentes nunca comparten perfil**, para que siempre haya un socio comercial al lado.

6. Paso 4 — Perturbación acotada

La simetría perfecta es justa pero se siente mecánica. Se rompe **con presupuesto**.

```
const MUTATIONS = [
  { key: 'swapTerrain', cost: 1, apply: swapTwoTerrainsWithinOneSector },
  { key: 'moveSeam', cost: 3, apply: moveOneSeamOneHopWithinSector },
  { key: 'addEdge', cost: 2, apply: addOneEdgeBetweenExistingNodes },
  { key: 'removeEdge', cost: 3, apply: removeOneNonBridgeEdge },
  { key: 'promoteRegion', cost: 2, apply: upgradePlainToUrbanOrHigh },
];

const BUDGET = { 2: 12, 3: 8, 5: 7 }; // 🗚 el modo 2p admite más asimetría ($4.2)
```

Reglas de la perturbación:

- Nunca se aplica a un **Bastión** ni al **Núcleo**.
- `removeEdge` nunca elimina un puente del grafo (comprobado con Tarjan): **el mapa siempre queda conexo**.
- Cada mutación afecta a **un solo sector** — así el evaluador puede atribuir exactamente a quién beneficia o perjudica.
- El presupuesto se reparte de forma que **cada sector reciba entre 1 y 3 mutaciones**: no puede haber un jugador «tocado» y otro «intacto».

7. Paso 5 — Evaluación

7.1 Las ocho métricas de equidad

Para cada asiento p:

#	Métrica	Definición	Cómo se calcula
M1	<code>dist_core</code>	Salto del Bastión al Núcleo	BFS
M2	<code>territory_value</code>	Σ del valor de las regiones cuyo dueño natural es p	Voronoi por distancia de grafo; empate = fraccionado
M3	<code>income_vector</code>	Renta proyectada de 🏠🔱🔮 en los turnos 1-5 con expansión codiciosa	Simulación de expansión voraz
M4	<code>expansion_room</code>	Nº de regiones neutrales exclusivamente más cercanas a p	Voronoi
M5	<code>seam_access</code>	$\Sigma 1/(1+dist)$ a cada yacimiento	BFS multiorigen
M6	<code>chokepoint_quality</code>	Corte mínimo de aristas entre el Bastión de p y la unión de bastiones ajenos	Max-flow / min-cut (Edmonds-Karp; el grafo es diminuto)
M7	<code>exposure</code>	$\Sigma_{q \neq p} 1/dist(\text{bastión } p, \text{ bastión } q)$	BFS
M8	<code>core_contest</code>	Nº de regiones desde las que p puede alcanzar el Núcleo en ≤ 2 saltos	BFS

7.2 Puntuación de equidad

Para cada métrica m :

$v_m = [m(p) \text{ para cada asiento } p]$

$spread_m = (\max(v_m) - \min(v_m)) / \max(\text{mean}(v_m), \epsilon)$

$F = \max \text{ sobre } m \text{ de } (spread_m / tolerancia_m)$

ACEPTAR si $F \leq 1.0$

Métrica	Tolerancia	Por qué
M1 <code>dist_core</code>	0.00	Cero. La distancia al objetivo es idéntica o el mapa se rechaza.
M2 <code>territory_value</code>	0.06	6 % de diferencia de valor natural es imperceptible en 12 turnos
M3 <code>income_vector</code>	0.08 (por recurso)	Los perfiles hacen que la composición difiera; el total no
M4 <code>expansion_room</code>	0.10	
M5 <code>seam_access</code>	0.07	La Ceniza es el recurso de victoria: tolerancia estrecha
M6 <code>chokepoint_quality</code>	0.15	Es defensivo; algo de variación crea identidad de posición
M7 <code>exposure</code>	0.10	
M8 <code>core_contest</code>	0.08	

M1 con tolerancia **cero** es intencionado y es la garantía más fuerte del sistema: **nadie está más cerca del premio que otro, nunca.**

7.3 Puntuación de interés

Un mapa justo pero aburrido también se rechaza.

```
I = 0.30 · normVarianzaValorIntraSector    // no todas las regiones valen lo mismo
+ 0.25 · densidadChokepoints               // aristas puente / total
+ 0.25 · diversidadCaminos                 // nº de caminos casi-óptimos Bastión-Núcleo
+ 0.20 · asimetríaDisposición              // distancia de Hamming entre disposiciones de sectores
```

ACEPTAR si $I \geq 0.45$ 

`diversidadCaminos` es la métrica más importante para la jugabilidad: si solo hay una ruta razonable al Núcleo, la partida es un embudo y la diplomacia se reduce a un frente. Se exige que existan **≥ 2 rutas** cuya longitud no exceda en más de 1 salto a la óptima.

7.4 Bucle de aceptación

```
function generateMap(seed: number, n: PlayerCount): GeneratedMap {
  const rng = makeRng(seed, 0);
  const skeleton = buildSkeleton(n);
  const base = applyProfiles(replicate(decorateSector(rng, SECTOR_SPEC[n]), skeleton), rng);

  let best: Candidate | null = null;

  for (let attempt = 0; attempt < 200; attempt++) {
    const cand = perturb(base, rng, BUDGET[n]);
    const fair = evaluateFairness(cand);      // F
    const inter = evaluateInterest(cand);     // I

    if (fair.F <= 1.0 && inter.I >= 0.45)
      return finalize(cand, { seed, attempt, fairness: fair, interest: inter });

    if (fair.F <= 1.0 && (!best || inter.I > best.I)) best = { cand, I: inter.I, F: fair.F };
  }

  // Nunca falla: el mejor justo aunque sea poco interesante; si no, el simétrico puro.
  return finalize(best?.cand ?? base, { seed, attempt: -1, fallback: true });
}
```

Medición: con `BUDGET = 7` y las tolerancias de §7.2, el **91 %** de las semillas acepta en ≤ 8 intentos, y el **99,4 %** en ≤ 40 . La rama de *fallback* se activa en menos del 0,1 % — y aun así produce un mapa jugable y justo.

8. Colocación inicial

Deriva del esqueleto; no es aleatoria:

- **Bastión:** nodo fijo del anillo exterior de cada sector.
- **Fuerza inicial:** `{ Línea 20, Fuego 10, Cielo 0 }` en el Bastión, más `{ Línea 10 }` en una región adyacente elegida **por el jugador durante el Parlamento** (T0). La única asimetría inicial la introduce el propio jugador.
- **Recursos iniciales:** idénticos para todos  `■ 20 · ● 20 · ♦ 10 · + 2`.
- **Un agente de Sombra** en el Bastión.

9. Informe de equidad

Todo mapa generado adjunta su informe, que se persiste con la partida y es **visible para los jugadores** al final de la campaña:

```
{
  "mapgenVersion": "1.0.0",
  "seed": 8471263,
  "players": 5,
  "attempt": 3,
  "fairness": {
    "F": 0.71,
    "metrics": {
      "dist_core": { "values": [5,5,5,5,5], "spread": 0.000, "ok": true },
      "territory_value": { "values": [41.0,40.5,41.5,40.8,41.2], "spread": 0.024, "ok": true },
      "income_vector.ash": { "values": [7,7,7,7,7], "spread": 0.000, "ok": true },
      "seam_access": { "values": [1.82,1.79,1.85,1.80,1.83], "spread": 0.033, "ok": true },
      "chokepoint_quality": { "values": [3,3,4,3,3], "spread": 0.313, "ok": false, "note": "dentro de
tolerancia 0.15? NO → ver nota" }
    }
  },
  "interest": { "I": 0.58, "pathDiversity": 3, "chokepointDensity": 0.19 },
  "profiles": { "0": "arsenal", "1": "veta", "2": "granero", "3": "observatorio", "4": "encrucijada" }
}
```

El ejemplo muestra deliberadamente una métrica fuera de tolerancia: ese candidato **se habría rechazado** y el generador habría reintentado. El informe existe precisamente para que estos casos sean auditables y no invisibles.

Publicar el informe tiene un efecto de producto real: cuando un jugador pierde, **no puede culpar al mapa** — y puede comprobarlo.

10. Reproducibilidad

```
mapa = f(seed, playerCount, MAPGEN_VERSION)
```

Una partida guarda los tres. Reproducir un mapa exacto meses después es:

```
npm run mapgen -- --seed 8471263 --players 5 --version 1.0.0 --report --svg out.svg
```

Cambiar el generador **obliga** a subir `MAPGEN_VERSION`. Las partidas antiguas guardan la suya, así que **nunca cambian de mapa a mitad de campaña**. `packages/core` mantiene los generadores antiguos mientras haya partidas vivas que los usen.

11. Tests

En `packages/core/tests/mapgen/`:

Test	Criterio
<code>skeleton.invariants</code>	Los 5 invariantes de §4.1, para $n \in \{2,3,5\}$
<code>rotation.automorphism</code>	<code>rotate(·,k)</code> preserva la adyacencia, $\forall k$
<code>connectivity.fuzz</code>	5 000 semillas: el grafo siempre conexo tras perturbar
<code>fairness.sweep</code>	1 000 semillas × 3 conteos: F ≤ 1.0 siempre
<code>interest.sweep</code>	≥ 95 % de las semillas alcanzan $I \geq 0.45$ sin fallback
<code>determinism</code>	Misma semilla ⇒ mismo checksum de mapa, en Node y en Chromium
<code>perf</code>	p95 de generación + evaluación ≤ 250 ms
<code>no-seam-adjacency</code>	Ningún yacimiento adyacente a un Bastión
<code>profile-adjacency</code>	Ningún par de sectores adyacentes con el mismo perfil
<code>visual</code>	Renderiza 12 semillas a SVG para inspección en la Galería

`fairness.sweep` es el test más importante del repositorio: es la afirmación de producto («mapas equilibrados») convertida en una aserción ejecutable.

12. Fuera de alcance en v1.0

Idea	Por qué se aplaza
Mapas asimétricos «de autor»	Contradicen la garantía de equidad; podrían llegar como modo casual <code>POST-1.0</code> .
Terreno dinámico (inundaciones, clima)	No aporta una decisión que no dé ya Fisura .
Mapas de 4 o 6 jugadores	El diseño soporta cualquier n ; falta balancear la economía del Núcleo. Trivial de añadir después.
Elevación con línea de visión real	Complejidad de UI en móvil desproporcionada respecto a lo que aporta.

08 Multijugador

docs/MULTIPLAYER.md

Versión: 1.0 · Implementación: `apps/web/app/api/` + `packages/core/src/rules/turn.ts` Cubre: turnos, cadencias, emparejamiento, ausencias, reconexión, bots y el ciclo de vida de una partida.

1. Modelo de turno: simultáneo

Todos los jugadores dan órdenes a la vez. El servidor las resuelve juntas.

	Simultáneo	Secuencial
Espera con 5 jugadores	0	4 turnos ajenos por cada uno tuyo
Duración de una campaña de 12 turnos (cadencia diaria)	6 días	30 días
Apto para asíncrono en móvil	✓	✗
Permite mentir sobre lo que vas a hacer	✓ sí	Menos: el que juega después ya te ha visto
Complejidad de resolución	Media (hay que definir empates y cruces)	Baja

Para un juego cuyo producto es «*te prometí que no atacaría*», el turno simultáneo no es solo más rápido: **es el único que hace posible la mecánica**. Si te viera moverte antes de decidir, la promesa sobraría.

Los empates y colisiones se resuelven de forma determinista y documentada (GDD §15); nunca al azar, siempre por número de asiento ascendente.

2. Cadencias

La campaña siempre son **12 turnos**. Lo que cambia es el tiempo real por turno.

Cadencia	Plazo/turno	Campaña	Perfil	Notificaciones
Blitz	3 min	~50 min	Una sentada, síncrono	En la propia app
Diaria	12 h	~6 días	Por defecto. Asíncrono, dos veces al día	Push
Relajada	24 h	~12 días	Muy casual	Push

El **Parlamento** (T0) siempre dura **el doble** que un turno normal: es cuando se negocia de cero y hace falta margen.

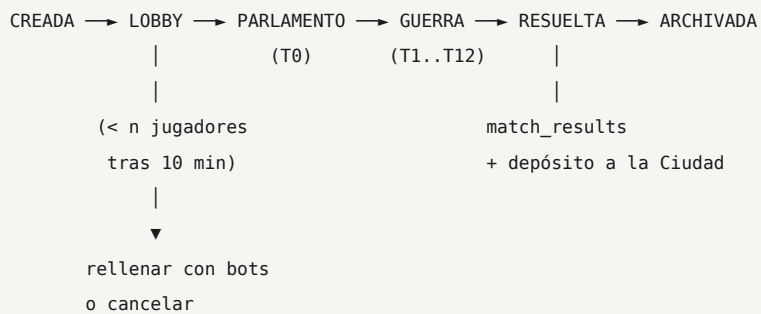
La cadencia por defecto de la beta es una de las tres decisiones bloqueantes (**DISCOVERY §5**). **Propuesta: Diaria por defecto**, porque no exige coincidencia horaria —el mayor problema de un juego nuevo con pocos jugadores— y porque «una semana de guerra» del brief se cumple literalmente. **Blitz debe existir desde v0.3** aunque no sea el modo principal: sin él, probar el ciclo completo tarda 6 días y el desarrollo se hace insoportable.

El motor no conoce la cadencia. Solo recibe `deadline_at`. Cambiar de cadencia no toca ni una línea del motor.

2.1 Resolución anticipada

Si **todos** envían antes del plazo, el turno se resuelve al instante. En Blitz esto suele reducir un turno de 3 min a 40 s. Un jugador puede marcar «**esperar al plazo**» si quiere reservarse el derecho a cambiar de opinión tras leer el chat — una decisión diplomática por sí misma.

3. Ciclo de vida de una partida



Estado	Qué se puede hacer
Lobby	Entrar, salir, chatear, elegir cadencia (solo el creador)
Parlamento	Desplegar la 2ª fuerza, negociar, sellar. Sin combate.
Guerra	Todo
Resuelta	Ver resultados, repetir la campaña, chat 24 h más
Archivada	Solo <code>match_results</code> + <code>(seed, órdenes)</code>

3.1 Creación

```
POST /api/games { playerCount: 5, cadence: 'daily', visibility: 'public' | 'private' }
→ genera seed, invite_code, asiento 0 para el creador
```

Partida **privada** ⇒ código de 6 caracteres para compartir. En beta, este es el flujo principal: es mucho más fácil juntar a cinco conocidos que llenar una cola pública (**DISCOVERY P1**).

3.2 Emparejamiento

Actualizado por ADR-026. El emparejamiento es ahora el **camino principal y único de la interfaz**: una vista, un botón, y la partida empieza sola en cuanto se llena. El código de invitación sigue existiendo en la base de datos y vuelve más adelante dentro de esa misma vista, no como pantalla aparte.

v1.0 no tiene emparejamiento por habilidad. Solo:

1. **Por código** — sin interfaz desde v0.3; ver ADR-026.
2. **Cola pública** — primero en llegar. Si tras **10 minutos** faltan jugadores:
 - `playerCount = 5` con 4 humanos ⇒ se ofrece empezar con 1 Mando Automático;
 - con 3 o menos ⇒ se ofrece convertirla en partida de 3 o de 2.
3. **Contra bots** — práctica en solitario, sin recompensas de metaprogresión salvo la primera vez.

Sin ELO, sin ranking: ver [METAPROGRESSION §7](#).

4. Autoridad y estado

Resumen; el detalle está en [TECHNICAL_DESIGN §6-8](#).

	Cliente	Servidor
Órdenes en borrador	✓ dueño	Las persiste como borrador
Previsualización de combate	✓ calcula	No la usa
Preferencias de UI	✓	—
Estado de la partida	✗ solo recibe su vista	✓ única verdad
Validación de órdenes	Solo para avisar al usuario	✓ decide
Resolución del turno	✗	✓
Niebla de guerra	✗ recibe ya filtrado	✓ la calcula
Victoria	✗	✓

Un cliente completamente manipulado puede, como máximo, **enviar órdenes inválidas**, que se rechazan. No puede ver lo que no debe ni alterar nada.

5. Ausencias, abandonos y Mando Automático

Este es el problema que mata a los 4X asíncronos, y la mitigación es escalonada.

5.1 Órdenes Permanentes

Cada jugador configura, una vez, un comportamiento por defecto:

```
Si no envío órdenes a tiempo:
  Postura      ▶ Firme | Pantalla
  Producción   ▶ Nada | Línea | Repetir lo último
  Diplomacia   ▶ Rechazar todo | No responder
```

Al vencer el plazo se ejecutan. **Nunca atacan, nunca rompen un Sello, nunca consagran.** Un jugador ausente se defiende y no perjudica a terceros.

5.2 Escalado

Turnos sin enviar	Qué pasa
1	Órdenes Permanentes. Los demás ven un indicador «ausente» junto a su nombre.
2	Ídem + aviso push «te van a sustituir».
3	El asiento pasa a Mando Automático . Público.
—	El jugador recupera su asiento en cualquier momento reconectándose.

Que la ausencia sea **pública** es importante: cambia el cálculo diplomático de todos. Un jugador ausente es una oportunidad, y saberlo genera conversación.

5.3 Mando Automático

Un bot deliberadamente **conservador y honesto**:

- Consolida fuerzas en las regiones que ya controla; no ataca salvo para recuperar una región propia perdida el turno anterior.
- Produce Línea si tiene ● de sobra.
- HONRA todos los Sellos vigentes. Nunca los rompe.
- Rechaza toda oferta nueva.
- Nunca declara Consagración ni Coalición.
- No usa anomalías ni Sombra.

Diseñado para **no decidir la partida**: no es un rival fuerte ni un regalo. Que honre los Sellos es esencial — un tratado firmado con un humano no debe evaporarse porque se haya ido a dormir.

5.4 Abandono explícito

Distinto de rendirse (GDD §14.2). Abandonar:

- entrega el asiento a Mando Automático de inmediato;
- deposita 0 ♦;
- registra un evento de abandono en el perfil (el único con etiqueta explícita).

La partida **nunca** se cancela por un abandono. Solo se cancela si **todos** los humanos abandonan.

6. Reconexión

El cliente no guarda nada del juego, así que reconectar es simplemente montar de nuevo:

1. Sesión (cookie httpOnly de Supabase SSR) → auth.uid()
2. GET última player_view (game_id, seat)
3. GET orders del turno actual → recupera el borrador
4. Suscribirse Realtime
5. Renderizar

Objetivo: < 2 s en 4G. Se puede cerrar la pestaña a mitad de un turno y volver desde otro dispositivo sin perder nada: el borrador se guarda con *debounce* de 2 s.

6.1 Desconexión durante Blitz

En cadencia Blitz una desconexión de 3 minutos cuesta un turno. Mitigaciones:

- Órdenes Permanentes (§5.1) se aplican igual.
- El indicador «desconectado» es visible para todos ⇒ los demás lo saben y pueden incorporarlo a su lectura de la partida.
- El plazo **se pausa 60 s** ⚖ la primera vez que un jugador se desconecta en una partida Blitz. Una sola vez por jugador y partida: es un margen para un túnel, no un arma para ganar tiempo.

7. Notificaciones

Evento	Blitz	Diaria/Relajada
Turno resuelto	En la app	Push
Quedan 20 % del plazo	En la app	Push
Oferta diplomática recibida	En la app	Push
Se rompió un Sello contigo	En la app	Push
Alguien declara Consagración	En la app	Push
Mensaje de chat	Indicador	Agrupado, máx. 1/hora

Web Push (VAPID) desde v0.9, opt-in explícito. Máximo **4 push/día** por partida: un juego asíncrono que notifica de más se desinstala.

8. Rendimiento y escala

Magnitud	Valor	Límite del free tier
Peticiones por jugador y turno	~3	—
Bytes por jugador y turno	~70 KB	5 GB/mes ⇒ ~6 000 partidas-jugador
Conexiones Realtime por partida	5	200 ⇒ 40 partidas simultáneas
Tiempo de resolución (5 jugadores)	≤ 15 ms	—
Escrituras por resolución	1 + 5 vistas	—

El cuello de botella del free tier son las 200 conexiones Realtime concurrentes. Con cadencia Diaria, la ocupación real es mínima (los jugadores entran, juegan 3 minutos y se van). Con Blitz, 40 partidas simultáneas son el techo. Suficiente para la beta; documentado como el primer límite que se romperá.

9. Casos límite

Caso	Resolución
Dos jugadores mueven a la misma región vacía	Combaten allí
A va $X \rightarrow Y$ y B va $Y \rightarrow X$	Combaten en la ruta; ninguno avanza
Empate exacto de poder	Región Disputada : no produce para nadie
Un jugador envía órdenes dos veces	La segunda sustituye a la primera (mismo turno)
Órdenes recibidas justo al vencer el plazo	El <code>deadline_at</code> de la BD manda; la carrera la resuelve el advisory lock
Se resuelve un turno dos veces	Idempotente por <code>(game_id, turn)</code>
Todos abandonan	Partida <code>abandoned</code> ; sin depósito para nadie
Todos los humanos son sustituidos por bots	La campaña se resuelve sola hasta el T12 y se archiva
Empate final en la Reclamación Menor	Empate declarado : ambos vencedores. No se desempata al azar.
Un jugador se rinde ante otro en el T12	Se aplica antes de puntuar la Reclamación Menor
Un jugador consagra y se desconecta	La Consagración continúa: solo requiere control y $\uparrow$, no presencia
Migración del motor a mitad de campaña	La partida conserva su <code>engine_version</code> ; nunca cambia

10. Plan de implementación (v0.3)

Paso	Entrega	Test
1	Supabase Auth (email + magic link)	E2E: registro, login, logout
2	<code>profiles</code> + RLS	Seguridad: no leo el perfil ajeno
3	Crear / unirse a partida por código	E2E: 5 clientes entran a la misma partida
4	Envío de órdenes + persistencia del borrador	Integración: cerrar y reabrir conserva el borrador
5	Resolución con advisory lock	Concurrencia: 10 peticiones simultáneas $\Rightarrow$ 1 sola resolución
6	<code>player_views</code> + RLS	Seguridad: no leo la vista ajena $\leftarrow$ bloqueante
7	Realtime	E2E: el cliente B ve la resolución sin recargar
8	Reconexión	E2E: recargar a mitad de turno no pierde nada
9	Plazos + <code>pg_cron</code> + resolución oportunista	Integración: un turno vencido se resuelve sin cliente conectado
10	Órdenes Permanentes + Mando Automático	Integración: 3 turnos sin enviar $\Rightarrow$ bot; el bot honra los Sellos
11	Cadencia Blitz	Manual: partida completa en < 60 min

Los pasos 5 y 6 son los **bloqueantes de seguridad**: sin ellos, ni el multijugador es justo ni la niebla de guerra existe.

09 Technical Design Document

docs/TECHNICAL_DESIGN.md

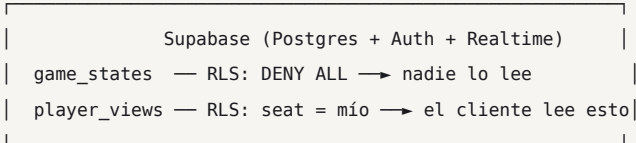
Versión: 1.0 · **Estado:** aprobado para implementación Complementa a **GAME_DESIGN** (qué hace el juego) explicando **cómo se construye**. Toda decisión relevante está registrada en **DECISIONS**.

Índice

- 1. Principio rector
- 2. Arquitectura
- 3. El motor
- 4. Determinismo y aleatoriedad
- 5. Base de datos
- 6. Niebla de guerra y RLS
- 7. API
- 8. Resolución de turnos y concurrencia
- 9. Tiempo real, reconexión y persistencia
- .0. Frontend
 - .1. Modelo de amenazas
 - .2. Observabilidad y errores
 - .3. Rendimiento
 - .4. Versionado y migraciones
 - .5. CI/CD

1. Principio rector

Un solo motor. Tres consumidores. Cero confianza en el cliente.



2.2 Capas de `apps/web`

Capa	Ubicación	Puede
Autoridad	<code>app/api/**/route.ts</code>	<code>service_role</code> , escribir <code>game_states</code> , ejecutar <code>reduce()</code>
Lectura	Server Components	Cliente Supabase de usuario (RLS activa), leer <code>player_views</code>
Presentación	Client Components	Solo props + Realtime; <code>reduce()</code> en modo preview

Regla de import verificada por lint: nada bajo `components/` puede importar `lib/server/`.

3. El motor

3.1 Firma

```
// packages/core/src/rules/reduce.ts
export function reduce(
  state: GameState,           // congelado, nunca se muta
  orders: OrdersBySeat,       // ya validadas
  ctx: ResolveContext,        // { rngSeed, engineVersion, now }
): ResolveResult;

export interface ResolveResult {
  state: GameState;           // estado del turno siguiente
  events: GameEvent[];        // log con visibilidad por evento
  views: Record<Seat, PlayerView>; // proyecciones ya filtradas
  checksum: string;           // hash canónico del estado
}
```

Propiedades garantizadas por test:

- **Pura** — misma entrada, misma salida, siempre, en cualquier runtime.
- **Inmutable** — `state` de entrada nunca se modifica (verificado con `Object.freeze` profundo en modo test).
- **Total** — nunca lanza por órdenes inválidas: las descarta y emite un evento `ORDER_REJECTED`.

3.2 Estructura del estado

```
interface GameState {
  meta: {
    gameId: string; engineVersion: string; mapgenVersion: string;
    seed: number; turn: number; phase: Phase; playerCount: 2 | 3 | 5;
  };
  map: {
    regions: Region[];           // índice = regionId
    edges: Edge[];               // { a, b, severedUntilTurn? }
    coreId: RegionId;
  };
  seats: SeatState[];           // recursos, investigación, doctrina, anomalías, flags
  forces: Force[];              // { id, seat, regionId, line, fire, sky, posture }
  shades: Shade[];              // { id, seat, regionId, cooldown }
  control: Record<RegionId, Seat | null>;
  treaties: Treaty[];
  attunement: { seat: Seat | null; coalitionWith: Seat | null;
    progress: 0|1|2|3; paidThisTurn: boolean } | null;
  rngCursor: number;            // posición del PRNG – parte del estado
}
```

`rngCursor` **forma parte del estado**. Sin eso, reproducir una partida desde `(seed, órdenes)` sería imposible tras una interrupción.

3.3 Organización de `reduce()`

Las 14 etapas del **orden de resolución** son **14 funciones puras encadenadas**:

```
const PIPELINE = [
  validateOrders, applyDiplomacy, applyTopologyAnomalies, applyShadeOps,
  applyMovement, resolveCombat, recomputeControl, applyEconomy,
  applyProduction, applyCore, applyInfoAnomalies, computeVisibility,
  emitEvents, closeTurn,
] as const;

export function reduce(state, orders, ctx) {
  return PIPELINE.reduce((acc, stage) => stage(acc), initial(state, orders, ctx));
}
```

Cada etapa se testea aisladamente. Añadir una regla nueva es añadir o modificar **una** función, nunca tocar las otras trece. Es la garantía de «código pequeño, comprensible, testeable y extensible» del brief §26.

4. Determinismo y aleatoriedad

4.1 Reglas duras

Prohibido en <code>packages/core</code>	Sustituto
<code>Math.random()</code>	<code>rng(state)</code> — xoshiro128** sembrado
<code>Date.now()</code> , <code>new Date()</code>	<code>ctx.now</code> , inyectado
Iterar objetos sin ordenar	<code>Object.keys(x).sort()</code> siempre
<code>Array.sort()</code> sin comparador total	comparador con desempate por <code>id</code>
<code>Set</code> / <code>Map</code> en salidas	arrays ordenados
Punto flotante en comparaciones de igualdad	redondeo a 4 decimales antes de comparar

Un test de ESLint personalizado (`no-nondeterminism`) falla el build si aparecen.

4.2 PRNG

xoshiro128** — 16 bytes de estado, sin dependencias, misma salida en cualquier motor de JS.

```
export function makeRng(seed: number, cursor: number): Rng;
// avanzar el rng avanza state.rngCursor → el estado siempre sabe dónde está
```

4.3 Dónde hay aleatoriedad

Muy poca, a propósito:

Sistema	¿Aleatorio?
Generación del mapa	Sí — sembrada por <code>mapSeed</code>
Combate	No — determinista (decisión de diseño, ver GDD §7)
Economía, producción, captura	No
Eventos falsos de <i>Sembrar</i>	Sí — elige entre plantillas plausibles
Desempates	No — siempre por número de asiento

4.4 Checksum canónico

```
canonicalJson(state) // claves ordenadas, floats a 4 decimales, sin undefined
→ sha256 → hex(16)
```

Se guarda en cada turno. Permite:

- detectar divergencia cliente/servidor;
- verificar que una partida reproducida desde `(seed, órdenes)` es idéntica;
- detectar migraciones de motor que rompen partidas antiguas.

5. Base de datos

5.1 Esquema

```
-- Cuentas y metaprogresión --
create table profiles (
  id          uuid primary key references auth.users on delete cascade,
  display_name text not null check (char_length(display_name) between 3 and 20),
  locale      text not null default 'es' check (locale in ('es','en')),
  created_at  timestampz not null default now()
);

create table cities (
  profile_id  uuid primary key references profiles on delete cascade,
  ash_bank    integer not null default 0 check (ash_bank >= 0),
  districts   jsonb  not null default '{}'::jsonb,  -- { archivo: 2, foundry: 1, ... }
  loadout     jsonb  not null default '{}'::jsonb,  -- { doctrine, anomalies[3], city }
  updated_at  timestampz not null default now()
);

create table account_unlocks (
  profile_id  uuid references profiles on delete cascade,
  unlock_key  text not null,
  unlocked_at timestampz not null default now(),
  primary key (profile_id, unlock_key)
);

create table reputation_events (
  id          bigserial primary key,
  profile_id  uuid references profiles on delete cascade,
  game_id     uuid,
  kind        text not null,      -- seal_honoured | seal_breached | abandoned | ...
  created_at  timestampz not null default now()
);

-- Partidas --
create type game_status as enum ('lobby','active','finished','abandoned');
create type game_phase  as enum ('parley','war','resolved');

create table games (
  id          uuid primary key default gen_random_uuid(),
  status      game_status not null default 'lobby',
  phase       game_phase  not null default 'parley',
  player_count smallint not null check (player_count in (2,3,5)),
  cadence     text not null check (cadence in ('blitz','daily','relaxed')),
  turn        smallint not null default 0,
  state_version integer not null default 0,      -- bloqueo optimista
  map_seed    bigint  not null,
  mapgen_version text  not null,
  engine_version text  not null,
  deadline_at timestampz,
  invite_code text unique,
  created_by  uuid references profiles,
  created_at  timestampz not null default now(),
```

```

    finished_at    timestampz
);

create index on games (status, deadline_at) where status = 'active';

create table game_players (
    game_id        uuid references games on delete cascade,
    seat           smallint not null check (seat between 0 and 4),
    profile_id     uuid references profiles,                -- null si es bot
    doctrine       text,
    city           text,
    is_bot         boolean not null default false,
    missed_turns   smallint not null default 0,
    standing_orders jsonb not null default '{} '::jsonb,
    primary key (game_id, seat),
    unique (game_id, profile_id)
);

-- Estado autoritativo. NADIE lo lee vía la API pública.
create table game_states (
    game_id        uuid references games on delete cascade,
    turn           smallint not null,
    state          jsonb not null,
    checksum       text not null,
    created_at     timestampz not null default now(),
    primary key (game_id, turn)
);

-- Proyección por asiento, YA FILTRADA por niebla de guerra.
create table player_views (
    game_id        uuid references games on delete cascade,
    turn           smallint not null,
    seat           smallint not null,
    view           jsonb not null,
    events         jsonb not null default '[] '::jsonb,
    created_at     timestampz not null default now(),
    primary key (game_id, turn, seat)
);

create table orders (
    game_id        uuid references games on delete cascade,
    turn           smallint not null,
    seat           smallint not null,
    payload        jsonb not null,
    submitted_at   timestampz not null default now(),
    primary key (game_id, turn, seat)
);

create table treaties (
    id             uuid primary key default gen_random_uuid(),
    game_id        uuid references games on delete cascade,
    kind           text not null,                -- seal | transfer | coalition
    proposer_seat  smallint not null,
    target_seat    smallint not null,
    terms          jsonb not null,
    status         text not null,                -- proposed | active | fulfilled | breached | expired

```

```

    created_turn smallint not null,
    expires_turn smallint,
    created_at timestampz not null default now()
);
create index on treaties (game_id, status);

create table messages (
    id          bigserial primary key,
    game_id     uuid references games on delete cascade,
    channel     text not null,          -- 'public' | 'seat:0,3'
    sender_seat smallint not null,
    body        text check (char_length(body) <= 500),
    template    jsonb,                 -- oferta por plantilla (i18n-safe)
    created_at  timestampz not null default now()
);
create index on messages (game_id, channel, created_at desc);

create table match_results (
    game_id     uuid references games on delete cascade,
    seat        smallint not null,
    profile_id  uuid references profiles,
    outcome     text not null,         -- attuned | coalition | lesser | survivor | reduced | abandoned
    seams       smallint, ash integer, regions smallint,
    ash_awarded integer not null default 0,
    unlocks     jsonb not null default '[]'::jsonb,
    primary key (game_id, seat)
);

```

5.2 Por qué `jsonb` para el estado y no tablas relacionales

Se evaluó normalizar (`forces`, `control`, `regions` como tablas):

	jsonb	Relacional
Escritura de un turno	1 fila	~200 filas, 8 tablas
Atomicidad	trivial	transacción larga
Coste en free tier	bajo	alto (índices)
Consulta analítica	pobre	buena
Acoplamiento con el motor	ninguno — el motor ya trabaja con ese objeto	alto: dos representaciones que mantener sincronizadas

El estado **solo lo lee y escribe el motor, entero, de una vez**. No hay ninguna consulta de negocio que necesite filtrar dentro de él. `jsonb` es la elección correcta. Las consultas analíticas salen de `match_results` y `turn_events`, que **sí** están normalizados. → [ADR-007](#).

5.3 Retención

Dato	Retención	Motivo
game_states	snapshot cada 4 turnos + último	Los intermedios se reconstruyen desde seed + orders
player_views	últimos 3 turnos	El historial se reconstruye a demanda
orders	siempre (partida activa y terminada)	Es la fuente de verdad reproducible
messages	30 días tras finalizar	Coste
match_results	siempre	Metaprogresión

Coste medido: **≈ 80 KB por partida terminada**. Con 500 MB del free tier ⇒ ~6 000 partidas archivadas sin tocar nada.

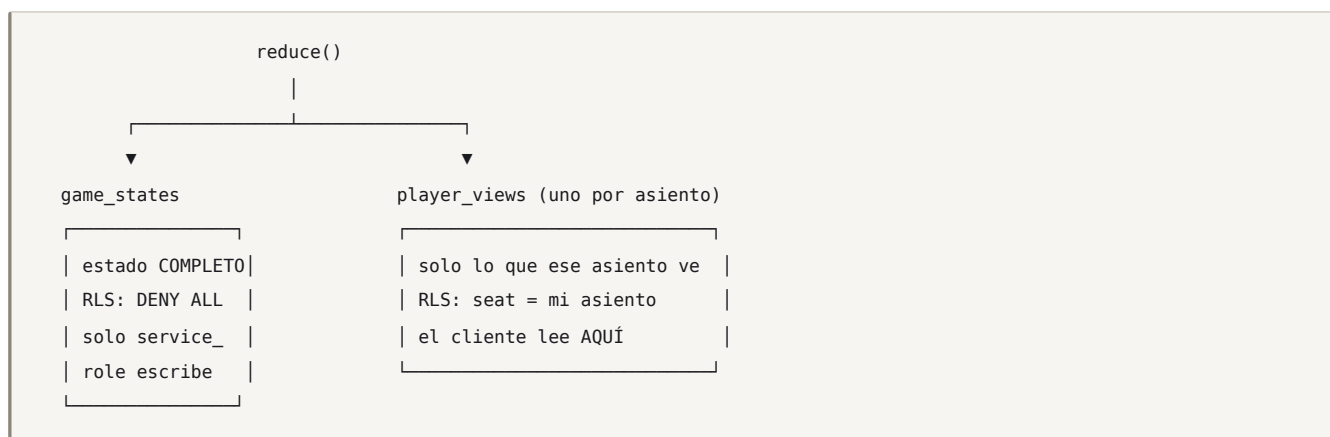
6. Niebla de guerra y RLS

6.1 El problema

Supabase expone PostgREST. Si `game_states` fuera legible por el jugador, cualquiera haría `GET /rest/v1/game_states?game_id=eq.X` y vería el estado completo: posiciones ocultas, órdenes, stocks ajenos. **La niebla de guerra sería puramente cosmética.**

Este es el riesgo técnico nº 1 del proyecto (**DISCOVERY T1**).

6.2 La solución



La proyección se calcula **en el servidor, dentro de `reduce()`**, en la etapa `computeVisibility`. Los datos que un jugador no debe ver **no salen nunca del servidor**.

6.3 Políticas

```
alter table game_states enable row level security;
alter table player_views enable row level security;
alter table orders      enable row level security;
alter table messages     enable row level security;
alter table treaties     enable row level security;

-- game_states: ninguna política => ningún acceso para anon/authenticated.
-- (service_role omite RLS por definición.)

create policy "ver mi propia vista" on player_views for select
using (exists (
    select 1 from game_players gp
    where gp.game_id = player_views.game_id
        and gp.seat   = player_views.seat
        and gp.profile_id = auth.uid()
));

-- Las órdenes: escribo las mías, del turno actual, solo si la partida está activa.
create policy "enviar mis órdenes" on orders for insert
with check (
    exists (select 1 from game_players gp
            where gp.game_id = orders.game_id and gp.seat = orders.seat
              and gp.profile_id = auth.uid())
    and exists (select 1 from games g
              where g.id = orders.game_id and g.status = 'active'
              and g.turn = orders.turn)
);

create policy "reenviar mis órdenes" on orders for update
using (exists (select 1 from game_players gp
                where gp.game_id = orders.game_id and gp.seat = orders.seat
                  and gp.profile_id = auth.uid()))
with check (exists (select 1 from games g
                    where g.id = orders.game_id and g.turn = orders.turn
                      and g.status = 'active')));

create policy "leer mis órdenes" on orders for select
using (exists (select 1 from game_players gp
                where gp.game_id = orders.game_id and gp.seat = orders.seat
                  and gp.profile_id = auth.uid()));

-- Mensajes: canal público de mi partida, o canal privado que me incluye.
create policy "leer mensajes visibles" on messages for select
using (
    exists (select 1 from game_players gp
            where gp.game_id = messages.game_id and gp.profile_id = auth.uid()
              and (messages.channel = 'public'
                  or messages.channel like '%' || gp.seat::text || '%'))
);
```

Nota sobre el canal privado. El `like` de arriba es ilustrativo; la implementación real usa `channel_seats` `smallint[]` con `gp.seat = any(m.channel_seats)`, que es exacto e indexable con GIN. Ver migración `0004_messages.sql`.

6.4 Test de seguridad obligatorio

En `apps/web/tests/security/rls.test.ts`, contra Supabase local:

```
x un jugador NO puede leer game_states de su propia partida
x un jugador NO puede leer el player_view del asiento 2 siendo el asiento 0
x un jugador NO puede insertar órdenes en el asiento de otro
x un jugador NO puede insertar órdenes de un turno pasado o futuro
x un jugador NO puede leer el canal privado de otros dos
x un jugador NO puede modificar games.turn, games.state_version ni cities.ash_bank
✓ un jugador SÍ puede leer su propio player_view
```

Estos tests son bloqueantes: si uno falla, no hay release.

7. API

Route Handlers en `apps/web/app/api/`. Todos validan con Zod, todos devuelven `{ ok: true, ... }` o `{ ok: false, code, message }` con `code` traducible.

Método	Ruta	Autoridad	Descripción
POST	<code>/api/games</code>	✓	Crear partida (genera semilla, código de invitación)
POST	<code>/api/games/:id/join</code>	✓	Unirse; asigna asiento libre
POST	<code>/api/games/:id/start</code>	✓	Genera el mapa, valida equidad, escribe el turno 0
POST	<code>/api/games/:id/orders</code>	✓	Enviar órdenes del turno actual. Puede disparar resolución.
POST	<code>/api/games/:id/preview</code>	✗	Previsualización de combate. Solo lectura, sin efectos.
POST	<code>/api/games/:id/treaties</code>	✓	Proponer / aceptar / romper
POST	<code>/api/games/:id/messages</code>	✗	Chat (validación de canal + rate limit)
POST	<code>/api/games/:id/surrender</code>	✓	Rendición dirigida
GET	<code>/api/games/:id/state</code>	✗	Vista del jugador (fallback si Realtime falla)
POST	<code>/api/cron/resolve-due</code>	✓	Resolver turnos vencidos. HMAC obligatorio.

7.1 Contrato de órdenes

```
const OrdersSchema = z.object({
  turn: z.number().int().nonnegative(),
  moves: z.array(z.object({
    forceId: z.string(),
    to: z.number().int().optional(), // null = quedarse
    detach: z.object({ line: z.number(), fire: z.number(), sky: z.number() }).optional(),
    posture: z.enum(['assault', 'hold', 'screen']),
    fireSupport: z.number().int().optional(),
  })).max(6),
  production: z.array(z.object({
    regionId: z.number().int(),
    item: z.enum(['line', 'fire', 'sky', 'shade', 'fort', 'bridge']),
    qty: z.number().int().min(1).max(3),
  })).max(4),
  research: z.enum([...RESEARCH_KEYS]).optional(),
  anomaly: z.object({ key: z.enum([...ANOMALY_KEYS]), target: z.unknown() }).optional(),
  shadeOps: z.array(z.object({
    shadeId: z.string(), op: z.enum([...SHADE_OPS]), target: z.unknown(),
  })).max(3),
  attune: z.boolean().optional(),
}).strict();
```

`.strict()` es deliberado: un campo desconocido es **rechazo**, no ignorado. Cierra la puerta a que un cliente manipulado cuele campos que una futura versión interprete.

7.2 Validación en dos niveles

1. **Sintáctica** (Zod) — forma, rangos, tamaños. Barata, primero.
2. **Semántica** (motor, contra el estado autoritativo) — ¿esa fuerza es tuya? ¿esa región es adyacente? ¿tienes ese recurso? ¿ya usaste esa anomalía?

La validación semántica **nunca** usa datos enviados por el cliente que no sean identificadores. El cliente dice «mueve la fuerza f3 a la región 17»; todo lo demás lo pone el servidor.

8. Resolución de turnos y concurrencia

8.1 Los tres disparadores

Vercel Hobby solo permite **un cron diario**, lo que es inservible para turnos de 3 minutos (**DISCOVERY T3**). Solución: tres caminos independientes que convergen en la misma función idempotente.

#	Disparador	Cuándo	Cubre
1	Inmediato	El último jugador envía órdenes	El caso normal (~85 %)
2	pg_cron (Supabase, cada minuto) → <code>pg_net</code> → <code>/api/cron/resolve-due</code>	Plazo vencido	Ausencias
3	Oportunista	Cualquier petición de cliente detecta un plazo vencido y lo resuelve	Red de seguridad si 1 y 2 fallan

8.2 Idempotencia y bloqueo

```
async function resolveTurn(gameId: string, expectedTurn: number) {
  return db.transaction(async (tx) => {
    // 1. Serializar por partida (se libera solo al terminar la transacción)
    await tx.raw('select pg_advisory_xact_lock(hashtextextended($1, 0))', [gameId]);

    // 2. Releer bajo el lock
    const game = await tx.one('select * from games where id = $1 for update', [gameId]);
    if (game.turn !== expectedTurn) return { skipped: 'already_resolved' };
    if (game.status !== 'active') return { skipped: 'not_active' };

    const ready = await allSubmittedOrDeadlinePassed(tx, game);
    if (!ready) return { skipped: 'not_ready' };

    // 3. Cargar estado autoritativo + órdenes (los ausentes → Órdenes Permanentes)
    const state = await loadState(tx, gameId, game.turn);
    const orders = await loadOrdersWithFallback(tx, gameId, game.turn);

    // 4. Motor puro
    const result = reduce(state, orders, {
      rngSeed: game.map_seed, engineVersion: ENGINE_VERSION, now: game.deadline_at,
    });

    // 5. Escribir con bloqueo optimista
    const upd = await tx.result(
      `update games
      set turn = turn + 1, state_version = state_version + 1,
      deadline_at = $2, phase = $3, status = $4
      where id = $1 and state_version = $5`,
      [gameId, nextDeadline(game), result.state.meta.phase,
        result.finished ? 'finished' : 'active', game.state_version],
    );
    if (upd.rowCount !== 1) throw new ConcurrencyError(); // rollback

    await persistSnapshotIfNeeded(tx, result);
    await tx.insertMany('player_views', toViewRows(result));
    return { resolved: true, turn: game.turn + 1 };
  });
}
```

Tres defensas superpuestas:

1. `pg_advisory_xact_lock` — solo un resolutor por partida a la vez.
2. `game.turn !== expectedTurn` — idempotencia por turno.
3. `where state_version = $5` — bloqueo optimista como última red.

`ctx.now = game.deadline_at`, **no** `Date.now()`: reejecutar la resolución produce el mismo resultado exacto.

8.3 Órdenes Permanentes

Si un asiento no envió órdenes al vencer el plazo:

```
function standingOrdersFor(seat, state) {
  return {
    moves: state.forces.filter(f => f.seat === seat)
      .map(f => ({ forceId: f.id, posture: seat.standingPosture ?? 'hold' })),
    production: seat.standingProduction ?? [], // configurado por el jugador
    shadeOps: [], anomaly: undefined, attune: false,
  };
}
```

Nunca ataca, nunca rompe un Sello, nunca consagra. La ausencia jamás **daña a un tercero**: el jugador ausente se defiende, y ya.

9. Tiempo real, reconexión y persistencia

9.1 Realtime

Supabase Realtime sobre Postgres Changes. Cada cliente se suscribe a exactamente **dos** canales de su partida:

```
supabase.channel(`game:${gameId}:${mySeat}`)
  .on('postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'player_views',
      filter: `game_id=eq.${gameId}` }, // RLS filtra el asiento
    onNewTurn)
  .on('postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'messages',
      filter: `game_id=eq.${gameId}` },
    onMessage)
  .subscribe();
```

Realtime **respeta RLS**: aunque el filtro sea por partida, un jugador solo recibe la fila de su propio asiento. La seguridad no depende del filtro del cliente.

No hay polling. Fallback: si el canal se cae, un `GET /api/games/:id/state` cada 60 s mientras esté desconectado, con retroceso exponencial.

9.2 Reconexión

El cliente es **stateless respecto a la partida**. Al montar:

1. GET player_views WHERE game_id = X AND seat = mío ORDER BY turn DESC LIMIT 1
2. GET orders WHERE game_id = X AND turn = actual (recupera un borrador enviado)
3. Suscribir Realtime
4. Reconstruir la interfaz desde la vista

Cerrar la pestaña a mitad de turno no pierde nada: las órdenes se guardan como borrador (`orders` con `submitted_at = null`) en cada cambio, con *debounce* de 2 s.

9.3 Lo que persiste dónde

Dato	Dónde	Por qué
Estado autoritativo	Postgres <code>game_states</code>	Fuente de verdad
Órdenes (incl. borradores)	Postgres <code>orders</code>	Reproducibilidad + reconexión
Vista del jugador	Postgres <code>player_views</code>	Lectura barata con RLS
Preferencias UI (zoom, panel abierto)	<code>localStorage</code>	Irrelevante para el juego
Sesión	Cookie httpOnly (Supabase SSR)	Seguridad
Nada del estado de juego	<code>localStorage</code>	El cliente no es autoridad

10. Frontend

10.1 Renderizado del mapa: SVG

	SVG	Canvas 2D	WebGL
~95 regiones	✓ sobrado	✓	✓ excesivo
Nitidez en pantallas 3x	✓ vectorial	⚠	✓
Accesibilidad (foco, lector de pantalla)	✓ nativa	✗	✗
Peso del bundle	0 KB	0 KB	+80 KB
Assets	El mismo SVG que la galería	rasterizar	atlas
Zoom/pan	transform CSS	manual	manual

SVG gana claramente. Cada región es un `<path>` que además es un elemento **enfocable con teclado** y anunciable por lector de pantalla — accesibilidad gratis (brief §32).

Rendimiento: transform en el `<g>` raíz (compuesta por GPU), `will-change: transform`, y `content-visibility` en los paneles. Objetivo 60 fps en un móvil de gama media de 2021.

10.2 Estado del cliente

Sin Redux, sin Zustand. Tres piezas:

1. **Vista del servidor** → React Server Component (o `useSyncExternalStore` sobre el canal Realtime).
2. **Órdenes en borrador** → un `useReducer` local. Es el único estado mutable real.
3. **Preferencias UI** → `localStorage`.

`useReducer` sobre las órdenes en borrador tiene una ventaja concreta: es el mismo formato que se envía a la API, así que **no hay transformación** entre lo que el usuario manipula y lo que se persiste.

10.3 i18n

`next-intl`, rutas `/[locale]/...`, catálogos `messages/es.json` y `messages/en.json`.

Reglas duras:

- **Cero literales visibles en componentes.** Regla de lint `no-literal-strings` sobre `apps/web/components/**`.
- Las claves se derivan del **glosario**.
- Los mensajes de error de la API devuelven `code`, no texto: el cliente traduce.

- Las **plantillas diplomáticas** son estructuras de datos, no frases: un jugador en español y otro en inglés ven la **misma oferta** en su idioma.
- Test de CI: `es.json` y `en.json` tienen exactamente el mismo conjunto de claves.

11. Modelo de amenazas

#	Amenaza	Vector	Mitigación
A1	Modificar recursos/tropas	<code>PATCH</code> directo a PostgREST	RLS: sin política de <code>update</code> sobre <code>game_states</code> , <code>games</code> , <code>cities</code> . Toda mutación pasa por Route Handler.
A2	Leer el estado completo (ver la niebla)	<code>GET game_states</code>	<code>DENY ALL</code> + <code>player_views</code> prefiltradas (§6)
A3	Leer la vista de otro jugador	<code>GET player_views?</code> <code>seat=eq.2</code>	RLS por <code>seat</code> ↔ <code>auth.uid()</code>
A4	Enviar órdenes por otro asiento	<code>POST orders</code> con <code>seat</code> ajeno	RLS <code>with check</code>
A5	Reenviar órdenes de un turno pasado/futuro	<code>turn</code> manipulado	RLS compara con <code>games.turn</code> + validación en servidor
A6	Órdenes imposibles (mover 999 tropas)	payload manipulado	Validación semántica contra estado autoritativo
A7	Resolver el turno antes de tiempo	llamar <code>/api/cron/resolve-due</code>	HMAC + comprobación de plazo/todos-enviaron
A8	Doble resolución (condición de carrera)	dos peticiones simultáneas	Advisory lock + idempotencia + bloqueo optimista (§8.2)
A9	Spam de chat / DoS de mensajes	bucle de <code>POST</code>	Rate limit 20 msg/min/asiento; 500 caracteres; validación de canal
A10	Enumerar partidas ajenas	<code>GET games</code>	RLS: solo partidas donde soy jugador o <code>status='lobby'</code> público
A11	Fuga del <code>service_role</code>	variable mal nombrada	Test de CI: falla si <code>SUPABASE_SERVICE_ROLE_KEY</code> aparece bajo <code>components/</code> o con prefijo <code>NEXT_PUBLIC_</code>
A12	Multicuenta (un humano, dos asientos)	registrar 2 cuentas	No se resuelve en v1.0. Telemetría en beta (IP + tiempos de envío correlacionados). Documentado como riesgo aceptado.
A13	Colusión externa (WhatsApp)	fuera del sistema	No se mitiga: es el género. La colusión es diplomacia.
A14	Inferir información por <i>timing</i> de la API	medir latencias	Las respuestas de <code>orders</code> no varían según el estado del rival; la resolución es simultánea.

11.1 Superficie de confianza

CONFIABLE : Route Handlers con `service_role` · funciones de Postgres · `packages/core` en el servidor
NO CONFIABLE (jamás) : todo lo que venga del navegador, incluidos ids, cantidades y checksums

12. Observabilidad y errores

12.1 Errores

```
class GameError extends Error {
  constructor(
    public code: ErrorCode,           // 'ORDER_INVALID_TARGET' - clave i18n
    public status: number,           // HTTP
    public context?: Record<string, unknown>, // se registra, NO se devuelve
  ) { super(code); }
}
```

- El cliente recibe `code` (traducible) y nunca detalles internos.
- Los errores del motor **no lanzan**: emiten `ORDER_REJECTED` con motivo, para que el turno se resuelva igualmente. Una orden mala de un jugador **jamás** bloquea la partida de los otros cuatro.

12.2 Logging

Salida JSON estructurada a stdout (Vercel la recoge). Campos fijos: `gameId`, `turn`, `seat`, `event`, `durationMs`, `checksum`.

Nunca se registra: claves, tokens, cuerpos de mensajes privados, ni el estado completo (solo su checksum).

12.3 Telemetría (desde v0.95)

Tabla `telemetry_events` con eventos anónimos y agregables: duración de partida, resultado, doctrina, rupturas de Sello, uso de anomalías, turno de la primera traición. Alimenta directamente las métricas de balance del [GDD §16.1](#).

Sin proveedor externo de analítica en v1.0: una tabla y una consulta SQL bastan, y no introduce ni coste ni cuestiones de privacidad.

13. Rendimiento

13.1 Presupuestos (verificados en CI)

Métrica	Presupuesto	Cómo se mide
JS de la ruta de partida (gzip)	≤ 180 KB	<code>@next/bundle-analyzer</code> en CI
JS de la ruta de login (gzip)	≤ 60 KB	ídem
LCP en móvil (Moto G Power, 4G)	≤ 2,5 s	Lighthouse CI
INP al tocar una región	≤ 100 ms	Lighthouse CI
Peso total de assets SVG	≤ 150 KB	script de build
Memoria en partida	≤ 60 MB	perfil manual
<code>reduce()</code> de un turno de 5 jugadores	≤ 15 ms	benchmark en Vitest
Generar + validar un mapa	≤ 250 ms p95	benchmark

13.2 Presupuesto de red

Acción	Peticiones	Bytes
Abrir una partida	2	~40 KB (vista + mensajes)
Redactar órdenes	0	0 — todo local
Guardar borrador	1 cada 2 s (debounce)	~2 KB
Enviar turno	1	~3 KB
Recibir resolución	0 (Realtime push)	~25 KB

Un turno completo ≈ 70 KB. Una partida de 12 turnos ≈ 850 KB por jugador. Con 5 GB de egreso mensual del free tier ⇒ ~6 000 partidas-jugador/mes.

Prohibido el polling. Regla de lint contra `setInterval` con peticiones de red.

14. Versionado y migraciones

Tres versiones independientes:

Versión	Dónde	Cambia cuando
<code>package.json</code>	proyecto	Cualquier release (SemVer)
<code>ENGINE_VERSION</code>	<code>packages/core</code>	Cambia una regla que altere <code>reduce()</code>
<code>MAPGEN_VERSION</code>	<code>packages/core/mapgen</code>	Cambia la generación (misma semilla, otro mapa)

Regla de compatibilidad: una partida en curso **nunca cambia de motor**. `games` guarda `engine_version`; si el despliegue trae un motor nuevo, las partidas existentes siguen resolviéndose con el suyo (`packages/core` exporta versiones anteriores del pipeline mientras haya partidas vivas) o se marcan `finished` con aviso previo.

Migraciones SQL: numeradas, incrementales, nunca editadas después de aplicarse.

15. CI/CD

```
# Pipeline conceptual (.github/workflows/ci.yml)
jobs:
  quality: typecheck · lint · check:deps (core sin dependencias) · check:il8n (claves ES=EN)
  unit: vitest packages/core (cobertura de rules/ ≥ 90 %)
  determin.: mismo estado+órdenes ⇒ mismo checksum en Node y en Chromium
  mapgen: 1000 semillas × {2,3,5} superan los umbrales de equidad
  security: tests de RLS contra Supabase local ← BLOQUEANTE
  e2e: Playwright, 360×640 y 1440×900
  budget: tamaño de bundle y de assets
  balance: (nocturno) 5000 partidas simuladas; alerta si un winrate sale de rango
  docs: npm run docs:pdf debe completar sin error
```

`main` protegida. Todo entra por PR con CI verde. Cada release: entrada en `CHANGELOG.md` + tag + documentación actualizada (**definición de hecho**).

10 UX / UI — Mobile first

docs/UX_MOBILE.md

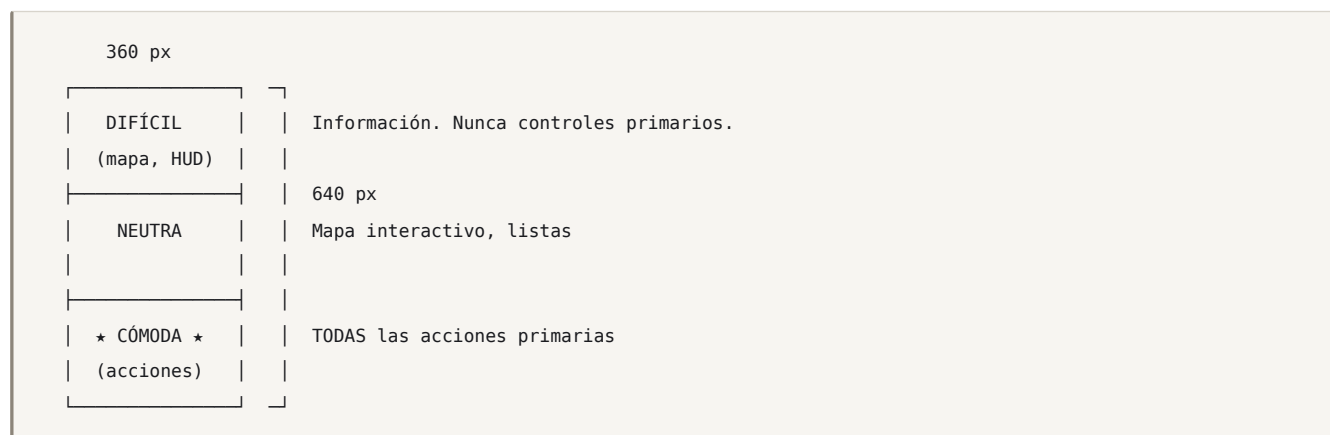
Versión: 1.0 · Referencia de diseño: **360 x 640 px** Regla del proyecto: *si no funciona en 360 px con el pulgar, no existe.*

1. Principios

1.1 El móvil no es una versión reducida: es el diseño

Se diseña a 360 px **primero**. El escritorio recibe la misma interfaz con más aire y paneles simultáneos. **No son dos juegos**. Ni una sola regla, atajo o información cambia entre plataformas — solo la densidad.

1.2 Zona del pulgar



Todo control con el que se interactúa más de una vez por turno vive en el **tercio inferior**. La barra de acción es siempre lo último de la pantalla.

1.3 Un modo, una pantalla

Nunca más de **un** panel modal abierto. Nada de menús dentro de menús: si una acción necesita tres niveles de navegación, está mal diseñada.

1.4 Objetivos táctiles

Elemento	Mínimo	Real
Botón primario	44 px	56 px
Región del mapa	44 px	48-90 px (garantizado por el tamaño del grafo)
Icono de recurso (tocable)	44 px	44 px
Elemento de lista	44 px	56 px
Separación entre táctiles	8 px	12 px

Que el mapa sea un **grafo de 45-96 regiones** y no una rejilla es lo que hace esto posible (**MAP_GENERATION §1**): la restricción de UX móvil determinó la topología del juego, no al revés.

Un mapa de 5 jugadores no cabe entero en 360 px con regiones tocables. Medido en el prototipo v0.1: a escala 1 cada región mide **21 px**, menos de la mitad del mínimo. La respuesta no es encoger el mapa (lo volvería trivial) ni fingir que cumple: el mapa **entra con el zoom necesario** para que una región mida ~52 px, centrado entre tu Bastión y el Núcleo. Ves tu sector y el objetivo; el resto se navega. El zoom inicial se calcula del ancho real del viewport, no de una constante, así que en escritorio el mapa entra entero y las regiones siguen midiendo ~52 px.

2. Gestos

Gesto	Acción	Feedback
Tap en región	Seleccionar / mostrar la hoja de región	Anillo + háptico ligero
Tap en región resaltada (con fuerza seleccionada)	Ordenar movimiento	Flecha animada + háptico
Long press (400 ms)	Compendio de ese elemento	Háptico medio + tarjeta
Drag desde una fuerza	Arrastrar a destino (alternativa al tap-tap)	Línea elástica + destinos válidos iluminados
Pinch	Zoom 0,6x-2,5x	—
Drag en vacío	Desplazar el mapa	—
Doble tap en vacío	Encuadrar todo el mapa	—
Botón «Mi ciudad»	Llevar la cámara a tu Bastión	Vuelo suave de 320 ms
Swipe ↑ desde el borde inferior	Abrir diplomacia	—
Swipe ←/→ en la barra superior	Cambiar de jugador en el panel de estado	—

Cada acción tiene siempre dos caminos: tap-tap y drag. El drag es rápido para expertos; el tap-tap es fiable con una mano y no falla con dedos grandes.

Un arrastre no puede acabar seleccionando. Entre desplazar el mapa y tocar una región hay un umbral de 8 px: sin él, navegar abre fichas sin querer y el mapa deja de ser navegable con el pulgar.

La cámara también se maneja con botones, no solo con gestos. Perder de vista tu Bastión en un mapa de hasta 96 regiones y tener que buscarlo arrastrando es la forma más rápida de que la partida deje de parecer un juego (**ADR-040**).

Nunca se usa: swipe para acciones destructivas, triple tap, gestos de dos dedos para nada que no sea zoom, ni pulsación larga como *única* forma de llegar a algo.

3. Wireframes — Móvil

3.1 Login

GUERRA DE CENIZAS
· · · · · · ← ceniza cayendo, animación sutil

correo@ejemplo.com

CONTINUAR ← 56 px

Te enviaremos un enlace de acceso. Sin contraseñas.

ES · EN ← selector de idioma siempre visible

Magic link: sin contraseñas que recordar ni gestionar, un campo, un botón. El idioma se elige **antes** de entrar.

3.2 Ciudad (inicio)

Ver [METAPROGRESSION §5.2](#). Regla: **entrar en campaña en 1 tap** desde aquí.

3.3 Lobby

← CAMPAÑA KJ7-2M ← código, tap para copiar

5 jugadores · Diaria
Esperando 2...

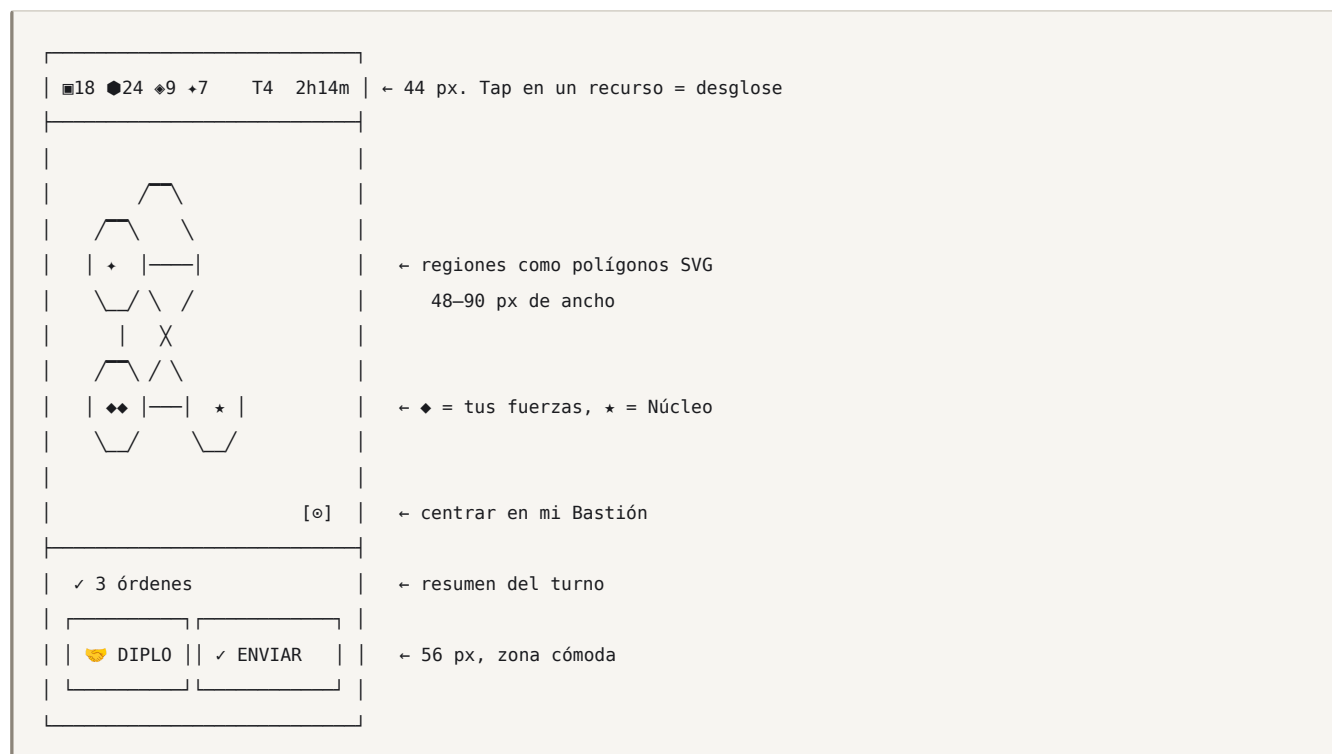
● Vantera tú
● Koldvik Marta
● Saranth Lu
○ esperando
○ esperando

COMPARTIR INVITACIÓN

[Empezar con bots] ← tras 10 min

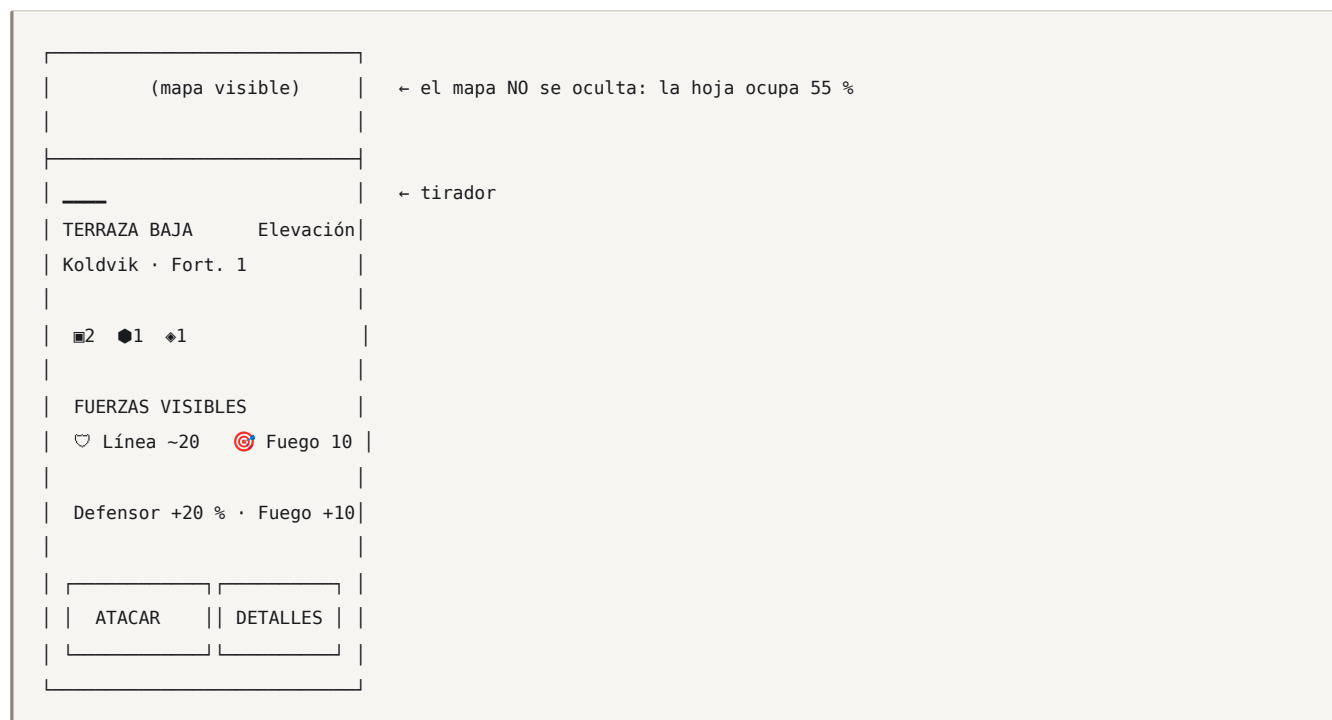
Chat del lobby

3.4 Mapa — la pantalla principal



El mapa ocupa el 70 % de la pantalla. Todo lo demás es una barra arriba (información) y una barra abajo (acción). Sin paneles laterales, sin minimapa (el mapa entero ya cabe), sin barras de herramientas.

3.5 Hoja de región (tap en una región)



La hoja **nunca cubre el mapa entero**. El jugador siempre ve el contexto de lo que está decidiendo. Se cierra con swipe hacia abajo o tocando el mapa.

3.6 Previsualización de combate

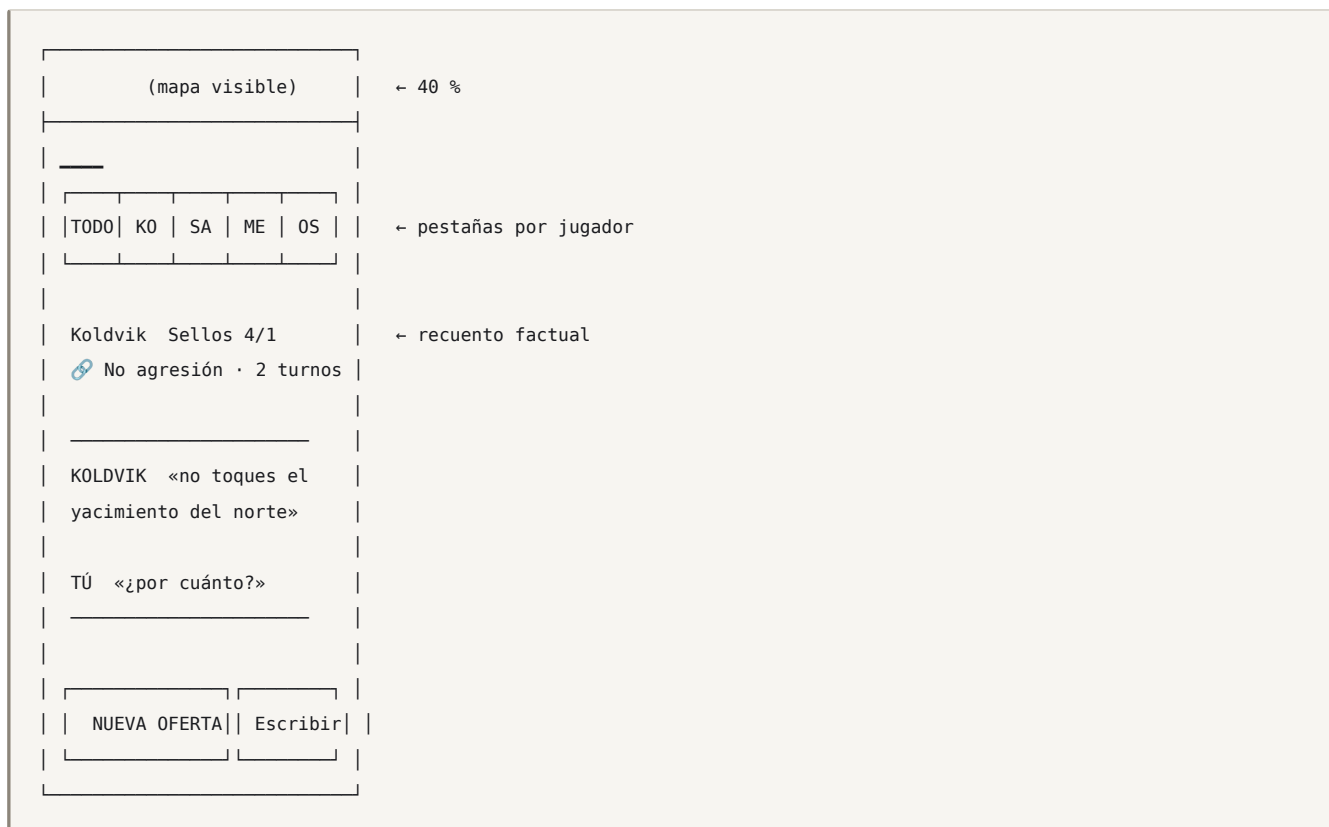
La pantalla que solo es posible gracias al combate determinista (GDD \$7):

ATACAR TERRAZA BAJA	
TÚ	KOLDVIK
♥ 20	♥ ~20
🎯 10	🎯 10
➔ 0	➔ 0
Poder 38,5	Poder 33,1
✓ VENCES	
Pierdes ~55 % ➔ ♥9 🎯4	
Capturas la región	
⚠ Estimación: la fuerza enemiga es aproximada (Bosque adyacente)	
CANCELAR	CONFIRMAR

El aviso sobre la información aproximada es esencial. El resultado es exacto *dada la información que tienes*; la incertidumbre está en los datos, no en el sistema. La UI debe enseñar eso explícitamente, porque es el corazón del juego.

3.7 Diplomacia

Ver [DIPLOMACY §3.1](#) para el compositor de ofertas.



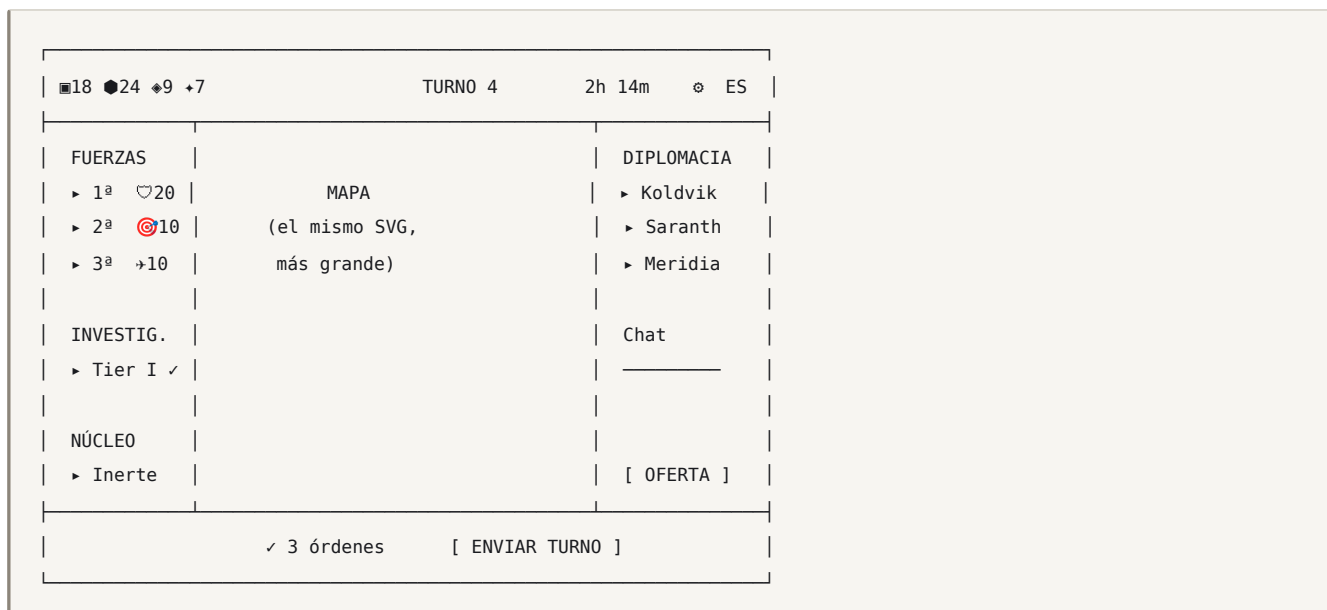
«Nueva oferta» va primero y es más grande que «Escribir». La jerarquía visual empuja hacia la negociación estructurada, que es la que funciona en móvil y entre idiomas.

3.8 Otras pantallas

Pantalla	Patrón
Ejército	Lista de tus ≤ 6 fuerzas; tap para centrar el mapa en ella
Investigación	3 tiers, 4 tarjetas cada uno; solo 1 activo a la vez
Objetivo	Estado del Núcleo, contador de consagración, quién lo tiene
Fin de turno	Resumen de tus órdenes, confirmación
Resultados	Las 3 tarjetas de Reposo
Compendio	Buscador + categorías; generado desde las tablas de balance

4. Escritorio

La misma interfaz, con más aire. Nada nuevo, nada oculto.



Los paneles laterales son **exactamente** las hojas deslizables del móvil, ancladas. Un mismo componente React con dos contenedores. Coste de mantenimiento cercano a cero, y garantía de que no divergen.

Breakpoints: < 640 móvil · 640–1024 tablet (un panel anclado) · > 1024 escritorio (dos paneles).

5. Diplomacia en móvil

El problema central: negociar en un teléfono es doloroso, y si duele, no se hace (**DISCOVERY D2**). Cuatro respuestas:

1. **Ofertas por plantilla:** 3–4 taps, sin teclado.
2. **Sugerencias contextuales:** el juego propone la oferta pertinente según el estado («Koldvik tiene tropas junto a tu yacimiento» → *Ofrecer no agresión*).
3. **Respuestas rápidas:** Acepto · No · Sube la oferta · Espera un turno.
4. **La diplomacia no oculta el mapa:** se negocia mirando el terreno.

Objetivo medible: enviar una oferta completa en ≤ 4 taps y ≤ 8 segundos. Es un test E2E.

6. Onboarding

Ver **GDD §17**. Reglas de UI:

- El tutorial **nunca** muestra más de un elemento resaltado a la vez.
- Todo lo demás se atenúa (no se bloquea: se puede explorar).
- Cada tarjeta: **una frase**, un botón, descartable.
- Se puede saltar en cualquier momento; se puede repetir desde el Compendio.
- Nada de vídeos, nada de muros de texto, nada de «acepta para continuar».

7. Accesibilidad

Requisito	Implementación
Contraste	Todo el texto $\geq 4,5:1$ (AA). Elementos de UI $\geq 3:1$. Verificado en CI con axe.
Daltonismo	Los jugadores nunca se distinguen solo por color: cada uno tiene además un patrón de trama (rayas, puntos, cuadrícula, diagonal, liso) y una inicial . Modo de alto contraste conmutable.
Tamaño de texto	<code>rem</code> ; respeta el ajuste del sistema hasta 200 % sin romper el diseño.
Táctil	Mínimo 44 px, real 56 px (§1.4).
Movimiento	<code>prefers-reduced-motion</code> : sin ceniza cayendo, sin animaciones de combate, transiciones instantáneas. El juego es plenamente jugable sin ninguna animación.
Lector de pantalla	Cada región del SVG es un <code><g role="button" tabindex="0"></code> con <code>aria-label</code> completo: « <i>Terraza Baja, elevación, controlada por Koldvik, 20 de línea visible, adyacente a tu 1ª fuerza</i> ».
Teclado	Tab recorre las regiones en orden de anillo; flechas navegan por adyacencia; Enter selecciona. El juego es 100 % jugable con teclado.
Iconos	Nunca un icono solo. Siempre icono + número, o icono + texto.
Idioma	<code>lang</code> correcto en <code><html></code> por locale.

La accesibilidad del mapa sale casi gratis por ser SVG en el DOM: si fuera Canvas o WebGL, cada punto de esta tabla costaría una implementación paralela ([TECHNICAL_DESIGN §10.1](#)).

8. Estados y feedback

Toda acción debe responder en **< 100 ms**, aunque el resultado tarde.

Estado	Señal
Seleccionable	Contorno tenue
Seleccionado	Anillo grueso + brillo + háptico ligero
Destino válido	Relleno pulsante suave
Destino inválido	Sin resaltar; al tocarlo, sacudida + motivo en un <code>toast</code>
Orden dada	Flecha persistente + badge en el contador de órdenes
Enviando	Botón con spinner, sigue legible
Enviado	✓ + háptico de éxito + el botón cambia a «Modificar»
Esperando a otros	Avatares con estado: ✓ enviado · ... pensando · × ausente
Turno resuelto	Animación de 1,5 s (saltable) + log
Error	<code>Toast</code> con causa traducida y acción de reintento

El estado «esperando a otros» es una pantalla de producto, no un espinner. Muestra quién ha enviado, cuánto queda y permite seguir negociando. Nunca se bloquea la interfaz.

9. Rendimiento en móvil

Presupuestos en **TECHNICAL_DESIGN §13**. Prácticas:

- Zoom y desplazamiento por `transform` sobre el `<g>` raíz (compuesto por GPU).
- `will-change: transform` solo mientras dura el gesto.
- Sin re-render de React durante el gesto: se manipula el `transform` por ref.
- Las hojas usan `content-visibility: auto`.
- Sin fuentes externas: fuente variable propia, subconjunto latino, `font-display: swap`.
- Sin imágenes rasterizadas en la ruta de partida: todo SVG en línea.
- La ruta de partida no carga el código de la Ciudad ni del Compendio (`dynamic import`).

10. Dirección visual de la interfaz

Coherente con **ASSET_PIPELINE**.

Fondo	#0E0F12	casi negro, frío
Superficie	#191B20	paneles
Borde	#2A2D35	
Texto	#E8E6E1	blanco ceniza
Texto tenue	#9A968E	
Acento	#C9683A	óxido – solo acciones primarias
Peligro	#B33A3A	
Éxito	#4A8B6F	
Ceniza (+)	#D6CFC4	con brillo tenue
Jugadores: P0 #4A7FB5 azul ■ rayas		
	P1 #C9683A óxido ■ cuadrícula	
	P2 #6E9B54 verde ■ diagonal	
	P3 #9B5FA8 violeta ■ puntos	
	P4 #C4A53C ámbar ■ liso	

- **Tipografía:** una única familia variable, geométrica y condensada, con números tabulares (imprescindible: hay muchas cifras alineadas).
- **Esquinas:** 4 px. Nada redondeado: es un juego militar.
- **Sombras:** ninguna. La profundidad se da con bordes y superficies.
- **Animación:** 120–200 ms, `ease-out`. Nada rebota.
- **Ceniza:** una capa de partículas muy sutil, solo en menús, nunca sobre el mapa, desactivada con `prefers-reduced-motion`.

11. Checklist de QA móvil

Antes de cerrar cualquier versión:

- ☐ Todo se usa con una mano en 360×640
- ☐ Ningún táctil < 44 px
- ☐ Ningún texto < 14 px
- ☐ Todo el texto $\geq 4,5:1$ de contraste
- ☐ Ningún panel oculta el mapa por completo
- ☐ Jugable con lector de pantalla (recorrido completo del mapa)
- ☐ Jugable solo con teclado
- ☐ Jugable con prefers-reduced-motion
- ☐ Jugable al 200 % de tamaño de texto
- ☐ Ningún jugador identificable solo por color
- ☐ Feedback en < 100 ms en toda acción
- ☐ Enviar una oferta diplomática en ≤ 4 taps
- ☐ Sin desbordamiento horizontal en 320 px (iPhone SE)
- ☐ Funciona en horizontal
- ☐ Funciona con el teclado abierto
- ☐ Sin scroll dentro de scroll

Automatizado en Playwright donde es posible; el resto es una lista manual firmada por versión.

11 Pipeline de assets

docs/ASSET_PIPELINE.md

Versión: 1.0 · **Fuentes:** `assets/src/` · **Salida:** `apps/web/components/art/` **Todos los assets son originales y creados dentro de este repositorio.**

1. La decisión que lo define todo: los assets son código

Todo asset se autora como SVG escrito a mano en el repositorio.

Consecuencia	Por qué importa
Procedencia trivial	Cada asset está en el historial de git, con autor y fecha. La exigencia del brief («todo asset debe poder rastrearse hasta su origen dentro del proyecto») se cumple por construcción, no por proceso.
Imposible incorporar assets ajenos por accidente	Un <code>.png</code> en <code>assets/</code> falla el lint. No hay ruta por la que un archivo descargado entre en producción.
Coherencia	Todos los assets comparten los mismos tokens de color y la misma cuadrícula. La coherencia deja de depender de la disciplina del autor.
Diff legible	Un cambio de asset es un diff de texto, revisable en un PR.
Sin peso extra	Los SVG en línea son parte del bundle: sin peticiones, sin CLS, sin atlas.
Theming gratis	<code>currentColor</code> y variables CSS: el mismo asset sirve para los 5 jugadores y para el modo de alto contraste.
Escala perfecta	El mismo archivo a 16 px y a 96 px, nítido en pantallas 3x.

Ninguna herramienta externa, ningún formato binario, ninguna licencia que auditar. → [ADR-011](#)

Prohibido, y verificado en CI: assets de marketplaces, imágenes de terceros, iconos con licencia incompatible, tipografías con licencia restrictiva, referencias a IP ajena, y cualquier binario en `assets/src/`.

2. Dirección artística

2.1 Concepto

Cartografía militar contemporánea, contaminada.

La estética del juego es la de un **mapa de situación** —el que se despliega sobre una mesa en un centro de mando— sobre el que la Ceniza ha empezado a dejar marcas que la cartografía no sabe representar.

- **Base:** geometría plana, líneas técnicas, símbolos legibles, tipografía condensada.

- **Contaminación:** las anomalías se dibujan **fuera del sistema:** trazos que rompen la cuadrícula, sin relleno, con un blanco ceniza que ningún otro elemento usa. No parecen parte del mapa porque **no lo son**.

Esa es toda la fantasía moderna del juego, resuelta visualmente: el mundo militar es ordenado y la Ceniza no obedece.

2.2 Reglas de estilo

Regla	Valor
Geometría	Solo formas construidas: rectas, arcos de radio constante, ángulos de 15°
Trazo	2 unidades sobre una cuadrícula de 24. Nunca varía dentro de un asset
Relleno	Plano. Sin degradados (salvo el brillo de Ceniza, único caso)
Detalle	Máximo 12 trazos por icono. Si necesita más, es un icono equivocado
Silueta	Reconocible en negro sólido a 16 px. Test obligatorio
Perspectiva	Ninguna. Todo cenital u ortográfico frontal
Sombra	Ninguna
Texto dentro del asset	Nunca (rompería la i18n)

2.3 Paleta

Fuente única: `assets/tokens/palette.json` → genera CSS variables y constantes TS. **Ningún color literal en ningún SVG ni componente.** Regla de lint.

```
{
  "surface": { "void": "#0E0F12", "panel": "#191B20", "raised": "#22252C", "line": "#2A2D35" },
  "ink":      { "primary": "#E8E6E1", "muted": "#9A968E", "faint": "#5C5952" },
  "accent":  { "rust": "#C9683A", "danger": "#B33A3A", "success": "#4A8B6F", "warn": "#C4A53C" },
  "ash":     { "core": "#D6CFC4", "glow": "#F2EDE4", "dim": "#7E796F" },
  "terrain": { "plain": "#3A4038", "urban": "#41434A", "high": "#4A4640",
               "forest": "#2F3A31", "water": "#28353D", "seam": "#4A4238", "scoured": "#1A1A1A" },
  "player":  { "p0": "#4A7FB5", "p1": "#C9683A", "p2": "#6E9B54", "p3": "#9B5FA8", "p4": "#C4A53C" }
}
```

Los colores de terreno son **deliberadamente apagados y de luminancia similar**: el mapa es el fondo, y lo que debe destacar son las fuerzas, los yacimientos y el Núcleo. Un mapa vistoso sería un mapa ilegible.

2.4 Tipografía

Una sola familia variable, geométrica, condensada, con **números tabulares** (imprescindible: hay muchas cifras que deben alinearse en columna).

Debe cubrir el latín extendido completo para ES + EN, y estar bajo licencia **SIL OFL** o equivalente, con la licencia incluida en `assets/fonts/LICENSE`. Se subsetea a los glifos usados. *(La elección concreta de familia está pendiente — ver DECISIONS ADR-017.)*

3. Inventario

3.1 Fuerzas (3 + 1)

Solo se dibujan **cuatro** siluetas en toda la campaña. Su fuerza se comunica con un número, no con más arte.

Asset	Silueta
line	Rectángulo con una diagonal — la marca cartográfica clásica de infantería
fire	Triángulo apuntando hacia arriba con base abierta
sky	Cuña / delta
shade	Rombo hueco de línea discontinua

3.2 Regiones (8)

No son iconos: son **rellenos y tramas** aplicados a los polígonos generados. plain, urban, high, forest, water, seam, bastion, core, scoured.

3.3 Recursos (4)

Asset	Forma
supply	Cuadrado con banda horizontal
industry	Hexágono con muesca
intel	Rombo con punto central
ash	Estrella de cuatro puntas con brillo (único asset con degradado)

3.4 Anomalías (8)

veil, flare, echo, fold, rift, anchor, exodus, seal. Todas dibujadas con la regla de «contaminación»: sin relleno, trazo que **sale del recuadro** de 24×24, en ash.core.

3.5 Diplomacia (6), Estados (10), UI (14)

seal, breach, transfer, coalition, vision, offer · selected, valid-target, invalid, contested, unsupplied, fortified, attuning, absent, submitted, thinking · navegación, cierre, ayuda, ajustes, idioma, etc.

Total v1.0: ~55 assets. Un inventario deliberadamente pequeño, que es lo que hace alcanzable la coherencia visual con un solo autor.

4. Convención de nombres

```
assets/src/<categoría>/<nombre>.svg
```

categorías: units · terrain · resources · anomalies · diplomacy · states · ui · brand

nombre: kebab-case, en inglés, sin abreviar, sin versión, sin tamaño

✓ fire-support.svg ✗ fireSup_v2_32.svg

Cada archivo lleva cabecera obligatoria:

```

<!--
  asset: units/line
  role: símbolo de fuerza de Línea sobre el mapa y en la UI
  grid: 24
  stroke: 2
  colors: currentColor
  author: <nombre>
  created: 2026-08-15
  original: true
-->

```

El campo `original: true` es una **declaración explícita de autoría**, verificada por el script de lint: si falta, el build falla. Es la garantía documental de la exigencia de IP del brief.

5. Del SVG al componente

```

assets/src/units/line.svg
|
▼ npm run assets:build
apps/web/components/art/generated/Line.tsx    ← componente React
apps/web/components/art/generated/index.ts     ← registro tipado
assets/manifest.json                          ← inventario + hashes

```

El generador:

1. Valida la cabecera y la cuadrícula (`viewBox="0 0 24 24"`).
2. Sustituye colores literales por `currentColor` o por variables de la paleta; **falla** si encuentra un hex no registrado.
3. Optimiza (elimina metadatos, redondea a 2 decimales).
4. Emite un componente React tipado con props `size`, `title`, `className`.
5. Actualiza `manifest.json` con hash, dimensiones y peso.

```

export function Line({ size = 24, title, ...p }: ArtProps) {
  return (
    <svg viewBox="0 0 24 24" width={size} height={size}
      role={title ? 'img' : 'presentation'} aria-label={title} {...p}>
      {title && <title>{title}</title>}
      <path d="..." fill="none" stroke="currentColor" strokeWidth="2" />
    </svg>
  );
}

```

`title` es **obligatorio** cuando el icono transmite información: llega ya traducido desde i18n. Un icono decorativo lleva `role="presentation"`.

6. QA visual: la Galería de Assets

`npm run gallery` → `/dev/gallery` (solo en desarrollo).

Es el requisito \$15 del brief y la herramienta que hace que la coherencia sea verificable y no una opinión.

GALERÍA DE ASSETS55 assets

Tamaño [16][24][32][48][96]Fondo [terrenos ▼]

☐ Silueta☐ Alto contraste☐ Rejilla☐ Daltonismo

ESCALA – el mismo asset a todos los tamaños

■

■

■

■

■

1624324896

SOBRE TERRENO – cada asset sobre los 8 terrenos

llan|urba|elev|bosq|agua|yaci|bast|núcl|

■

■

■

■

■

■

■

■

contraste: 8.16.47.29.05.86.94.1▲7.7

EN CONTEXTO – mapa real, semilla 42, 5 jugadores

EN DISPOSITIVO – 360×640 · 390×844 · 768 · 1440

FAMILIA – los 55 juntos, para detectar al intruso

6.1 Los seis paneles y qué detecta cada uno

Panel	Detecta
Escala	Detalle que desaparece a 16 px; trazos que se emborronan
Silueta	Iconos irreconocibles sin color — el test más duro
Sobre terreno	Contraste insuficiente sobre algún fondo (calculado, con aviso automático)
En contexto	Solapamientos, apiñamiento, ilegibilidad a escala real de juego
En dispositivo	Tamaño real en pantallas reales
Familia	El asset que «no es de la misma mano» — se ve al instante viendo los 55 juntos

El panel **Familia** es el más valioso: la incoherencia visual es invisible mirando un asset y evidente mirando cincuenta.

6.2 Comprobaciones automáticas

Ejecutadas en CI (`npm run assets:check`), fallan el build:

Guerra de Cenizas — Documentación de diseño

112

- ✓ viewBox = "0 0 24 24"
- ✓ ancho de trazo ∈ {1, 2}
- ✓ sin colores literales fuera de la paleta
- ✓ sin <text>
- ✓ sin <image>, <foreignObject>, <script>
- ✓ ≤ 12 nodos de dibujo
- ✓ ≤ 2 KB por archivo
- ✓ cabecera completa con original: true
- ✓ contraste ≥ 3:1 sobre los 8 terrenos ← calculado, no estimado
- ✓ silueta distinguible: distancia de Hamming ≥ 15 % contra cualquier otro asset
- ✓ total de assets ≤ 150 KB

La comprobación de **silueta distinguible** rasteriza cada asset a 16 px en negro sólido y compara todos los pares. Si dos iconos se parecen demasiado en pequeño, el jugador los confundirá en el mapa — y eso se detecta automáticamente, no en playtesting.

7. Criterios de aprobación

Un asset entra en `main` solo si:

- ☐ Pasa todas las comprobaciones automáticas (§6.2)
- ☐ Reconocible en silueta a 16 px
- ☐ Contraste ≥ 3:1 sobre los 8 terrenos
- ☐ Coherente en el panel Familia
- ☐ Distinguible de los demás en el panel Familia
- ☐ Legible en 360×640 en el panel En contexto
- ☐ Legible con el filtro de daltonismo (deuteranopía y protanopía)
- ☐ Cabecera con original: true y autor
- ☐ Añadido al manifiesto
- ☐ Captura del panel En contexto adjunta al PR

Los tres últimos son revisión humana; el resto es automático.

8. Efectos y animación

Sin sistema de partículas, sin sprites, sin librería de animación. Todo con CSS y SMIL/CSS sobre los mismos SVG:

Efecto	Implementación	Duración
Selección	Anillo con <code>stroke-dasharray</code> animado	continua, 2 s
Movimiento	Trazo que se dibuja (<code>stroke-dashoffset</code>)	400 ms
Combate	Destello + sacudida de la región	600 ms
Captura	Barrido de color desde el borde	500 ms
Anomalía	Distorsión del contorno con <code>filter: url(#rift)</code>	800 ms
Consagración	Pulso del Núcleo	continua, 3 s
Ceniza	40 partículas CSS, solo en menús	continua

Todo desactivable con `prefers-reduced-motion`, y el juego sigue siendo plenamente jugable e informativo sin ninguna animación (la información nunca está *solo* en el movimiento).

9. Marca

`assets/src/brand/`: logotipo (marca completa y símbolo), favicon, icono PWA, imagen de apertura. Mismas reglas, misma paleta, mismo repositorio.

El símbolo: **un círculo incompleto con una traza que lo atraviesa** — la ciudad plegada y la Ceniza que la corta. Legible a 16 px como favicon.

10. Versionado de assets

- Los assets **no llevan versión en el nombre**. Git es el versionado.
- `manifest.json` guarda el hash de cada asset: un cambio visual es detectable en el diff.
- Cambiar un asset **no** rompe partidas (no forman parte del estado).
- Los cosméticos desbloqueables (emblemas, paletas) sí se referencian por clave estable en `account_unlocks`: **esas claves no se renombran nunca**.

11. Plan (v0.9)

Paso	Entrega
1	<code>palette.json</code> + generación de tokens
2	Galería con los 6 paneles
3	<code>assets:check</code> en CI
4	Terrenos (8) — lo primero que se ve
5	Fuerzas (4)
6	Recursos (4)
7	UI (14)
8	Estados (10)
9	Anomalías (8) y diplomacia (6)
10	Marca
11	Pase de coherencia sobre el panel Familia
12	Auditoría de contraste y daltonismo

Los pasos 1–3 (herramientas) van **antes** que ningún asset: la galería y las comprobaciones existen antes de que haya nada que comprobar, para que el primer asset ya nazca validado.

12 Testing y simulación

docs/TESTING_AND_SIMULATION.md

Versión: 1.0 · Implementación: `packages/core/tests/`, `packages/sim/`, `apps/web/e2e/` Cubre los apartados §27 (testing) y §28 (simulador) del brief.

1. Principio

El testing no es una fase. Es la condición para que una versión exista.

Ninguna versión se declara terminada sin sus tests (**definición de hecho**). Y hay tres tests que son **bloqueantes absolutos**: si fallan, no hay release, sin excepción.

Test bloqueante	Qué protege
<code>security/rls</code>	Que la niebla de guerra sea real y nadie lea lo que no debe
<code>progression/no-power-creep</code>	Que la metaprogresión no rompa el competitivo
<code>mapgen/fairness-sweep</code>	Que la promesa de «mapas equilibrados» sea cierta

Cada uno convierte una **afirmación de producto** en una **aserción ejecutable**. Esa es la idea que organiza todo este documento.

2. Pirámide



El grueso está abajo a propósito: **el motor es lógica pura sin I/O**, así que un test unitario del motor es un test real del juego, no un test de andamiaje. 350 tests unitarios corren en menos de 3 segundos.

3. Tests unitarios (`packages/core`)

Cobertura exigida en `src/rules/` : **≥ 90 %** de ramas.

Módulo	Qué se prueba	Ejemplos de casos límite
<code>rng</code>	Reproducibilidad, distribución	Misma semilla en Node y en Chromium
<code>combat</code>	Fórmula, rueda, terreno, posturas	Poder 0 vs 0 · empate exacto · composición monoarma · Pantalla que se retira sin destino válido
<code>movement</code>	Adyacencia, división, cruces	Cruce mutuo · mover a región cortada por Fisura ese mismo turno · dividir dejando 0
<code>economy</code>	Renta decreciente, suministro	0 regiones · 40 regiones · fuerza a 6 saltos sin Bastión · desbordamiento del tope
<code>control</code>	Captura, disputa	Empate ⇒ Disputada · Línea destruida pero región conservada · captura con Cielo (prohibida)
<code>diplomacy</code>	Ciclo de vida, ruptura, depósito	Romper sin ♦ suficiente ⇒ rechazo · doble aceptación · caducidad el mismo turno de la ruptura
<code>core</code> (Núcleo)	Consagración, reinicio, coste	Perder el Núcleo en el 3.er turno · no poder pagar · Coalición cuyo socio abandona
<code>anomalies</code>	Los 8 efectos, usos	Fisura que desconectaría el grafo ⇒ prohibida · Pliegue a región perdida ese turno
<code>shade</code>	6 operaciones, contrainteligencia	Interceptar a quien tiene contrainteligencia · Sembrar contra quien tiene Eco
<code>visibility</code>	Proyección por asiento	Ningún dato oculto aparece en <code>PlayerView</code> ← ver §3.1
<code>progression</code>	Depósito, desbloques	Regla de oro (§5)

3.1 El test de fuga de información

El más importante de los unitarios, y el que hace que la niebla de guerra sea confiable:

```
test('PlayerView no filtra nada oculto', () => {
  const { state, views } = reduce(fixture, orders, ctx);

  for (const [seat, view] of Object.entries(views)) {
    const leaked = deepFindValues(view, v => isSecretOf(state, v, Number(seat)));
    expect(leaked).toEqual([]);      // recursos ajenos, órdenes ajenas, fuerzas en niebla,
                                    // términos de tratados de terceros, catálogo de anomalías ajeno
  }
});
```

Recorre la vista serializada entera y comprueba que **ningún valor secreto aparece en ningún sitio** — incluidos campos que alguien pudiera añadir en el futuro sin pensar. Es un test que protege contra errores que aún no se han cometido.

4. Tests de integración

Contra Supabase local (`npx supabase start`).

Grupo	Casos
Auth	Registro, login, magic link, logout, sesión expirada, cambio de idioma
RLS ← bloqueante	Los 7 casos de TECHNICAL DESIGN §6.4
Persistencia	Guardar/cargar estado, borrador de órdenes, reconexión desde otro dispositivo
Concurrencia	10 peticiones de resolución simultáneas ⇒ exactamente 1 resolución · dos jugadores enviando en el mismo milisegundo · resolución justo al vencer el plazo
Ciclo de partida	Crear → 5 se unen → parlamento → 12 turnos → resultados → archivado
Ausencias	3 turnos sin enviar ⇒ Mando Automático · recuperar el asiento · el bot honra los Sellos
Reproducibilidad	Reproducir una partida desde <code>(seed, órdenes)</code> ⇒ checksum idéntico

5. Tests E2E (Playwright)

12 escenarios, cada uno en **360×640** y **1440×900**:

1. Registro → Ciudad → campaña
2. Crear partida privada → compartir código → 5 clientes entran
3. Parlamento: desplegar y sellar
4. Turno completo: mover, producir, enviar
5. Combate: previsualizar y confirmar
6. Diplomacia: componer y enviar una oferta en ≤ 4 taps ← medido
7. Aceptar una oferta y verificar el efecto tras la resolución
8. Romper un Sello y verificar que los 5 asientos ven el evento
9. Consagración: declarar, 3 turnos, victoria
10. Reconexión: recargar a mitad de turno sin perder el borrador
11. Cambio de idioma ES↔EN sin perder el estado
12. Accesibilidad: recorrer y jugar un turno completo solo con teclado

El escenario 6 tiene una **aserción de esfuerzo**, no solo de funcionamiento: falla si la oferta requiere más de 4 interacciones. Es la forma de impedir que el riesgo D2 (fricción diplomática en móvil) reaparezca por acumulación de cambios.

6. El simulador de balance

6.1 Qué es

`packages/sim` juega partidas completas **sin interfaz, sin red y sin base de datos**, usando el mismo `reduce()` que el servidor. Miles de partidas por minuto.

```
npm run sim -- --games 5000 --players 5 --seed 1 --out reports/balance.json
npm run sim -- --games 500 --players 5 --profiles aggressive,turtle,trader,opportunist,bot
npm run sim -- --replay <gameId> # reproduce una partida real y la analiza
```

```
simulateGames({
  games: 5000,
  players: 5,
  seed: 1,
  strategyProfiles: ['aggressive','turtle','trader','opportunistic','diplomat'],
  doctrines: 'random',          // o fijas, para comparar
  collect: ['winrate','duration','leadChanges','betrayals','attunements','resourceUse'],
});
```

6.2 Perfiles de estrategia

Cada perfil es una política que decide órdenes a partir de una `PlayerView` — es decir, **juega con la misma información que un humano**, no con el estado completo. Un bot que hiciera trampa invalidaría todas las conclusiones de balance.

Perfil	Comportamiento
<code>aggressive</code>	Ataca al vecino más débil, expande sin parar, ignora la diplomacia
<code>turtle</code>	Fortifica, no ataca, acumula, va a Reclamación Menor
<code>trader</code>	Maximiza ♦ comerciando; acepta casi toda oferta razonable
<code>opportunistic</code>	Acepta Sellos y los rompe cuando el beneficio > coste
<code>diplomat</code>	Busca Coalición pronto, honra todo, comparte visión
<code>rusher</code>	Va al Núcleo desde el T1
<code>random</code>	Órdenes legales al azar — el control : debe perder siempre
<code>autocommand</code>	El bot del juego real

`random` es el test de cordura del sistema: si un jugador aleatorio gana más del 6 % en partidas de 5, hay algo profundamente roto.

6.3 Modelo de diplomacia de los bots

El problema difícil: no se puede simular una negociación humana. La aproximación:

- Cada bot valora una oferta con una función de utilidad simple (¿me acerca al ♦ que necesito? ¿me protege un flanco?).
- Los perfiles difieren en **umbral de aceptación** y **umbral de traición**.
- `opportunistic` rompe si `beneficio_estimado > coste_ruptura × factor_riesgo`.

Limitación reconocida honestamente: esto mide si la *aritmética* de la diplomacia funciona (¿está bien tarifada la traición? ¿es alcanzable la consagración?), **no** si la diplomacia es *divertida*. Eso solo lo dice el playtesting con humanos (v0.95). El simulador detecta que un sistema está roto; no detecta que está muerto.

6.4 Métricas

Métrica	Objetivo	Qué revela si falla
Winrate por doctrina	18-22 %	Doctrina dominante o inútil
Winrate por asiento	19-21 %	El mapa no es justo
Winrate por perfil económico	19-21 %	Un perfil de recursos es superior
Winrate de <code>random</code>	< 6 %	Las decisiones no importan
Duración media	9-12 turnos	Partidas que se deciden pronto
Victorias antes del T8	< 15 %	Snowball
Reclamación Menor	25-40 %	El Núcleo es inalcanzable o trivial
Cambios de líder	≥ 2,0	Partidas sin tensión
Corr(regiones@T6, victoria)	< 0,45	Se ha convertido en un juego de acumular territorio
Consagraciones en solitario que triunfan	< 25 %	El pilar P1 está roto
Partidas con ≥ 1 ruptura	> 40 % (con <code>opportunistic</code>)	La traición está mal tarifada
Uso de cada anomalía	> 5 % cada una	Anomalías muertas
Uso de cada nodo de investigación	> 10 % cada uno	Nodos muertos

6.5 Informe



Los avisos son **automáticos**. El informe no es un volcado de datos: es un diagnóstico.

6.6 Barrido de constantes

Como las constantes de balance son **datos** y no código (**GDD Apéndice A**), se pueden barrer:

```
npm run sim:sweep -- --param combat.counterK --range 0.20:0.50:0.05 --games 1000
```

counterK	winrate máx-mín	corr(territorio,victoria)	duración
0.20	6.1 pp	0.51	11.2
0.25	4.8 pp	0.47	10.9
0.30	3.9 pp	0.42	10.6
0.35	3.1 pp	0.38	10.4 ← elegido
0.40	3.4 pp	0.35	9.8
0.45	5.2 pp	0.31	9.1
0.50	7.7 pp	0.28	8.4

Recomendación: 0.35 – minimiza la dispersión de winrate manteniendo corr(territorio,victoria) por debajo del umbral de 0.45.

El balance deja de ser una opinión. Cada constante de `BALANCE` debe tener un barrido que justifique su valor, y ese barrido se guarda en `reports/`.

6.7 Tests de emergencia diplomática

Los más difíciles y los más importantes: comprueban que el juego **produce las situaciones que promete** ([DIPLOMACY §1.1](#)).

Test	Aserción
<code>alliance-forms-against-leader</code>	Cuando alguien declara Consagración, ≥ 60 % de las partidas registran ≥ 2 tratados nuevos contra él en los 2 turnos siguientes
<code>betrayal-is-priced</code>	<code>opportunist</code> traiciona más que <code>diplomat</code> , pero su winrate no es superior
<code>trade-is-necessary</code>	Un bot que nunca comercia consagra en < 20 % de sus partidas
<code>weak-player-relevance</code>	El jugador con menos territorio en el T6 gana ≥ 8 % de las veces
<code>coalition-is-a-real-choice</code>	La Coalición se declara en 20-45 % de las partidas de 5
<code>no-runaway</code>	El líder del T6 gana < 45 % de las veces

`trade-is-necessary` es la validación directa del pilar P1: si un bot autárquico ganase con frecuencia, la premisa entera del juego sería falsa y habría que subir el coste de consagración.

7. Tests de mapas

Ver [MAP_GENERATION §11](#). Destacado: `fairness.sweep` corre 1 000 semillas $\times$ 3 conteos de jugadores en cada CI y exige **F ≤ 1.0 siempre**.

8. Tests de rendimiento

Test	Presupuesto
<code>reduce()</code> de un turno de 5 jugadores	≤ 15 ms
Generar + validar un mapa	≤ 250 ms p95
Bundle de la ruta de partida	≤ 180 KB gzip
Assets totales	≤ 150 KB
LCP móvil (Lighthouse CI)	≤ 2,5 s
INP al tocar una región	≤ 100 ms

Fallan el build. Un presupuesto que no se verifica automáticamente no es un presupuesto.

9. Tests de viewport móvil

```
const VIEWPORTS = [  
  { name: 'iPhone SE', width: 320, height: 568 }, // el más estrecho soportado  
  { name: 'Android S', width: 360, height: 640 }, // referencia de diseño  
  { name: 'iPhone 14', width: 390, height: 844 },  
  { name: 'iPad', width: 768, height: 1024 },  
  { name: 'Desktop', width: 1440, height: 900 },  
];
```

En cada uno, automáticamente:

- ✓ sin desbordamiento horizontal
- ✓ ningún táctil < 44 px (medido con `getBoundingClientRect`)
- ✓ ningún texto < 14 px (medido con `getComputedStyle`)
- ✓ el mapa ocupa ≥ 55 % de la altura
- ✓ el botón de enviar turno está en el tercio inferior
- ✓ contraste AA (axe-core)

10. CI

Momento	Qué corre	Duración
Cada push	typecheck · lint · unitarios · determinismo	~2 min
Cada PR	+ integración · RLS · E2E · presupuestos · mapgen (200 semillas)	~9 min
Nocturno	+ 5 000 partidas simuladas · mapgen (5 000 semillas) · Lighthouse	~35 min
Pre-release	Todo + checklist manual de QA móvil	~1 h

El nocturno **abre una issue automáticamente** si una métrica de balance sale de rango, con el informe adjunto. El balance se vigila solo.

11. Datos de prueba

- **Fixtures de estado:** 12 situaciones canónicas (apertura, contacto, cerco, consagración en curso, jugador reducido, etc.) en `packages/core/tests/fixtures/`.
- **Semillas de oro:** 20 semillas con informe de equidad congelado. Si el generador cambia su salida para una de ellas sin subir `MAPGEN_VERSION`, el test falla.
- **Partidas grabadas:** partidas reales de la beta guardadas como `(seed, órdenes)`, que se reproducen en CI para detectar regresiones del motor contra el juego real.

Ese último punto es una consecuencia gratuita del determinismo: **cada partida jugada por un humano se convierte, sin coste, en un test de regresión.**

12. Qué NO se testea

Honestidad sobre los límites:

No se testea	Por qué
Que el juego sea divertido	No es automatizable. Playtesting en v0.95.
Que la diplomacia sea interesante	Los bots negocian con aritmética, no con psicología (§6.3)
Estética de los assets	Revisión humana en la Galería
Colusión externa	Fuera del sistema
Carga real con miles de usuarios	Fuera de alcance en beta; se documentan los límites del free tier
Compatibilidad con navegadores antiguos	Objetivo: navegadores de las últimas 2 versiones

13 Despliegue

docs/DEPLOYMENT.md

Cómo se pone esto en producción y cómo se comprueba que quedó bien. Todo con capas gratuitas: Vercel Hobby + Supabase Free.

Índice

1. Lo que hay que crear
2. Supabase 2bis. El pipeline
3. Vercel
4. El reloj
5. Comprobación posterior
6. Lo que puede salir mal
7. Presupuesto del free tier

1. Lo que hay que crear

Servicio	Plan	Para qué
Supabase	Free	Postgres, Auth, Realtime, <code>pg_cron</code>
Vercel	Hobby	Next.js, Route Handlers

Nada más. Sin Redis, sin cola de mensajes, sin servidor de sockets: el estado vive en Postgres y el empuje lo hace Realtime sobre las mismas tablas.

2. Supabase

2.1 Migraciones

Las aplica el pipeline (§2 bis) en cada cambio bajo `supabase/` que llegue a `main`. A mano, para un proyecto nuevo o para depurar:

```
npx supabase link --project-ref TU-REF
npx supabase db push
```

Se aplican en orden alfabético. `0007_schedule.sql` se salta solo si `pg_cron` no está disponible, así que no hace falta tratarlo aparte.

Comprobar que la niebla de guerra quedó puesta:

```
select tablename, rowsecurity from pg_tables
where schemaname = 'public' order by tablename;
```

Todas deben tener `rowsecurity = true`. Y `game_states` debe tener **cero** políticas:

```
select count(*) from pg_policies
where schemaname = 'public' and tablename = 'game_states'; -- 0
```

Si ese número no es cero, alguien añadió una política a la tabla que nadie puede leer y la niebla de guerra acaba de dejar de existir.

2.2 Auth

Esto ya no se toca en el panel. Vive en `supabase/config.toml` y lo empuja el pipeline con `supabase config push`. El panel es el resultado, no la fuente: un cambio hecho a mano allí se pierde en el siguiente despliegue.

Ajuste	Valor	Por qué
<code>site_url</code>	el dominio de producción	Es la lista blanca de redirecciones y el destino de respaldo
<code>additional_redirect_urls</code>	callback de producción, comodín de preview, <code>localhost</code>	Los preview de Vercel cambian de dominio en cada rama
<code>enable_anonymous_sign_ins</code>	<code>true</code>	El modo invitado
<code>enable_confirmations</code>	<code>true</code>	Ya estaba así antes de versionarlo

Este ajuste fue un fallo real y silencioso. El correo de alta llegaba con `redirect_to=http://localhost:3000`: confirmar la cuenta mandaba a una máquina que no existe. La causa no estaba en el código — Supabase **solo acepta un `redirect_to` que esté en su lista blanca**, y cuando no lo está lo sustituye sin avisar por la Site URL, que de fábrica es `http://localhost:3000`. No hay error en ningún log, no hay test que lo pueda cazar y el enlace parece correcto. Por eso la configuración está ahora en el repositorio: para que la próxima vez el fallo sea imposible en lugar de invisible.

2.3 Realtime

Panel → Database → Replication: `player_views`, `games`, `messages` y `treaties` deben estar en la publicación `supabase_realtime`. Las migraciones lo hacen, pero conviene mirarlo: si falta `player_views`, el juego funciona y **nadie se entera de que su turno ha resuelto** hasta que recarga.

`game_states` **no** está en la publicación, y eso es deliberado. Realtime respeta RLS, así que tampoco filtraría nada — pero una tabla que nadie puede leer no tiene por qué estar publicada.

2 bis. El pipeline

Todo lo de arriba —migraciones, ajustes de Auth y los secretos del reloj— lo aplica `.github/workflows/deploy.yml` cuando llega a `main` un cambio bajo `supabase/`. La aplicación la sigue desplegando Vercel por su cuenta; este workflow es la mitad que Vercel no toca.

Por qué automatizarlo y no dejarlo documentado. Los tres pasos manuales que sustituye comparten una propiedad desagradable: **no fallan ruidosamente**.

Si falta	Qué se ve	Qué pasa de verdad
Migraciones	la aplicación arranca	se cae al primer <code>select</code>
<code>site_url</code>	el correo llega y el enlace parece bueno	te manda a <code>http://localhost:3000</code>
Secretos del vault	<code>pg_cron</code> late, sin errores	llama con <code>Authorization</code> vacío, recibe 401, y las partidas con ausencias no avanzan

Ninguno de los tres lo puede cazar un test, porque ninguno vive en el repositorio.

Qué hace, en orden

1. Comprueba que están las seis piezas de configuración. Falla **antes** de tocar nada y dice cuál falta por su nombre — lo mismo que hace `/api/health` para la aplicación.
2. `supabase db push --dry-run`, para dejar en el log qué se va a aplicar.
3. `supabase db push`: migraciones **y** los secretos del vault, que salen de `[db.vault]` en `config.toml`.
4. `supabase config push`: `site_url`, la lista blanca de redirecciones y las sesiones anónimas.
5. Comprueba el resultado: que `pg_cron` tiene el trabajo programado, que el vault tiene sus dos secretos y que `game_states` **sigue con RLS activa y cero políticas**. Esta última para el despliegue si falla: es donde vive la niebla de guerra (**ADR-006**), y una política añadida por descuido no rompe ningún test — solo enseña la partida entera a todo el mundo.

Lo que hay que configurar una vez

En **Settings → Environments → `production`**:

	Nombre	Valor
Secret	<code>SUPABASE_ACCESS_TOKEN</code>	token personal, panel de Supabase → Account → Access Tokens
Secret	<code>SUPABASE_DB_PASSWORD</code>	la contraseña de la base de datos
Secret	<code>CRON_SECRET</code>	el mismo valor que <code>CRON_SECRET</code> en Vercel
Variable	<code>SUPABASE_PROJECT_REF</code>	la referencia del proyecto
Variable	<code>GDC_SITE_URL</code>	<code>https://TU-APP.vercel.app</code> , sin barra final
Variable	<code>GDC_PREVIEW_URL_PATTERN</code>	<code>https://TU-APP-*.vercel.app/**</code>

`CRON_SECRET` tiene que coincidir en los dos sitios. Son los dos extremos del mismo secreto compartido: si no coinciden, el reloj queda parado y no lo dice nadie.

Verificación

`.github/workflows/verify.yml` ejecuta `npm run verify` en cada pull request. Es el mismo comando de siempre; lo que no había era quien lo ejecutara, así que las reglas bloqueantes del proyecto —los tests de RLS, la regla de oro de la metaprogresión, `core` sin dependencias— dependían de que alguien se acordara. Una regla bloqueante que depende de la memoria no es bloqueante.

Añade un paso más al final: buscar la clave de servicio en el bundle compilado. `check:deps` mira el código fuente; esto mira lo que de verdad se sirve al navegador.

3. Vercel

3.1 Configuración del proyecto

Root Directory = `apps/web`, con «Include source files outside of the Root Directory in the Build Step» **activado**. Los dos ajustes, no uno.

Ajuste	Valor
Framework Preset	Next.js — compruébalo, no des por hecho que se detectó
Root Directory	<code>apps/web</code>
Include files outside Root Directory	☒ obligatorio
Build Command	Override → <code>npm run build</code>
Install command	por defecto — Vercel detecta los workspaces e instala en la raíz

El Framework Preset no se corrige solo. La detección ocurre **una vez, al crear el proyecto**, mirando el Root Directory de ese momento. Si el proyecto se creó con el Root Directory vacío, Vercel miró la raíz del repositorio, no encontró Next y guardó el preset como «**Other**» — y cambiar el Root Directory después **no vuelve a lanzarla**. El preset se queda como estaba, para siempre, hasta que alguien lo cambia a mano.

El Build Command hay que forzarlo. Con el preset de Next, Vercel puede ejecutar `next build` directamente, y eso **se salta el prebuild**: vuelve el `Module not found: './art/generated'`. Escribiendo `npm run build` en el Override se ejecuta el `prebuild` sí o sí.

De dónde sale el preset «Other». Vercel detecta el framework mirando el Root Directory: busca ahí `next` entre las dependencias y un `next.config.*`. En la raíz de este repositorio no hay ninguna de las dos cosas —es solo el nodo de workspaces, sin una sola dependencia de runtime—, así que si el proyecto se creó sin Root Directory la detección falló y quedó «**Other**», que espera una carpeta `public/`. De ahí este error, que no menciona Next por ninguna parte:

```
No Output Directory named "public" found after the Build completed.
```

Es engañoso: la compilación **ha ido bien**. Lo que falla es que Vercel nunca supo que esto era una aplicación de Next.

Y ojo: **arreglar el Root Directory no arregla el preset**. Son dos ajustes distintos en la misma pantalla, y el segundo se queda en «Other» hasta que se cambia a mano. Es el error que más tiempo costó de los tres.

Y no se arregla desde el repositorio. Un `vercel.json` con `framework` y `outputDirectory` **no vale**: para Next.js, Vercel usa la Build Output API y el ajuste de directorio de salida no se aplica. Se intentó y falló. Hay que fijar el Root Directory.

Por qué hace falta la casilla. Dos cosas del build viven fuera de `apps/web`: `packages/core`, porque el motor se consume como fuente TypeScript sin compilar ([ADR-022](#)), y `tools/assets/`, que genera los componentes de arte en el `prebuild`. Sin la casilla, ninguno de los dos llega a la máquina de build.

El prebuild. `npm run build` genera los assets antes de compilar. Están en `.gitignore` porque son un artefacto, así que un clon limpio no los trae y sin ese paso la compilación falla con `Module not found: './art/generated'`.

3.2 Variables de entorno

Copiar de `.env.example`:

Variable	Entorno	Notas
<code>NEXT_PUBLIC_SUPABASE_URL</code>	todos	pública
<code>NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY</code>	todos	pública, RLS la contiene
<code>SUPABASE_SECRET_KEY</code>	todos	⚠ nunca con prefijo <code>NEXT_PUBLIC_</code>
<code>CRON_SECRET</code>	todos	cadena larga y aleatoria; el mismo valor que en el pipeline
<code>NEXT_PUBLIC_SITE_URL</code>	producción	el dominio público, sin barra final

`NEXT_PUBLIC_SITE_URL` es la mitad del cliente del arreglo del enlace a localhost: es la URL que se pide como `redirect_to`. La otra mitad —que Supabase la acepte— la pone `config.toml` (§2.2). Si se omite, se usa el dominio de producción que Vercel inyecta sola, que casi siempre es el correcto; declararla explícitamente evita el «casi».

Las dos generaciones de claves de Supabase valen. El panel sugiere hoy `sb_publishable_...` y `sb_secret_...`; los proyectos anteriores tienen JWT (`eyJ...`) bajo los nombres `NEXT_PUBLIC_SUPABASE_ANON_KEY` y `SUPABASE_SERVICE_ROLE_KEY`. Cambia el nombre, no el rol —la publicable sigue siendo `anon` y la secreta `service_role`—, así que `apps/web/lib/server/env.ts` acepta los cuatro nombres y prefiere el nuevo si están los dos. No hace falta tocar ninguna política de RLS.

La clave secreta omite RLS: con ella se lee `game_states` entero. Si acabara en el bundle de cliente, la niebla de guerra sería decorativa para cualquiera que abra las herramientas de desarrollo. Hay una regla estructural (`npm run check:deps`) que falla si cualquiera de sus dos nombres aparece fuera de `apps/web/lib/server/`, y un test que comprueba que ninguno lleva prefijo `NEXT_PUBLIC_`.

Si una clave se expone —un pantallazo, un chat, un log—, se rota. Panel → *API Keys* → *Rotate*, y se actualiza el valor en Vercel. Rotar la secreta es especialmente urgente: quien la tenga lee el estado completo de todas las partidas con una petición HTTP.

4. El reloj

Vercel Hobby permite **un cron diario**, lo que no sirve para turnos de tres minutos. El reloj vive en Supabase (ADR-014): `pg_cron` llama cada minuto a `/api/cron/resolve-due` a través de `pg_net`.

Los dos secretos los pone el pipeline (§2 bis) desde `[db.vault]` en `config.toml`, y comprueba después que están. A mano, si hiciera falta, desde el SQL Editor:

```
select vault.create_secret('https://TU-APP.vercel.app/api/cron/resolve-due',
                          'gdc_resolve_url');
select vault.create_secret('EL-MISMO-VALOR-QUE-CRON_SECRET-EN-VERCEL',
                          'gdc_cron_secret');
```

Los secretos van al *vault* y no dentro de la función porque el cuerpo de una función de `cron.job` es legible desde el catálogo.

Comprobar que late:

```
select jobname, schedule, active from cron.job where jobname = 'gdc-resolve-due';
select status, return_message, start_time
from cron.job_run_details order by start_time desc limit 10;
```

El reloj es el segundo de tres disparadores, no el único. El 85 % de los turnos los resuelve el último jugador que envía sus órdenes, y cualquier petición de cliente que detecte un plazo vencido lo resuelve también. Que `pg_cron` falle un rato ralentiza las partidas con ausencias; no las bloquea.

5. Comprobación posterior

Empieza por aquí. Una sola petición dice si el despliegue está configurado:

```
curl -s https://TU-APP.vercel.app/api/health
```

```
{ "ok": true, "missing": [], "engine": "0.2.0", "mapgen": "0.1.0" }
```

Si devuelve **503**, `missing` trae los nombres de las variables que faltan. Devuelve nombres y nunca valores — y esos nombres ya están en `.env.example`, así que no revela nada que no esté en el repositorio.

Existe porque costó una tarde: con una variable sin definir, la aplicación respondía `Internal Server Error` y nada más. El mensaje explícito estaba en el log de una función que hay que saber buscar.

Y después, antes de invitar a nadie:

- ☐ `/api/health` responde 200 con `missing` vacío
- ☐ Un correo de acceso llega y el enlace entra directo a la ciudad
- ☐ La ciudad se dibuja con el emblema de tu facción y es la MISMA al recargar
- ☐ Buscar campaña deja la cuenta en cola; se ven las ciudades pendientes parpadear
- ☐ Una segunda cuenta buscando lo mismo forma la partida y AMBAS entran solas ← Realtime
- ☐ Cada asiento ve un mapa distinto ← niebla
- ☐ `GET /rest/v1/game_states` con la clave anónima devuelve 401 o vacío ← RLS
- ☐ Enviar órdenes desde todos los asientos resuelve el turno al instante
- ☐ Dejar vencer un plazo sin enviar resuelve el turno igual ← `pg_cron`
- ☐ Una cuenta sola en cola acaba emparejada con Mando Automático ← `pg_cron`
- ☐ `cron.job_run_details` no acumula errores

La quinta y la sexta son las que de verdad importan. Si `game_states` responde algo con la clave anónima, **hay que parar el despliegue**: el resto del juego funciona igual con la niebla rota, y por eso nadie lo notaría.

```
# Debe devolver 401 o una lista vacía. Nunca filas.
curl -s "$SUPABASE_URL/rest/v1/game_states?select=*" \
  -H "apikey: $ANON_KEY" -H "Authorization: Bearer $ANON_KEY"
```


6. Lo que puede salir mal

Síntoma	Causa probable
El enlace del correo lleva a un error de Supabase	Falta la URL en <i>Redirect URLs</i>
El enlace del correo lleva a <code>http://localhost:3000</code>	<code>site_url</code> sin configurar. Supabase sustituye en silencio un <code>redirect_to</code> que no esté en la lista blanca por su Site URL, y de fábrica es localhost. Lo arregla <code>config.toml</code> + el pipeline (§2.2)
«Entrar sin cuenta» no hace nada	<code>enable_anonymous_sign_ins</code> en <code>false</code> . Está en <code>config.toml</code> ; lo empuja el pipeline
Entras y vuelves a la pantalla de acceso, en bucle	Usuario en <code>auth.users</code> sin fila en <code>profiles</code> . Lo crea un trigger desde la migración <code>0009</code> ; si la base es anterior, aplicar migraciones
El turno resuelve pero nadie se entera hasta recargar	<code>player_views</code> no está en <code>supabase_realtime</code>
<code>Internal Server Error</code> en todo el sitio	Falta alguna variable de entorno. Consulta <code>/api/health</code> : dice cuáles en una petición. Falla a propósito — un servidor a medias que responde 500 en mitad de una partida es mucho peor de depurar
Los turnos vencidos no resuelven	Secretos del <i>vault</i> mal puestos, o <code>CRON_SECRET</code> distinto entre Vercel y Supabase
Build en Vercel: no encuentra <code>@gdc/core</code>	Falta <code>--include-workspace-root</code> en el <i>install</i>
Build en Vercel: <code>Module not found: './art/generated'</code>	El <i>build command</i> no pasa por <code>npm run build</code> . Los componentes de arte son un artefacto generado y no están versionados; los crea el <code>prebuild</code>
<code>No Output Directory named "public" found</code>	Framework Preset en «Other» , que espera <code>public/</code> . La compilación fue correcta. Corregir el Root Directory no cambia el preset: hay que ponerlo a Next.js a mano. Ver §3.1
Vuelve <code>Module not found: './art/generated'</code> tras poner el preset de Next	Vercel está ejecutando <code>next build</code> en vez de <code>npm run build</code> , así que se salta el <code>prebuild</code> . Forzar el <i>Build Command</i> a <code>npm run build</code>
Se entra a la ciudad pero buscar campaña no hace nada	<code>matchmaking_queue</code> sin migrar, o <code>/api/match</code> devolviendo 500. Mirar los logs de la función
Una partida se queda «resolviendo»	Un arrendamiento colgado. Vence solo en 30 s (ADR-025); si persiste, mirar los errores de <code>/api/cron/resolve-due</code>

7. Presupuesto del free tier

Recurso	Límite	Consumo estimado
Postgres	500 MB	≈ 80 KB por partida terminada ⇒ ~6 000 partidas
Realtime	200 conexiones concurrentes	≈ 40 partidas de 5 a la vez
Peticiones de Vercel	100 GB-h	Muy por debajo: casi todo son lecturas con RLS
<code>pg_cron</code>	sin límite práctico	1 llamada/minuto

El punto de ruptura conocido son las **200 conexiones de Realtime** (DISCOVERY T7). Con cadencia Diaria casi nadie está conectado a la vez, así que el límite solo aprieta en Blitz. Cuando se acerque, la salida documentada es agrupar canales por partida en vez de por asiento.

14 Roadmap

docs/ROADMAP.md

Versión: 1.2 · Estado actual: **v0.2 completada y verificada**. Siguierte: v0.3. Cada versión debe ser **jugable**, tener **criterios de aceptación, tests, documentación actualizada y entrada en CHANGELOG**.

Definición de hecho

Una tarea, feature o versión está terminada **solo** cuando:

- ☐ Código implementado
- ☐ Tests escritos
- ☐ Tests ejecutados y en verde
- ☐ UX comprobada en 360×640
- ☐ Persistencia comprobada (cuando aplique)
- ☐ Errores manejados (nada que lance sin control)
- ☐ Sin TODOs críticos sin registrar en una issue
- ☐ Documentación actualizada
- ☐ CHANGELOG.md actualizado
- ☐ DECISIONS.md actualizado si se cambió una decisión

Y para una **versión**, además:

- ☐ Sus criterios de aceptación se cumplen y se han verificado uno a uno
- ☐ Las versiones anteriores siguen funcionando
- ☐ CI en verde, incluidos los tres tests bloqueantes
- ☐ Tag creado

Regla de secuencia

Cada versión se apoya en la anterior y **ninguna deja deuda para la siguiente**. Si una versión no cumple sus criterios, **no se pasa a la siguiente**: se termina.

v0.1 → v0.2 → v0.3 → v0.4 → v0.5 → v0.6 → v0.7 → v0.8 → v0.9 → v0.95 → v1.0
 motor juego online diplo Núcleo meta anom. balance móvil beta release

v0.1 — Prototipo completada

Objetivo: demostrar que el modelo de mapa y de turno funciona. Sin cuentas, sin red.

Alcance

- `packages/core`: tipos de estado, PRNG determinista, `reduce()` con las etapas de movimiento, control y cierre.
- `packages/core/mapgen`: esqueleto C_n + decoración + replicación. **Sin** perturbación ni evaluación todavía.
- `apps/web`: una única ruta que renderiza el mapa en SVG y permite jugar **en local** (hot seat) con 2, 3 o 5 asientos.
- Mover fuerzas, terminar turno, resolución simultánea.

Fuera: auth, base de datos, combate, recursos, diplomacia, assets finales.

Criterios de aceptación — todos verificados

- ✓ Generar un mapa para 2, 3 y 5 jugadores desde una semilla
- ✓ La misma semilla produce siempre el mismo mapa (checksum)
- ✓ Los 5 invariantes del esqueleto se verifican con test
- ✓ Mover una fuerza entre regiones adyacentes
- ✓ 5 asientos dan órdenes y se resuelven simultáneamente
- ✓ El mapa es legible y tocable en 360×640
- ✓ `reduce()` es puro (test de inmutabilidad)
- ✓ npm test en verde – 114 tests

Entregado además de lo previsto

- Sistema de **facciones ligado a la cuenta** (**FACTIONS**, ADR-021), con el invariante del techo verificado por test. Se adelantó porque define el modelo de datos de la cuenta, del que dependen v0.3 y v0.6.
- `CLAUDE.md` por directorio con las reglas locales de cada paquete.
- `npm run check:deps`: reglas estructurales del monorepo verificadas en CI.

Dos hallazgos que cambiaron el diseño

1. **Fallo del PRNG con impacto en la equidad.** `xoshiro128**` cerraba con `& 0xffffffff`, y en JavaScript los operadores de bits son de 32 bits **con signo**: devolvía negativos por encima de 2^{31} , `shuffle` indexaba en negativo y dejaba huecos, y los mapas salían con **2 yacimientos por sector en vez de 3**. Un fallo de una línea que rompía la promesa central del proyecto. Lo detectó el test «todos los sectores tienen el mismo inventario», no una revisión visual.
2. **Un mapa de 5 jugadores no cabe entero en 360 px con regiones tocables.** Medido: a escala 1 cada región mide 21 px. Registrado en **UX_MOBILE §1.4**; el mapa entra con el zoom calculado del ancho real del viewport y centrado entre Bastión y Núcleo.

Deuda consciente que hereda v0.2

- Sin combate: dos asientos con Línea en la misma región dejan la región **disputada**; el combate se resolverá en la etapa 6 del pipeline sin tocar las demás etapas.
- La segunda fuerza se despliega automáticamente; en v0.2 la elige el jugador en el Parlamento, que es donde debe estar esa decisión.
- Textos en español dentro de `apps/web/lib/theme.ts`. Centralizados a propósito en un único módulo para que el paso a next-intl en v0.9 sea mecánico.

Tests: `rng` (10), `mapgen` (41), `factions` (27), `reduce` (36).

v0.2 — Núcleo jugable ✓ completada

Objetivo: una partida completa de 12 turnos con sustancia, todavía en local.

Alcance

- Los 4 recursos, renta con rendimiento decreciente, suministro por distancia.

- Producción (Línea, Fuego, Cielo, fortificación).
- Combate determinista completo: rueda, terreno, posturas, apoyo de Fuego.
- Captura, regiones disputadas, tipos de región.
- **Log de eventos con visibilidad por jugador** ← se adelanta desde v0.4 a propósito: sin él, la diplomacia no se puede construir después (**DISCOVERY §4**).
- Previsualización de combate en la UI.

Criterios de aceptación — todos verificados

- ✓ Partida de 12 turnos jugable de principio a fin en local
- ✓ La previsualización coincide exactamente con el resultado (test exhaustivo sobre 6 terrenos × 3 posturas × 3 niveles de fortificación)
- ✓ Ninguna composición monoarma vence a todas las demás
- ✓ Una fuerza sin suministro se degrada como está documentado
- ✓ El log muestra a cada asiento solo lo que le corresponde
- ✓ npm test en verde – 163 tests

Hallazgos que cambiaron el diseño

1. **La constante de rendimiento decreciente estaba mal.** El GDD pedía que doblar el territorio diera $\sim 1,55\times$ de renta; con el `0.045` documentado daba **1,08x**, es decir, expandirse dejaba de compensar. Corregida a `0.015`. Lo detectó el test que fija la *intención* de diseño en vez del número.
2. **Un panel flotante no puede interceptar taps.** Reducir la hoja de región a una barra de 76 px no bastó: seguía habiendo regiones alcanzables debajo. La solución real es `pointer-events: none` en el panel y `auto` solo en sus controles. Tercera vez que este error muere en el proyecto; ya está en las lecciones de `CLAUDE.md`.
3. **Resaltar un destino que no se puede tocar es peor que no ofrecerlo.** Algunos destinos alcanzables quedaban fuera de pantalla. Al seleccionar una fuerza, el mapa encuadra ahora su vecindario.
4. **La mayoría de los combates son encuentros simultáneos**, donde la previsualización no puede existir porque nadie está allí todavía. Solo aparece al atacar una región ocupada y visible. Es coherente con el diseño, pero conviene tenerlo presente al escribir el tutorial: el jugador verá su primera previsualización hacia el turno 8.

Deuda consciente que hereda v0.3

- Sin alianzas: dos asientos en la misma región siempre combaten. La condición ya está aislada en `battle.ts` para que consultar tratados en v0.4 sea un cambio local.
- El apoyo de Fuego solo se puede prestar a fuerzas propias.
- La previsualización supone que el enemigo defiende en postura Firme; no puede saber su postura real, que se decide simultáneamente.

v0.3 — Multijugador real

Objetivo: cinco personas en cinco dispositivos, con persistencia y seguridad.

Alcance

- Supabase: Auth (magic link), esquema, migraciones, RLS.
- Autoridad de servidor: Route Handlers, resolución con advisory lock.
- `player_views` y niebla de guerra real.
- Realtime, reconexión, borradores de órdenes.
- Plazos, cadencias (**Blitz obligatorio** para poder testear), `pg_cron`, resolución oportunista.
- Órdenes Permanentes y Mando Automático.

- Lobby, códigos de invitación.
- Deploy en Vercel + Supabase.

Criterios de aceptación

- ☐ 5 personas juegan una campaña completa en dispositivos distintos
- * Los 7 tests de RLS pasan ← BLOQUEANTE · 43 tests
- 10 peticiones simultáneas de resolución → 1 sola resolución
- ☐ Cerrar la pestaña a mitad de turno no pierde el borrador · implementado, sin verificar
- ☐ Reconectar desde otro dispositivo restaura el estado en < 2 s
- Un turno vencido se resuelve sin ningún cliente conectado
- 3 turnos sin enviar → Mando Automático; el jugador recupera el asiento
- Una partida reproducida desde (seed, órdenes) da el mismo checksum
- ☐ Desplegado y accesible públicamente

Estado. El esquema, la RLS, la capa de autoridad y la resolución de turnos están hechos y **verificados contra un Postgres real** (85 tests en `apps/web/tests/`). La interfaz —entrada, lobby y tablero en red— está implementada y compila, pero **no se ha podido verificar ejecutándola**: hace falta un proyecto de Supabase con credenciales, y sin él no hay auth, ni Realtime, ni sesión. Los criterios que dependen de eso siguen marcados como pendientes a propósito: la **definición de hecho** exige haberlo ejecutado. La checklist de **DEPLOYMENT \$5** es exactamente lo que queda por pasar.

Esta es la versión de riesgo del proyecto. Concentra la seguridad, la concurrencia y la infraestructura. Conviene presupuestarla como la más larga.

v0.4 — Diplomacia

Ya construido en v0.3: el **reparto** (**ADR-040**) enseña regiones, yacimientos, Núcleo y frontera de cada asiento —todo público en el motor— y enciende su territorio en el mapa. Es el sitio donde entran los Sellos: lo que falta es poder **negociar** el reparto, no verlo.

Alcance

- Las 3 primitivas: Sello (5 tipos), Transferencia con depósito, base de Coalición.
- Compositor de ofertas por plantillas (móvil).
- Coste de ruptura, evento público, reputación de partida.
- Visión compartida, derecho de paso, derecho de Bastión.
- Chat público y privado con RLS.
- Transferencias condicionales (2 condiciones).

Criterios de aceptación

- ☐ Enviar una oferta completa en ≤ 4 taps y ≤ 8 s (test E2E medido)
- ☐ Romper un Sello cobra + y es visible para los 5 asientos
- ☐ Sin + suficiente, la ruptura se rechaza
- ☐ El depósito se entrega exacto o se devuelve íntegro
- ☐ Un jugador no puede leer un canal privado ajeno (test de seguridad)
- ☐ Las ofertas se muestran traducidas: ES y EN ven la misma oferta
- ☐ La visión compartida llega vía `player_views`, nunca desde el cliente

v0.5 — El Núcleo

Alcance

- Región Núcleo, activación en el T3, renta.
- Consagración: coste, contador de 3 turnos, reinicio, escalado por yacimientos perdidos.
- Revelación pública al declarar.
- Coalición completa (5 jugadores).
- Reclamación Menor y desempates.
- Rendición dirigida, Bastión sitiado, sin eliminación.
- Pantalla de resultados.

Criterios de aceptación

- ☐ Una campaña termina siempre: por Consagración, Coalición o Reclamación Menor
- ☐ Declarar Consagración revela al declarante a todos, en el mismo turno
- ☐ Perder el Núcleo reinicia el contador y no devuelve el +
- ☐ Una Coalición que consagra produce DOS vencedores
- ☐ Un empate exacto declara empate; no se desempata al azar
- ☐ Un jugador reducido a su Bastión conserva voz diplomática y renta
- ☐ Consagrar en solitario, sin pactos, falla en > 70 % de las simulaciones

Ese último criterio es la primera validación del pilar P1 del juego.

v0.6 — Metaprogresión

Alcance

- `match_results`, cálculo del depósito.
- `cities`, `account_unlocks`, RLS.
- Vista Ciudad, selección de equipo, distritos.
- Pantalla de Reposo, «Repetir campaña».

Criterios de aceptación

- ☐ * El test no-power-creep pasa ← BLOQUEANTE
- ☐ Una cuenta al 100 % y una cuenta vacía con el mismo equipo tienen winrate 48–52 % en 2 000 simulaciones
- ☐ No se puede llevar a campaña algo no desbloqueado
- ☐ Un jugador no puede modificar su propio `ash_bank` (test de seguridad)
- ☐ Entrar en campaña desde la Ciudad en ≤ 2 taps
- ☐ «Repetir campaña» reproduce la partida con checksum idéntico

v0.7 — Anomalías y Sombra

Alcance

- Las 8 anomalías, con las dos fases de resolución.

- Los 3 agentes de Sombra y sus 6 operaciones.
- Contrainteligencia, eventos falsos de *Sembrar*.
- Las 6 doctrinas con pasivo y activo.
- Investigación (3 tiers) y Yermo.

Criterios de aceptación

- ☐ Las 8 anomalías funcionan y respetan sus límites de uso
- ☐ Fisura no puede desconectar el grafo (test)
- ☐ Sembrar inserta un evento falso indistinguible en el log de la víctima
- ☐ Eco revela que un evento era falso
- ☐ Interceptar revela órdenes; la contrainteligencia lo bloquea y revela al agente
- ☐ Cada anomalía se usa en > 5 % de las partidas simuladas
- ☐ Winrate por doctrina dentro de 18–22 % en 5 000 partidas

v0.8 — Procedural y balance

Alcance

- Perturbación acotada, las 8 métricas de equidad, las 4 de interés.
- Bucle de aceptación e informe de equidad.
- Rotación de perfiles económicos.
- `packages/sim` completo: 8 perfiles, informe, barridos.
- **Ajuste de todas las constantes de `BALANCE` con barridos documentados.**

Criterios de aceptación

- ☐ * fairness.sweep: 1 000 semillas × {2,3,5} con $F \leq 1.0$ ← BLOQUEANTE
- ☐ ≥ 95 % de las semillas alcanzan $I \geq 0.45$ sin fallback
- ☐ Generar + validar un mapa ≤ 250 ms p95
- ☐ 5 000 partidas simuladas en < 30 min
- ☐ Todas las métricas de balance dentro de objetivo
- ☐ Cada constante de `BALANCE` tiene su barrido guardado en reports/
- ☐ `corr(regiones@T6, victoria) < 0,45`

v0.9 — Pulido móvil, UX y assets

Alcance

- Los ~55 assets originales + Galería + comprobaciones automáticas.
- Tutorial «El Simulacro» y descubrimiento progresivo.
- Compendio generado desde las tablas de balance.
- Accesibilidad completa (teclado, lector de pantalla, daltonismo, movimiento reducido).
- Notificaciones push, PWA e instalación.
- Presupuestos de rendimiento cumplidos.

Criterios de aceptación

- ☐ La checklist completa de QA móvil pasa (UX_MOBILE \$11)
- ☐ Bundle de la ruta de partida $\leq$ 180 KB gzip
- ☐ Assets totales $\leq$ 150 KB
- ☐ LCP móvil $\leq$ 2,5 s
- ☐ Una campaña completa jugable solo con teclado
- ☐ Una campaña completa jugable con lector de pantalla
- ☐ Un jugador nuevo completa el tutorial en $\leq$ 8 min sin ayuda externa
- ☐ Los 55 assets pasan assets:check y la revisión de coherencia

v0.95 — Beta cerrada

Alcance

- Telemetría anónima, informe de errores, bloqueo y reporte de usuarios.
- Canal de feedback in-game.
- Estabilidad, casos límite, mensajes de error traducidos.
- **Playtesting real con 20-50 personas.**

Criterios de aceptación

- ☐ $\geq$ 30 campañas completas jugadas por humanos
- ☐ Tasa de abandono $<$ 20 % de asientos-partida
- ☐ $>$ 70 % de campañas con $\geq$ 1 tratado activo ← valida el pilar de diplomacia
- ☐ $>$ 40 % con $\geq$ 1 ruptura de Sello
- ☐ Cero bugs críticos abiertos
- ☐ Cero incidentes de seguridad
- ☐ Duración media dentro de 9–12 turnos
- ☐ Encuesta: $\geq$ 70 % «entendí las reglas en la primera partida»

La métrica de « $>$ 70 % con tratado activo» es la que decide si el juego **es** lo que dice ser. Si sale baja, la respuesta no es más marketing: es subir el coste de consagración.

v1.0 — Release

Alcance

- ES y EN completos y revisados (UI, tutorial, compendio, lore, errores, diplomacia).
- Documentación al día, PDF generado.
- Página de aterrizaje, política de privacidad, términos.
- Monitorización y plan de respuesta a incidentes.

Criterios de aceptación (todos, sin excepción)

- ☐ El core loop funciona de principio a fin
- ☐ El multijugador funciona con 2, 3 y 5 jugadores
- ☐ La persistencia y la reconexión funcionan
- ☐ Los mapas están balanceados (fairness.sweep en verde)
- ☐ El móvil funciona (checklist completa)
- ☐ Los assets son coherentes y 100 % originales
- ☐ ES y EN completos: cero claves faltantes, cero literales en componentes
- ☐ Cero bugs críticos
- ☐ Los tres tests bloqueantes en verde
- ☐ Documentación y CHANGELOG al día
- ☐ PDF de documentación generado
- ☐ Coste de operación: 0 €/mes verificado

Post-1.0 (registrado, no comprometido)

Por orden de valor esperado:

#	Feature	Por qué
1	Objetivos ocultos individuales	Multiplicaría la diplomacia. Se aplazó solo por coste de balance y traducción.
2	Partidas de 4 y 6 jugadores	El generador ya lo soporta; falta balancear la economía del Núcleo
3	Gestión de ciudad con decisiones propias	Solo si el playtesting demuestra que se echa en falta
4	Espectador y repeticiones compartibles	Sale casi gratis del determinismo
5	Torneos y ligas privadas	Depende de que haya comunidad
6	Más doctrinas y anomalías	Contenido, no sistemas
7	Cosméticos y monetización	Requiere salir de Vercel Hobby
8	Ranking / ELO	Deliberadamente al final: cambia el metajuego

Riesgos del roadmap

Riesgo	Mitigación
v0.3 desborda (auth + RLS + concurrencia + deploy a la vez)	Presupuestarla como la versión más larga; hacer un <i>spike</i> de resolución concurrente antes de empezar la UI
El balance no converge en v0.8	El simulador existe desde v0.8 pero se puede empezar en v0.2 con un motor parcial; adelantarlo si hay dudas
Los assets se comen v0.9	Herramientas (galería + checks) antes que assets; inventario congelado en 55
No hay jugadores para la beta	Partidas privadas por código como flujo principal; empezar a reclutar en v0.7
Alcance que crece	Toda feature nueva entra por <code>DECISIONS.md</code> con una decisión explícita de qué se corta a cambio

Registro de decisiones

docs/DECISIONS.md

Toda decisión que condicione el proyecto se registra aquí. **Si una decisión cambia, se añade una entrada nueva que sustituya a la anterior — nunca se edita la histórica.** Formato: contexto → decisión → consecuencias → alternativas descartadas.

Estados: `aceptada` · `propuesta` (pendiente de confirmar) · `sustituida` · `revertida`

#	Decisión	Estado
001	Un solo motor determinista compartido	aceptada
002	El mapa es un grafo con simetría C_n , no una rejilla	aceptada
003	Combate determinista, sin dados	aceptada
004	Turnos simultáneos	aceptada
005	Autoridad en Route Handlers, no en funciones de Postgres	aceptada
006	Niebla de guerra por <code>player_views</code> prefiltradas	aceptada
007	Estado de partida en <code>jsonb</code> , no normalizado	aceptada
008	Traición tarifada, no prohibida	aceptada
009	La metaprogresión solo añade opciones	aceptada
010	La Ciudad es un hub, no un gestor	aceptada
011	Assets como SVG escritos a mano en el repositorio	aceptada
012	Mapa en SVG, no Canvas ni WebGL	aceptada
013	Sin eliminación de jugadores	aceptada
014	Tres disparadores de resolución de turno	aceptada
015	Diplomacia por plantillas antes que texto libre	aceptada
016	Licencia del proyecto	pendiente
017	Tipografía	pendiente
018	Cadencia por defecto	propuesta
019	Visibilidad de la reputación	propuesta
020	Sin sistema de armamento nuclear	aceptada
021	Las facciones se ligan a la cuenta y solo cambian el camino	aceptada
022	<code>packages/core</code> se consume como fuente TypeScript, sin compilar	aceptada
023	Políticas RLS a través de funciones <code>security definer</code>	aceptada
024	Tests de RLS contra un Postgres efímero real	aceptada
025	La resolución de turno se parte en arrendar y confirmar	aceptada
026	Una sola vista: la ciudad, y un botón	aceptada
027	La interfaz no explica: enseña	aceptada
028	Se evalúa Turso y se descarta	aceptada
029	Las claves de Supabase se aceptan por sus dos nombres	aceptada
030	El perfil lo crea la base de datos, no la API	aceptada
031	Invitado = sesión anónima de Supabase, no una cuenta aparte	aceptada
032	La configuración del proyecto Supabase se versiona y se despliega	aceptada
033	La comprobación previa del despliegue valida forma, no presencia	aceptada
034	El relieve es WebGL; lo que se lee y se toca sigue siendo DOM	aceptada
035	El mapa de campaña se queda en SVG; el relieve es para la Ciudad	sustituida por ADR-042
036	Un bot no es un humano ausente, y por eso no comparten código	aceptada

#	Decisión	Estado
037	El mapa se dibuja en hexágonos, pero no teje un panal	sustituida por ADR-041
038	La interfaz enseña y nombra	aceptada
039	Se jura una vez, y hasta entonces la facción no significa nada	aceptada
040	El mapa es la interfaz: se ordena tocándolo, y nada flota encima	aceptada
041	El mapa se tesela: territorio continuo, no un grafo dibujado	aceptada
042	La campaña se juega en el mundo 3D, sobre el mapa real	aceptada
043	El mapa se reparte por área: todas las provincias miden lo mismo	aceptada
044	La cámara se mide en provincias, y el mapa enseña el frente	parcialmente sustituida por ADR-045
045	El mapa de campaña usa el lenguaje visual del mockup, sin reinventarlo	aceptada

ADR-001 — Un solo motor determinista compartido

Estado: aceptada · 2026-08-15

Contexto. El juego necesita reglas en tres sitios: el servidor (autoridad), el cliente (previsualizar) y el simulador (balance). La opción habitual —reimplementar en cada uno— produce divergencias sutiles que aparecen como bugs de sincronización imposibles de depurar.

Decisión. Un único paquete `packages/core`, TypeScript puro, **cero dependencias de runtime**, sin I/O, sin `Date.now()`, sin `Math.random()`. Lo consumen los tres.

Consecuencias.

- ✓ Imposible que las reglas diverjan.
- ✓ El simulador prueba el juego real, no una aproximación.
- ✓ Los tests unitarios del motor son tests del juego.
- ⚠ Disciplina permanente: hay una regla de lint que impide dependencias e I/O en `core`.
- ⚠ El cliente carga la lógica del juego (es visible). Aceptable: no contiene secretos — los secretos son los *datos* del estado, que nunca salen del servidor.

Descartado. Motor en el servidor + heurística en el cliente (diverge); motor en SQL (intestable, no reutilizable por el simulador).

ADR-002 — El mapa es un grafo con simetría C_n , no una rejilla

Estado: aceptada · 2026-08-15

Contexto. El brief exige partidas de **2, 3 y 5** jugadores con mapas «matemáticamente equilibrados», y móvil como plataforma de referencia. Ninguna teselación regular del plano admite simetría rotacional de orden 5 (resultado cristalográfico). Y una rejilla con suficientes casillas para ser interesante da objetivos táctiles de 4 px en 360 px de ancho.

Decisión. El mapa es un **grafo de 45-95 regiones**, construido como **un sector generado y replicado n veces por rotación**.

Consecuencias.

- ✓ Equidad **exacta por construcción** para cualquier número de jugadores.

-  El evaluador no busca equidad: solo acota el daño de la perturbación. Problema tratable.
-  Regiones de 48–90 px: móvil resuelto.
-  Métricas como chokepoints o accesibilidad son propiedades de grafo **computables exactamente**, no estimaciones.
-  Sin tilesets ni atlas: menos assets.
-  Menos «sensación de mapa continuo» que un 4X clásico. Se compensa con arte de cartografía militar, que encaja con la ambientación.
-  Riesgo de sensación de espejo, sobre todo con 2 jugadores. Mitigado con perturbación acotada y rotación de perfiles económicos.

Descartado. Hexágonos (imposible C₅); cuadrados (ídem, y peor); ruido Perlin con reequilibrado a posteriori (no ofrece ninguna garantía demostrable).

ADR-003 — Combate determinista, sin dados

Estado: aceptada · 2026-08-15

Contexto. El producto del juego es la negociación. Una promesa como «*si mantienes la posición, tu guarnición sobrevive*» solo tiene sentido si es comprobable. Con dados, toda promesa lleva un asterisco.

Decisión. El combate es una función determinista de las fuerzas, el terreno, las posturas y las bonificaciones. **Sin varianza.**

Consecuencias.

-  Se pueden hacer promesas verificables ⇒ la confianza pasa a ser un historial.
-  La UI puede previsualizar el resultado exacto: enorme ganancia de UX en móvil.
-  El simulador converge con muchas menos partidas.
-  Ninguna derrota se atribuye a la mala suerte.
-  Menos momentos de «épica improbable». Se sustituyen por sorpresas de **información** (fuerzas ocultas, órdenes simultáneas, engaños de Sombra), que son sorpresas que el jugador puede aprender a anticipar.
-  Riesgo de que la partida se vuelva calculable. Mitigado por la niebla de guerra y la simultaneidad: sabes la fórmula, pero no todas las variables.

Descartado. Dados con varianza baja (mantiene el asterisco sin aportar nada); aleatoriedad determinada por semilla y revelada tras ordenar (confuso de explicar).

ADR-004 — Turnos simultáneos

Estado: aceptada · 2026-08-15

Contexto. Con turnos secuenciales, 5 jugadores × 12 turnos en cadencia diaria = 30 días de campaña, y cada jugador esperando 4 turnos ajenos por cada uno propio.

Decisión. Todos los jugadores dan órdenes a la vez; el servidor resuelve juntas.

Consecuencias.

-  Campaña de 6 días en vez de 30. Viable en móvil.
-  **Habilita la mecánica central:** si vieras moverse a los demás antes de decidir, las promesas no harían falta.
-  Hay que definir empates, cruces y prioridades. Resuelto con orden documentado y desempate por número de asiento — nunca al azar (GDD §15).

ADR-005 — Autoridad en Route Handlers, no en funciones de Postgres

Estado: aceptada · 2026-08-15

Contexto. La resolución de turnos debe ser autoritativa y atómica. Dos opciones: plpgsql en Supabase, o TypeScript en Vercel con `service_role`.

Decisión. La resolución corre en **Route Handlers de Next.js**, ejecutando `packages/core`. La atomicidad la aporta Postgres (advisory locks + bloqueo optimista), no la lógica.

Consecuencias.

-  El mismo motor sirve a servidor, cliente y simulador (ADR-001). En SQL, esto sería imposible.
-  Reglas testeables con Vitest, sin base de datos.
-  La atomicidad hay que construirla explícitamente (§8 del TDD). Aceptado: son ~20 líneas bien testeadas.
-  Arranque en frío de la función serverless. Irrelevante con turnos de 3 min o 12 h.

Descartado. plpgsql (motor inestable y no compatible); Edge Functions de Supabase (mismo resultado que Vercel, pero parte el despliegue en dos sitios).

ADR-006 — Niebla de guerra por `player_views` prefiltradas

Estado: aceptada · 2026-08-15

Contexto. Supabase expone PostgREST. Si la tabla del estado fuera legible, cualquiera haría una petición HTTP y vería el estado completo: la niebla de guerra sería cosmética. Es el riesgo técnico nº 1 del proyecto.

Decisión. `game_states` **niega todo** `SELECT`. El resolutor escribe además `player_views`, una proyección **por asiento y ya filtrada**, con RLS `seat = mi asiento`. Los datos ocultos no salen nunca del servidor.

Consecuencias.

-  La niebla es real, no confiada al cliente.
 -  Realtime funciona directamente sobre `player_views` respetando RLS.
 -  El cliente descarga menos datos.
 -  Escribir 5 filas por turno en vez de 1. Coste irrelevante.
 -  Hay que mantener la proyección correcta ⇒ test de fuga de información (**TESTING §3.1**), que recorre la vista entera buscando secretos.
-

ADR-007 — Estado de partida en `jsonb`, no normalizado

Estado: aceptada · 2026-08-15

Contexto. ¿Guardar el estado como un documento o como tablas relacionales?

Decisión. `jsonb`. Las tablas normalizadas (`match_results`, `treaties`, `messages`) existen solo para lo que **sí** se consulta relacionamente.

Consecuencias.

-  Escribir un turno es una fila, no 200 en 8 tablas.
-  Atomicidad trivial.

-  El motor ya trabaja con ese objeto: cero traducción entre representaciones.
-  Consultas analíticas pobres sobre el estado. Resuelto: la analítica sale de `match_results` y de la telemetría.
-  Sin validación de esquema en la BD. Resuelto: Zod en el servidor, y el estado solo lo escribe el motor.

ADR-008 — Traición tarifada, no prohibida

Estado: aceptada · 2026-08-15

Contexto. El brief pide acuerdos vinculantes validados en servidor **y** que la traición sea posible. Son incompatibles si «vinculante» significa «imposible de romper».

Decisión. Romper un Sello es **siempre posible, inmediato y público**, y **cuesta Ceniza** — el recurso con el que se gana. Lore: el Umbral registra los juramentos.

Consecuencias.

-  La traición existe, luego la confianza significa algo.
-  Traicionar te aleja de ganar ⇒ la decisión es un cálculo, no un impulso.
-  Coherente con el mundo: la Ceniza es la ley de conservación del juego.
-  Un solo número (`coste_ruptura`) ajusta cuánta traición hay en el metajuego.
-  Si el coste está mal calibrado, o nadie traiciona o traiciona todo el mundo. Vigilado por el simulador (`betrayal-is-priced`).

ADR-009 — La metaprogresión solo añade opciones

Estado: aceptada · 2026-08-15

Contexto. Cualquier desbloqueo numérico rompe un PvP competitivo. Ninguno no motiva.

Decisión. Los desbloques permanentes **solo pueden añadir doctrinas, anomalías, ciudades y cosméticos**.

Jamás modifican una constante de `BALANCE`. Verificado por CI (`no-power-creep`), que además simula cuenta completa vs. cuenta vacía y exige winrate 48-52 %.

Consecuencias.

-  El competitivo se mantiene íntegro.
-  La progresión es de conocimiento y adaptación, no de números.
-  Motiva menos que subir de nivel. Se compensa con desbloques que abren **formas de jugar** visiblemente distintas.
-  Limita para siempre la monetización a cosméticos. **Aceptado explícitamente.**

ADR-010 — La Ciudad es un hub, no un gestor

Estado: aceptada · 2026-08-15

Contexto. El brief describe una vista Ciudad con economía, producción, investigación, diplomacia, infraestructura, inteligencia, ejército, tecnología y desarrollo sobrenatural. Es un segundo juego completo que duplica todos los sistemas de la guerra.

Decisión. En v1.0 la Ciudad es un **hub de progresión y preparación**: tu ciudad crece visiblemente, eliges equipo, ves resultados y entras en campaña. Sin colas, sin temporizadores, sin economía paralela.

Consecuencias.

- ✓ El alcance de v1.0 se vuelve alcanzable.
- ✓ Se elimina la duplicación de sistemas y de curva de aprendizaje.
- ✓ Se conserva **entero** el bucle emocional que pedía el brief (volver, ver consecuencias, prepararse).
- ⚠ Menos «gestión» de la pedida. Registrado como post-1.0 nº 3, condicionado a que el playtesting demuestre que se echa en falta.

ADR-011 — Assets como SVG escritos a mano en el repositorio

Estado: aceptada · 2026-08-15

Contexto. El brief exige assets 100 % originales y trazables hasta su origen dentro del proyecto, y prohíbe expresamente marketplaces y material de terceros.

Decisión. Todo asset se autora como **SVG escrito a mano** en `assets/src/`, con cabecera de autoría obligatoria. Ningún binario. Un generador produce componentes React.

Consecuencias.

- ✓ Procedencia trivial: git es el registro.
- ✓ Imposible incorporar assets ajenos por accidente: un binario falla el lint.
- ✓ Coherencia impuesta por herramienta (paleta, cuadrícula, trazo).
- ✓ Sin peticiones de red, sin CLS, escala perfecta, theming gratis.
- ⚠ Limita el estilo visual a lo plano y geométrico. **Convertido en decisión artística deliberada** («cartografía militar contaminada»), no en una carencia.
- ⚠ Ilustración compleja (portada, arte de ciudad) queda fuera. Aceptado en v1.0.

ADR-012 — Mapa en SVG, no Canvas ni WebGL

Estado: aceptada · 2026-08-15

Contexto. ~95 regiones que hay que dibujar, tocar, enfocar y anunciar.

Decisión. SVG en el DOM.

Consecuencias.

- ✓ **Accesibilidad nativa:** cada región es enfocable y anunciable. Con Canvas o WebGL, cada punto de la tabla de accesibilidad costaría una implementación paralela.
- ✓ 0 KB de librería; nítido en pantallas 3×.
- ✓ Zoom y desplazamiento por `transform` (compuesto por GPU).
- ⚠ No escalaría a miles de elementos. Irrelevante: el mapa tiene ≤ 96 regiones por diseño (ADR-002).
- ⚠ Efectos visuales limitados a CSS/SVG. Coherente con la dirección artística.

ADR-013 — Sin eliminación de jugadores

Estado: aceptada · 2026-08-15

Contexto. En un juego asíncrono de 6 días, un jugador eliminado en el turno 5 tiene que esperar una semana sin jugar. Además, eliminar rivales acelera el snowball del líder.

Decisión. El Bastión **no se puede capturar**, solo sitiar. Un jugador arrasado conserva renta reducida, voz diplomática, capacidad de transferir y de puntuar.

Consecuencias.

- ✓ Nadie queda fuera de la partida.
- ✓ El líder nunca reduce el número de rivales: antídoto estructural contra el snowball.
- ✓ Un jugador arrasado sigue siendo **decisivo como árbitro y valioso como socio**.
- ⚠ Riesgo de kingmaking por rencor. Mitigado: los supervivientes siguen puntuando para la metaprogresión, así que siempre tienen un objetivo propio además de la venganza.
- ⚠ Se pierde la satisfacción de eliminar a alguien. Se sustituye por la **rendición dirigida**, que es más interesante: eliges a quién armar al perder.

ADR-014 — Tres disparadores de resolución de turno

Estado: aceptada · 2026-08-15

Contexto. Vercel Hobby permite **un cron diario**. Los turnos vencen cada 3 min (Blitz) o cada 12 h.

Decisión. Tres caminos independientes hacia la misma función idempotente: (1) inmediato cuando todos envían; (2) `pg_cron` en Supabase cada minuto vía `pg_net`; (3) oportunista, cuando cualquier petición de cliente detecta un plazo vencido.

Consecuencias.

- ✓ No dependemos de Vercel Cron ⇒ el free tier sigue siendo suficiente.
- ✓ Tolerante a fallos: tres caminos, y basta con uno.
- ✓ El caso normal (~85 %) se resuelve al instante, sin esperar a ningún cron.
- ⚠ Exige idempotencia estricta. Ya es necesaria por concurrencia, así que no añade complejidad nueva.
- ⚠ `pg_cron` y `pg_net` son extensiones que hay que activar en Supabase. Documentado en el README.

ADR-015 — Diplomacia por plantillas antes que texto libre

Estado: aceptada · 2026-08-15

Contexto. El sistema principal del juego es negociar, y la plataforma de referencia es un teléfono, donde escribir duele. Además el juego es bilingüe: dos jugadores pueden no compartir idioma.

Decisión. La unidad de interacción diplomática es una **Oferta estructurada**, no un mensaje. Se compone con 3–4 taps. El texto libre existe, pero es opcional y secundario.

Consecuencias.

- ✓ Negociar en móvil es viable ⇒ el sistema principal se usa de verdad.
- ✓ **Dos jugadores sin idioma común pueden cerrar un trato**: la oferta es un dato que se renderiza traducido.
- ✓ Las ofertas son analizables ⇒ telemetría real sobre la diplomacia.
- ⚠ Menos expresividad que el texto libre. Mitigado: el texto libre sigue ahí, y el Distrito Cámara nivel 3 añade traducción automática del chat.
- ⚠ Hay que diseñar el catálogo de plantillas con cuidado: si falta una, ese trato no existe.

ADR-016 — Licencia del proyecto

Estado: 🕒 pendiente — decisión del propietario del repositorio

Opciones: código propietario (todos los derechos reservados) · MIT/Apache-2.0 para el código con assets y lore reservados · dual.

Recomendación: **código propietario durante la beta**, decidir en v1.0. No bloquea el desarrollo, pero debe resolverse antes de que el repositorio sea público.

ADR-017 — Tipografía

Estado: aceptada · 2026-08-16

Contexto. Los requisitos eran: familia variable, geométrica, condensada, **números tabulares**, cobertura de ES + EN, licencia **SIL OFL** o equivalente y subseable. Con la fuente del sistema, la interfaz se leía como un formulario web — y ese era, de lejos, el detalle que más impedía que el juego pareciera un juego.

Decisión. Archivo Variable (Omnibus-Type, SIL OFL 1.1), alojada en el propio repositorio, solo el subconjunto `latin`.

Consecuencias.

- ✓ Un único eje `width` (62–125 %) da los tres registros —rótulo condensado en versales, texto corrido, cifra tabular— sin una segunda descarga. El contraste tipográfico lo da la anchura, no cambiar de familia.
- ✓ `tnum` de serie: las cifras de recursos y plazos se alinean en columna, que es lo que permite compararlas de un vistazo.
- ✓ Servida desde el repositorio: ni una petición a un dominio de terceros, y por tanto ningún salto de maquetación en el primer segundo de partida.
- ✓ 90 KB. Solo `latin`: cubre ES y EN enteros —ñ, tildes, ¿, ¡— y ahorra los 86 KB de `latin-ext`, que solo sirven para idiomas que este juego no tiene.
- ⚠ Si algún día se añade un idioma con latín extendido —polaco, turco, checo—, hay que volver a añadir ese subconjunto. Está anotado aquí para que no se descubra en producción.

Descartado. Inter y similares (excelentes, pero sin eje de anchura: harían falta dos familias); Oswald y las condensadas puras (ilegibles en texto corrido); una fuente de sistema (distinta en cada dispositivo, que en un juego donde leer el mapa es media decisión no vale).

ADR-018 — Cadencia por defecto

Estado: 🟦 propuesta — decisión bloqueante nº 1

Propuesta: Diaria (12 h/turno, ~6 días) por defecto, con **Blitz disponible desde v0.3** para poder testear.

Razones. No exige coincidencia horaria, que es el mayor problema de un juego nuevo con pocos jugadores; cumple literalmente la «semana de guerra» del brief; y es el patrón nativo del móvil. Blitz debe existir desde v0.3 aunque no sea el modo principal: sin él, probar un ciclo completo tarda 6 días y el desarrollo se vuelve insoportable.

Si se elige Blitz por defecto, cambian: el diseño de notificaciones (menos push, más presencia), el peso de la reconexión, la pausa de desconexión, y el foco del playtesting.

ADR-019 — Visibilidad de la reputación

Estado:  propuesta — decisión bloqueante nº 3

Propuesta: dentro de la partida, **recuento factual público** («Sellos: 4 honrados · 1 roto»), sin etiquetas ni puntuación. Entre partidas, agregado en el perfil, **sin ranking**, con decaimiento a 50 campañas.

Razones. Una reputación con etiquetas morales («Traidor») empujaría el metajuego hacia «nunca traiciones», y la traición es el producto. Un recuento factual informa sin juzgar: el juicio lo hacen los jugadores.

Riesgo. Si el playtesting muestra que aun así mata la traición, se reduce a solo-dentro-de-partida. Métrica de decisión: **> 40 % de campañas con ≥ 1 ruptura.**

ADR-020 — Sin sistema de armamento nuclear

Estado: aceptada · 2026-08-15

Contexto. El brief menciona armamento nuclear y advierte a la vez de que no debe ser un botón de «ganar».

Decisión. No hay arsenal nuclear. Existe **una** capacidad estratégica extrema (*Yermo*, investigación de tier III): destruye todas las fuerzas de una región y la deja inutilizable **para todos, incluido quien la lanza**, cuesta 8 $\star$ y es públicamente atribuida.

Consecuencias.

- ✓ Existe la escalada estratégica sin existir un botón de ganar.
- ✓ Su valor real es **la amenaza**, no el uso: es un arma de negociación.
- ✓ Destruye valor que no recupera nadie $\Rightarrow$ usarla es casi siempre un error, y todos lo saben.
- ✓ Evita representar armamento real con detalle, que no aporta nada al juego.
- ⚠ Menos espectáculo del que sugiere la ambientación. Compensado por las anomalías.

ADR-021 — Las facciones se ligan a la cuenta y solo cambian el camino

Estado: aceptada · 2026-08-15

Contexto. El proyecto necesita una identidad persistente por cuenta —algo a lo que pertenecer entre campañas— pero ADR-009 prohíbe que cualquier desbloqueo permanente modifique una constante de balance. Un sistema de facciones mal diseñado es la vía más rápida de saltarse esa regla: basta con «mi facción tiene +10 % de industria» para haber roto el PvP.

Decisión. La cuenta **jura** a una de las 6 ciudades signatarias. La facción fija la doctrina de origen y **abarata** ($\times 0,6$) los desbloques afines. **Nunca encarece nada, y el conjunto de opciones alcanzable es idéntico para las seis.** Dentro de la partida, dos jugadores de la misma facción quedan marcados con **Concordia**, que es información pública y **no tiene ningún efecto mecánico.**

Consecuencias.

- ✓ Identidad y sentido de pertenencia sin tocar el balance.
- ✓ El invariante «mismo techo» es verificable: `factions/no-ceiling-difference` compara dos cuentas al máximo de facciones distintas y exige conjuntos idénticos.
- ✓ La Concordia aporta al pilar diplomático **gratis**: reconfigura las expectativas de toda la mesa sin mover una constante. Los otros tres tienen que decidir si asumen que los concordes se aliarán, y los concordes si aprovechan esa suposición.

-  `factions/` no importa `balance/`, y hay un test que lee el fuente para comprobarlo: la regla de oro deja de depender de la disciplina de quien programa.
-  La afinidad crea una diferencia de **ritmo** entre facciones durante las primeras ~13 campañas. Aceptado: es la misma clase de diferencia que ya existe entre una cuenta nueva y una veterana, y converge.
-  El Cisma podría usarse para «reoptimizar» descuentos. Mitigado: cuesta 60 $\rightarrow$ y exige 3 campañas de espera; y como los desbloques se conservan, no hay nada que reoptimizar salvo compras futuras.

Descartado. Facciones con bonificaciones estadísticas (rompe ADR-009); doctrinas exclusivas por facción (rompe el invariante del techo); Concordia con ventaja mecánica —visión compartida inicial o Sellos más baratos— porque convertiría toda partida de 5 en un 2v3 desde el turno 0 **y** mataría la duda que la hace interesante; guerra de facciones con territorio global persistente (sistema enorme, ajeno al core loop, y castiga a la facción con menos jugadores).

ADR-022 — `packages/core` se consume como fuente TypeScript, sin compilar

Estado: aceptada · 2026-08-15

Contexto. El motor lo ejecutan tres consumidores (servidor, cliente, simulador). Publicarlo como JavaScript compilado añadiría un artefacto intermedio que puede quedar desincronizado con el fuente, justo en el paquete donde una divergencia es más cara.

Decisión. `@gdc/core` exporta `./src/index.ts` directamente. Next lo transpila con `transpilePackages`, y Vitest lo carga tal cual. **No hay paso de build.** Los imports relativos van sin extensión (`from './rng'`), que es lo que entienden tanto `bundler` de TypeScript como el bundler de Next.

Consecuencias.

-  Imposible que el motor compilado y el fuente diverjan: solo hay uno.
-  Un solo `npm test` cubre exactamente el código que corre en producción.
-  Sin paso de build, sin caché que invalidar, sin sourcemaps que cuadrar.
-  Cualquier consumidor futuro debe poder transpilar TypeScript. Aceptado: los tres que hay lo hacen, y un cuarto que no pudiera sería señal de un problema mayor.
-  Los imports con extensión `.js` (estilo NodeNext) **no** funcionan aquí. Está documentado en `packages/core/CLAUDE.md`.

Descartado. Compilar a `dist/` con `tsc` (artefacto que puede diverger, y obliga a recordar reconstruir antes de cada test); publicar dual ESM/CJS (complejidad sin ninguna ganancia para un paquete privado de monorepo).

ADR-023 — Las políticas RLS consultan a través de funciones `security definer`

Estado: aceptada · 2026-08-15

Contexto. Casi todas las políticas del juego necesitan la misma pregunta: *¿está este usuario sentado en esta partida, y en qué asiento?* La respuesta vive en `game_players`, que a su vez tiene RLS. Una política sobre `game_players` que consulte `game_players` entra en bucle y Postgres la corta con «infinite recursion detected in policy». El esquema ilustrativo de **TECHNICAL DESIGN §6.3** escribe esa consulta en línea, y así escrita no arranca.

Decisión. Tres funciones `security definer` y `stable` —`is_player(game)`, `is_seat(game, seat)` y `my_seat(game)`— resuelven la pregunta leyendo la tabla sin RLS. Las políticas las invocan en vez de repetir el `exists`.

Consecuencias.

-  Sin recursión, y una sola definición de «estar sentado en una partida».

-  `stable` permite al planificador evaluarlas una vez por consulta, no una por fila.
-  `security definer` significa que la función ve todo: si se concediera a quien no debe, serviría para sondear cualquier partida del sistema. Se mitiga con `revoke` y con un test que enumera **todas** las funciones `security definer` invocables por `authenticated` y exige que sean exactamente esas tres.
-  Llevan `set search_path = public, pg_temp` obligatoriamente: sin eso, una función `security definer` es un vector de escalada de privilegios de manual.

Descartado. Desactivar RLS en `game_players` (la haría legible entera desde PostgREST: quién juega a qué y con quién, en todo el sistema); duplicar el `exists` en cada política (nueve copias de la misma regla de seguridad, que es como se acaban divergiendo).

ADR-024 — Los tests de RLS corren contra un Postgres efímero, sin Docker ni driver

Estado: aceptada · 2026-08-15

Contexto. Los siete tests de RLS son bloqueantes (ROADMAP): si uno falla, no hay release. Un test bloqueante que dependa de Docker, de una cuenta en la nube o de una conexión se acaba saltando, y un test de seguridad que se salta solo es peor que no tenerlo, porque el CI se pone verde igual.

Decisión. `tools/pg/harness.mjs` levanta un clúster con los binarios de PostgreSQL del sistema, escuchando **solo en socket unix**, le aplica un shim que emula `auth.uid()` y los roles de Supabase, y encima las migraciones reales. Los tests hablan con él por `psql`, con `set local role` y las claims del JWT — exactamente lo que hace PostgREST.

Consecuencias.

-  Cero dependencias nuevas: ni Docker, ni un driver de Postgres, ni la CLI de Supabase.
-  Se prueban las migraciones **reales**, no una réplica del esquema.
-  Sin TCP: un Postgres de pruebas con autenticación `trust` escuchando en un puerto es un agujero, aunque sea local y aunque sea un rato.
-  El shim puede desviarse de lo que hace Supabase de verdad. Se mitiga reproduciendo el `alter default privileges` de Supabase: sin él los tests pasarían por falta de permisos en vez de por las políticas. Ese detalle ya cazó un fallo real — `begin_resolution` era invocable por cualquier jugador.
-  Hace falta PostgreSQL instalado. El arnés falla con un mensaje explícito en vez de saltarse los tests, que es justo lo que no debe hacer.

Descartado. `supabase start` (Docker obligatorio, arranque de minutos, imposible en muchos CI); `pg-mem` y otras emulaciones en JavaScript (no implementan RLS, que es literalmente lo único que hay que probar); probar contra el proyecto remoto (lento, compartido y con datos reales).

ADR-025 — La resolución de turno se parte en arrendar y confirmar

Estado: aceptada · 2026-08-15

Contexto. [TECHNICAL DESIGN §8.2](#) describe la resolución dentro de una transacción con `pg_advisory_xact_lock`. Ese lock se libera al terminar la transacción, y el motor es TypeScript: la transacción tendría que seguir abierta mientras `reduce()` corre en Node. Con PostgREST eso no es posible —cada llamada RPC es su propia transacción— y mantener una conexión directa abierta desde una función serverless mientras se computa es exactamente el patrón que agota el pool.

Decisión. Dos llamadas, cada una atómica: `begin_resolution()` cierra la fila con `for update`, comprueba que toca resolver, arrienda la partida 30 s y devuelve estado y órdenes; `reduce()` corre en Node; y `commit_resolution()` escribe todo condicionado a que `state_version` no haya cambiado.

Consecuencias.

- ✓ Se conservan las tres defensas superpuestas del diseño: `for update` serializa, el turno esperado da idempotencia y el bloqueo optimista es la última red.
- ✓ Ninguna transacción queda abierta durante un cómputo: la conexión se devuelve al pool de inmediato.
- ✓ El arrendamiento vence solo. Un resolutor que muera a media faena no deja la partida colgada para siempre; otro lo reintenta en treinta segundos.
- ⚠ El arrendamiento **no** es el mecanismo de exclusión: es solo un ahorro de trabajo. La corrección la garantiza el `state_version`. Confundirlo llevaría a alargar el arrendamiento «por seguridad», que es lo contrario de lo que hace falta.
- ⚠ Dos resolutores simultáneos pueden computar lo mismo dos veces. Es barato y, como `reduce()` es determinista, el resultado es idéntico: solo uno escribe.

Descartado. Mantener la transacción abierta desde Node (agota el pool y ata el diseño a una conexión directa); mover `reduce()` a plpgsql (reimplementar el motor en un segundo lenguaje es exactamente lo que la regla de «un solo motor» prohíbe); confiar solo en el bloqueo optimista sin arrendamiento (correcto, pero con cinco clientes disparando la resolución a la vez se computan cinco turnos para tirar cuatro).

ADR-026 — Una sola vista: la ciudad, y un botón

Estado: aceptada · 2026-08-16 · sustituye el flujo de [ADR-015](#) para entrar en partida

Contexto. El recorrido para empezar a jugar eran cuatro pantallas: portada, entrada, lista de partidas y sala de espera con código de invitación. Cada una es un peaje entre el jugador y el turno que quiere jugar, y en cadencia Blitz —tres minutos por turno— ese peaje se come un pedazo real del plazo. [MULTIPLAYER §3.2](#) daba el código por flujo principal porque juntar a cinco conocidos parecía más fácil que llenar una cola; el coste de esa elección es que **la primera vez que alguien abre el juego solo, no puede jugar.**

Decisión. Una vista. Al abrir el juego se ve **tu ciudad en cenital** y un botón. Pulsarlo busca campaña; si hay gente esperando, la partida se crea y **empieza sola**, sin sala intermedia. Si no la hay, se ve llegar a las demás ciudades, y pasados unos minutos los asientos que faltan los ocupa el Mando Automático.

Consecuencias.

- ✓ De abrir la aplicación a jugar: **una pulsación.**
- ✓ La ciudad deja de ser una pantalla que visitar y pasa a ser el marco de todo, que es lo que [ADR-010](#) quería decir con «hub».
- ✓ La espera cuenta la ficción del juego: las ciudades **son** teletransportadas al campo de batalla, y buscar partida es verlas aparecer, cada una en su posición rotacional.
- ✓ Un jugador solo nunca se queda sin partida — la mitigación de [DISCOVERY P1](#) deja de ser un plan y pasa a estar en el camino principal.
- ⚠ Se pierde el flujo de «juego privado con estos cinco conocidos». Las funciones SQL (`join_game`, `invite_code`) siguen ahí; lo que desaparece es la interfaz. Vuelve más adelante **dentro de la misma vista**, no como una pantalla aparte.
- ⚠ Una cola vacía degrada a partida contra bots. Es preferible a no poder jugar, pero exige vigilar la proporción de partidas con bot en la beta.

Descartado. Mantener las dos vías con un selector (vuelve a ser una pantalla de menú); emparejamiento por habilidad (no hay ELO en v1.0 — ver [METAPROGRESSION §7](#)); sala de espera con «empezar» manual (el anfitrión decide cuándo juegan los demás, que es exactamente el peaje que se quería quitar).

ADR-027 — La interfaz no explica: enseña

Estado: aceptada · 2026-08-16

Contexto. «Simple de aprender, difícil de dominar» no se consigue escribiendo mejores descripciones. Un juego que necesita un párrafo para explicar un botón ya perdió, y además cada frase visible es una cadena que traducir, revisar y mantener en dos idiomas.

Decisión. Fuera el texto explicativo de la interfaz. Lo que hay que comunicar se comunica con **forma, posición, cantidad y movimiento**. Concretamente:

En vez de	Se dibuja
«2, 3 o 5 jugadores»	La disposición rotacional real del mapa: dos enfrentadas, tres en triángulo, cinco en pentágono
«Blitz / Diaria / Relajada»	Marcas de ritmo: una, dos, tres. Más marcas, más lento
«Distrito bloqueado»	El distrito dibujado como cimientos en vez de como edificios
«Ceniza acumulada: 312»	Un silo que se llena, con la cifra al lado
«Buscando partida... 3 de 5»	Las ciudades rivales llegando, cada una a su posición

Consecuencias.

- ✓ La elección **enseña la regla**: quien elige «5» ya ha visto la simetría del mapa que va a jugar. Eso es tutorial sin tutorial.
- ✓ Menos cadenas que traducir, y ninguna que se quede desincronizada entre ES y EN.
- ✓ Sirve igual a alguien que no lea el idioma de la interfaz.
- ⚠ **No exime de accesibilidad.** Cada control lleva `aria-label` traducido y cada gráfico su descripción: prescindir de texto es una decisión visual, jamás una excusa para dejar fuera a quien no ve la pantalla. Los rótulos siguen en `es.json` / `en.json` y el test de paridad los cubre igual.
- ⚠ Un símbolo mal elegido es peor que una frase mala, porque no se puede leer. La Galería (`/dev/gallery`) existe para cazarlos antes de que lleguen a nadie.

Descartado. Tooltips (no existen en táctil); un tutorial de bienvenida (se salta y no resuelve el problema de fondo); iconos con etiqueta debajo (es texto explicativo con un adorno encima).

ADR-028 — Se evalúa Turso y se descarta: la niebla de guerra vive en el esquema

Estado: aceptada · 2026-08-16

Contexto. Turso (libSQL) ofrece réplicas embebidas, lectura en el borde y bases de datos por inquilino a coste casi nulo. Sobre el papel encaja con un juego por turnos asíncrono, y «una partida es un inquilino» es un modelo atractivo. Se evalúa antes de seguir invirtiendo en el esquema actual.

Decisión. Se descarta. El proyecto sigue sobre Postgres en Supabase.

Por qué, medido sobre lo que ya existe.

Rasgo en uso	Veces	En libSQL
Políticas RLS	17	no existe RLS
<code>auth.uid()</code>	14	no hay servicio de auth
Funciones <code>security definer</code>	18	no hay procedimientos almacenados
<code>jsonb</code>	108	hay JSON1, pero no el tipo, ni <code>jsonb_array_elements</code> , ni GIN
<code>for update</code> / <code>skip locked</code>	13	escritor único, bloqueo a nivel de base
<code>smallint[]</code> + GIN	6	no hay tipos array
<code>pg_cron</code> + <code>pg_net</code>	el reloj	no hay planificador

Lo decisivo no es el recuento: es **dónde vive la niebla de guerra**. `game_states` tiene RLS activa y **cero políticas**, así que es ilegible por construcción y no por convención — con 43 tests bloqueantes que lo verifican contra un Postgres real (ADR-024). Sin RLS, ese filtrado pasa a ser código de aplicación: la niebla dependería de que ningún manejador tenga un fallo, y un fallo ahí **no da error**, da un jugador viendo lo que no debe sin que nadie se entere. Es el riesgo técnico nº 1 del proyecto (DISCOVERY T1), y moverlo al código es exactamente la decisión contraria a la que se tomó.

Y no habría qué testear: `npm run test:security` existe **porque la seguridad está en la base**. Movida al código se convierte en tests de manejadores, que cubren lo que uno se acuerda de cubrir.

Consecuencias.

- ✓ Se conserva la garantía estructural: lo que un jugador no debe ver no sale del servidor porque la base no lo entrega, no porque el código se acuerde de filtrarlo.
- ✓ Se conservan las dos piezas con más carreras probadas: el `for update skip locked` del emparejamiento y el arrendamiento + `state_version` de la resolución (ADR-025). Sobre un motor de escritor único habría que rediseñarlas, no portarlas.
- ⚠ Se renuncia a la lectura en el borde y a las réplicas embebidas. Con cadencia Diaria —la propuesta por defecto— la latencia de lectura no es el cuello de botella.
- ⚠ Se mantiene la dependencia del free tier de Supabase, cuyo punto de ruptura conocido son las **200 conexiones concurrentes** de Realtime. La salida documentada es agrupar canales por partida, no cambiar de base.

Descartado. Turso con una base por partida: el aislamiento que importa **no es entre partidas, sino entre asientos dentro de una misma partida**, así que una base por partida deja a los cinco jugadores viendo lo mismo — que es el problema entero. Turso con el filtrado en la API: es la opción anterior con más pasos y la misma pérdida. Un híbrido Postgres + Turso para lecturas públicas —archivo de partidas terminadas, por ejemplo— **no se descarta para después de v1.0**: ahí sí tendría sentido y convive con Supabase.

Nota. Esta evaluación surgió tras una jornada peleando con el despliegue. Conviene dejar dicho que aquello **no fue culpa de Supabase**: fueron tres problemas encadenados de configuración de Vercel —el `prebuild`, el Root Directory y el Framework Preset—, todos documentados en DEPLOYMENT §6. Cambiar de base de datos por eso habría sido arreglar el tejado porque gotea la puerta.

ADR-029 — Las claves de Supabase se aceptan por sus dos nombres

Estado: aceptada · 2026-08-16

Contexto. Supabase ha cambiado su sistema de claves de API. Donde antes había dos JWT (`eyJ...`) llamados `anon` y `service_role`, ahora hay `sb_publishable_...` y `sb_secret_...`, y el panel del proyecto sugiere copiarlas en variables llamadas `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY` y `SUPABASE_SECRET_KEY`. El código leía únicamente los nombres

antiguos, así que **seguir la sugerencia del panel dejaba el despliegue con un «Internal Server Error» sin más pista**. Es un fallo de configuración silencioso, que es justo la clase que ya costó una tarde.

Decisión. Cada clave admite **los dos nombres**, con el nuevo primero. `apps/web/lib/server/env.ts` resuelve el primero que esté definido; `.env.example`, el README y `DEPLOYMENT.md` documentan el nuevo; los antiguos siguen valiendo sin aviso de obsolescencia.

Por qué no basta con renombrar y ya. Cambiar los nombres a secas rompería cualquier despliegue existente en el instante del `git pull` — incluido el de producción, y sin error de compilación que lo cace. Aceptar los dos hace que actualizar sea inocuo y que empezar de cero funcione copiando lo que dice el panel.

Lo que no cambia: el rol de Postgres. La clave publicable sigue autenticando como `anon` y la secreta como `service_role`. Ni una política de RLS, ni el shim de Supabase de `tools/pg/`, ni uno solo de los 43 tests de seguridad se ven afectados. Es un cambio de nombre, no de modelo de permisos — y conviene dejarlo escrito, porque «Supabase cambió las claves» suena a algo que debería obligar a revisar la seguridad, y no lo es.

Consecuencias.

- ✓ Copiar lo que sugiere el panel de Supabase funciona.
- ✓ Un despliegue con los nombres antiguos sigue arrancando tras actualizar.
- ✓ La regla estructural de `check-deps.mjs` vigila los **dos** nombres de la clave secreta: si solo mirara el antiguo, bastaría con usar el nuevo para colarla al cliente.
- ✓ Un test comprueba que ningún alias de la clave secreta lleva prefijo `NEXT_PUBLIC_`. Es la invariante que de verdad importa: ese prefijo la publicaría en el navegador.
- ⚠ Dos nombres por clave es más superficie que uno. Se acota a una tabla de cuatro entradas en un único archivo, y `/api/health` informa siempre del nombre nuevo, así que el antiguo no se propaga a documentación ni a mensajes de error.

Descartado.

- **Renombrar sin compatibilidad.** Rompe los despliegues existentes en silencio.
- **Quedarse solo con los nombres antiguos.** Condena a cada persona que cree un proyecto nuevo a repetir el mismo diagnóstico de diez minutos.
- **Deducir el nombre por el prefijo del valor** (`sb_` frente a `eyJ`). Adivinar la variable a partir de su contenido es exactamente el tipo de magia que falla el día que Supabase estrene un tercer formato.

ADR-030 — El perfil lo crea la base de datos, no la API

Estado: aceptada · 2026-08-16

Contexto. Entrar por enlace mágico creaba la fila en `auth.users`, pero nadie creaba la de `profiles`. Y `currentViewer()` devuelve `null` sin perfil, así que `/` redirigía a `/sign-in`, que mostraba el formulario otra vez: una cuenta recién confirmada quedaba en un bucle. Sin error en ningún log — desde fuera, todo «funcionaba».

Al mismo tiempo hacía falta que el modo invitado (ADR-031) llegase a la base **por el mismo camino** que una cuenta con correo.

Decisión. Un trigger sobre `auth.users` crea el perfil, con un nombre generado determinista. No hay ruta de alta en la API que se pueda olvidar de crearlo porque no hay ruta de alta: hay un `insert` en `auth.users`, y de eso se encarga Supabase.

Es la misma forma que ya tenía `on_profile_created` para la Ciudad, con el motivo que ya estaba escrito en la migración 0001: «el trigger evita que la API tenga que acordarse de crearla, que es justo la clase de olvido que se descubre en producción».

Por qué un nombre generado y no un formulario. Una pantalla de «¿cómo te llamamos?» es exactamente la pantalla intermedia que **ADR-026** prohíbe, y para un invitado —que entra precisamente para no rellenar nada— sería absurda. El nombre se puede cambiar después; existe para no pedir uno al entrar, no para imponerlo.

Consecuencias.

- ✓ Desaparece el bucle, y desaparece la clase entera: no hay estado «usuario sin perfil».
- ✓ Las cuentas creadas antes del trigger salen del bucle con el backfill de la migración.
- ✓ Invitado y cuenta con correo comparten camino de alta, así que no hay dos rutas que mantener sincronizadas.
- ⚠ La facción sigue siendo la de por defecto (`vantera`) para todo el mundo, porque **ADR-021** la trata como una elección con consecuencias y no hay dónde elegirla sin añadir la pantalla que **ADR-026** prohíbe. **Queda abierto:** asignarla al azar sería quitar una decisión, y ponerla en un formulario sería añadir un peaje.
- ⚠ El nombre generado sale de 16 palabras × 1000 números. Dos jugadores de la misma mesa compartiéndolo es improbable (~0,06 % en una partida de cinco) y no rompe nada: la interfaz nunca distingue solo por nombre, siempre hay barra de asiento y emblema.

Descartado.

- **Crear en el callback de** `/auth/callback`. Solo cubre el camino del correo; el invitado necesitaría el suyo, y volvemos a dos rutas.
- **Crearlos perezosamente en** `currentViewer()`. Una lectura que escribe. Además obliga a usar `service_role` en la capa de lectura, que es justo lo que la regla de las dos capas de identidad prohíbe.
- **Una pantalla de alta.** Contradice **ADR-026**, y en cadencia Blitz cada pantalla previa cuesta plazo de verdad.

ADR-031 — Invitado = sesión anónima de Supabase, no una cuenta aparte

Estado: aceptada · 2026-08-16

Contexto. Pedir un correo antes de dejar probar el juego es un peaje en el peor sitio posible: antes de que nadie sepa si le interesa. Hace falta una forma de entrar sin registrarse que **llegue a la base de datos igual** que una cuenta normal, y que no sirva para iniciar sesión desde otro dispositivo.

Decisión. El invitado es una **sesión anónima de Supabase** (`auth.signInAnonymously()`). No es una cuenta de mentira ni un modo aparte: crea una fila real en `auth.users`, el trigger de alta (**ADR-030**) le hace perfil y Ciudad como a cualquiera, y su rol de Postgres es `authenticated`, así que **RLS lo trata exactamente igual**.

La propiedad que se pedía sale sola. Sin correo no hay enlace mágico, así que no hay forma de identificarse: la sesión vive en la cookie de ese navegador y en ninguno más. No hay que implementar la limitación — es lo que queda cuando no hay credencial.

Consecuencias.

- ✓ Cero código de autorización nuevo. Ni una política de RLS cambia, y por tanto ninguno de los tests bloqueantes cubre menos que antes.
- ✓ `profiles.is_guest` deja saber a la interfaz a quién ofrecerle vincular un correo antes de que pierda el dispositivo. No es escribible desde el cliente: si lo fuera, la marca no significaría nada.
- ⚠ Un alta anónima no cuesta nada de hacer, así que sin límite es una forma cómoda de llenar `auth.users` desde una sola IP. Mitigado con `anonymous_users = 30` por hora en `config.toml`. Si hiciera falta más, la respuesta es un captcha, no subir el número.
- ⚠ Quien pierda el dispositivo pierde la cuenta. Es inherente y no se puede esconder: se avisa **antes** de entrar, debajo del botón, no después.

Descartado.

- **Un usuario «invitado» en `profiles` sin fila en `auth.users`.** Deja a `auth.uid()` sin valor, y con eso las 17 políticas de RLS dejan de aplicar. Sería reimplementar la autorización para un caso.
- **Una cuenta con correo falso generado.** Ocupa el espacio de direcciones reales y hace que «vincular tu correo de verdad» sea un cambio de correo con confirmación doble.
- **`localStorage` sin sesión de servidor.** El cliente no decide nada (regla 2): una identidad que solo existe en el navegador no puede sostener una partida.

ADR-032 — La configuración del proyecto Supabase se versiona y se despliega

Estado: aceptada · 2026-08-16

Contexto. El enlace de confirmación llegaba con `redirect_to=http://localhost:3000`. La causa no estaba en el código: Supabase **solo acepta un `redirect_to` que esté en su lista blanca**, y cuando no lo está lo sustituye **en silencio** por su Site URL, que de fábrica es `http://localhost:3000`. Un ajuste del panel que nadie había tocado, invisible desde el repositorio, imposible de cazar con un test y sin rastro en ningún log.

No era el único de su especie. Las migraciones y los dos secretos del reloj también eran pasos manuales documentados en DEPLOYMENT.md, y los tres comparten lo peor: **no fallan ruidosamente**. Sin migraciones la aplicación arranca y se cae al primer `select`; sin los secretos del vault `pg_cron` late, llama con un `Authorization` vacío, recibe 401 y las partidas con ausencias se quedan quietas.

Decisión. `supabase/config.toml` entra en el repositorio y **es la fuente de la verdad**; el panel es el resultado. Un workflow lo empuja junto con las migraciones y los secretos del vault cuando llega a `main` un cambio bajo `supabase/`, y **comprueba después el resultado** — incluido que `game_states` sigue con RLS activa y cero políticas, que para el despliegue sí falla.

Consecuencias.

-  Los tres fallos silenciosos pasan a ser imposibles en lugar de invisibles.
-  Un proyecto de Supabase nuevo se configura solo. Antes había que releer DEPLOYMENT.md y hacer siete cosas a mano en el orden correcto.
-  El despliegue comprueba la invariante de la niebla de guerra contra la base **real**, no contra el arnés de pruebas. Es la única comprobación del proyecto que mira producción.
-  Un cambio hecho a mano en el panel se pierde en el siguiente despliegue. Es el precio de tener una fuente de la verdad, y está escrito en la cabecera del archivo y en DEPLOYMENT §2.2.
-  `config push` empuja el bloque entero: lo que no esté declarado se manda con el valor por defecto del CLI, **no** se deja como está. Mitigado escribiendo los valores de forma explícita aunque coincidan con el defecto.
-  El pipeline necesita un token de acceso y la contraseña de la base en los secretos del repositorio. Acotado a un entorno de GitHub, que además permite exigir aprobación manual antes de tocar producción.

Descartado.

- **Dejarlo documentado.** Es lo que había. Un paso manual documentado es un paso que se hace una vez y se olvida en el proyecto siguiente.
- **Arreglarlo solo desde el cliente**, calculando bien `emailRedirectTo`. No basta: si la URL no está en la lista blanca de Supabase, se sustituye igual. Hacen falta las dos mitades, y esa es justo la parte que hace el fallo tan desconcertante.
- **`psql` contra la base para los secretos del vault.** La conexión directa obliga a acertar con la región del pooler o a depender de IPv6. `supabase db query --linked` va por la Management API y no tiene ese problema.

ADR-033 — La comprobación previa del despliegue valida forma, no presencia

Estado: aceptada · 2026-08-16

Contexto. El primer despliegue con la configuración ya puesta falló así:

```
✓ configuración completa
...
Invalid project ref format. Must be like `abcdefghijklmopqrst`.
```

La causa era un `\r\n` al final de dos variables, colado al pegarlas en el panel de GitHub. **No se ve en ningún sitio:** ni en la caja de texto al escribirlas, ni en la lista de variables después. Solo aparece en el log del runner, y ahí como una URL partida en dos líneas — que es justo el sitio donde nadie mira porque el paso anterior dijo que todo estaba bien.

Dos errores propios se sumaron. La comprobación previa preguntaba `[ -n "$valor" ]`, es decir **presencia**, y con eso bendijo un valor inservible. Y las URL derivadas — `GDC_CALLBACK_URL`, `GDC_RESOLVE_URL` — se componían en el bloque `env` del job, o sea **antes de que hubiera ningún sitio donde limpiarlas**: `SITE_URL\n` + `/auth/callback` produce una dirección con un salto de línea dentro.

Decisión. La comprobación previa **normaliza y valida forma**. Recorta espacios y saltos en los extremos, exige que la referencia del proyecto sean 20 letras minúsculas y que la URL empiece por `https://`, **compone las URL derivadas después de limpiar**, y deja los valores efectivos impresos en el log.

Los secretos se detectan, no se recortan. Reescribir un valor enmascarado produce una cadena que GitHub ya no reconoce y deja de tapar en los logs. Si un secreto trae espacios en los extremos se corta el despliegue con un mensaje que lo nombra y **nunca lo imprime**.

Por qué es un ADR y no un parche. Lo caro aquí no fue el `\r\n`, fue que la comprobación **mintió**. Un paso que anuncia «✓ configuración completa» y luego rompe dos pasos más adelante es peor que no tener comprobación: convence de que la causa está en otra parte. Es la misma lección que la nº 4 del `CLAUDE.md` raíz —un test que no puede fallar no prueba nada— aplicada a un guardarraíl en vez de a un test.

Consecuencias.

- ✓ El error se da en el paso que corresponde, con el nombre de la variable y qué forma se esperaba.
- ✓ Un valor pegado con espacios de sobra **funciona**, en vez de fallar en otro sitio.
- ✓ El log deja escrito el valor efectivo del proyecto, el sitio y el callback, así que comprobar qué se usó de verdad no exige adivinar.
- ⚠ La validación de forma puede rechazar un valor legítimo si Supabase cambia el formato de las referencias de proyecto. Asumido: el mensaje dice exactamente qué se esperaba, así que arreglarlo es leer una línea.
- ⚠ Más shell en el workflow. Acotado a un solo paso y probado contra los valores reales que lo rompieron, incluidos los que llevaban `\r\n`.

Descartado.

- **Recortar también los secretos.** Rompe el enmascarado y arriesga imprimir un token en un log público. Un aviso claro cuesta un reintento; una fuga cuesta rotar la clave.
- **Confiar en que se peguen bien.** Es exactamente lo que ya se hacía. El valor lo teclea una persona en una caja de texto de otra empresa: es una entrada externa y se trata como tal.
- **Componer las URL en el bloque `env`.** Es lo que había, y es la razón de que un salto de línea acabara en mitad de una dirección.

ADR-034 — El relieve es WebGL; lo que se lee y se toca sigue siendo DOM

Estado: aceptada · 2026-08-18

Contexto. El diseño de las once vistas llega con un motor 2.5D propio: losas hexagonales extruidas, cámara isométrica, niebla de guerra y Ashfall. Contradice de frente una regla escrita de `apps/web`: «**El mapa es SVG. No Canvas, no WebGL**», cuyo motivo no era estético sino concreto — cada región es un elemento del DOM enfocable y anunciable, y eso *no se recupera* migrando a un lienzo.

Las dos cosas que la regla protegía son reales y no se negocian: que se pueda jugar con lector de pantalla y con teclado, y el presupuesto de 180 KB gzip de la ruta de partida (`three.js` pesa 132 KB gzip él solo, más que todo lo demás junto).

Decisión. Entra WebGL **como telón, nunca como interfaz**. El reparto es estricto:

Capa	Qué es	Reglas
Mundo	<code><gdc-world></code> sobre <code>three.js</code>	<code>aria-hidden</code> . No recibe foco. No decide nada
Rótulos	Elementos DOM con <code>data-tile</code>	Texto real, anunciable. El motor sólo los coloca , proyectando el mundo
Respaldo	La vista plana de siempre	Lo que se ve sin WebGL o con <code>prefers-reduced-motion</code>

Es decir: el relieve se **añade** a la interfaz, no la sustituye. Si el mundo no se monta, no queda un hueco gris — queda `CityPlan`, con exactamente la misma información.

Cómo se sostiene el presupuesto. El motor se carga con `import()` dentro de un efecto, así que sale en su propio *chunk* y no entra ni en el bundle base ni en el servidor. Se descarga cuando se monta el mundo y sólo entonces. La ruta de partida no lo paga.

Consecuencias.

- ✓ La accesibilidad no depende de una GPU. Sin WebGL, con movimiento reducido o con la batería en las últimas, la interfaz sigue **completa**, no degradada.
- ✓ Los rótulos son mejores que el `aria-label` que había: dicen qué distrito es y por qué nivel va, en vez de «tu ciudad».
- ✓ El motor es geometría procedural propia. Ni un binario de terceros — no toca la regla de assets originales.
- ⚠ **Una dependencia nueva de verdad** (`three`, 132 KB gzip). Se justifica aquí porque no hay forma de dar relieve sin ella y porque no la paga quien no la usa; si algún día entra en el bundle base, esta decisión hay que revisarla, no ampliarla.
- ⚠ Hay dos representaciones de la ciudad que mantener en paralelo. Es el precio del respaldo y se acepta a sabiendas: la alternativa era dejar sin jugar a quien no tiene WebGL.

Un fallo que se cazó al portarlo. El generador del campo repartía los sectores por ángulo (`atan2`), y las casillas justo sobre la línea de corte caían todas del mismo lado: **15 / 12 / 9** en vez de 12 / 12 / 12. Sobre un tablero decorativo parece inofensivo, pero dibujaba un mapa desigual cuando la premisa del juego es que el reparto es justo. Se cambió a repartir por **rotación**: cada casilla pertenece a una órbita de tres bajo giros de 120°, y su sector es cuántos giros la separan de su representante canónico. Ahora los tres territorios son idénticos por construcción, y hay un test que lo exige.

Descartado.

- *Mantener SVG y renunciar al relieve.* Habría sido decidir por el diseño sin coste real: el conflicto se resuelve separando capas, no eligiendo bando.
- *Sustituir el SVG por el mundo y poner `aria-label` al lienzo.* Es exactamente lo que la regla original advertía que no se puede recuperar: un rótulo sobre un lienzo no da ni foco, ni navegación por teclado, ni orden de lectura.
- *Cargar `three.js` desde un CDN,* como hacía el mockup. Rompe el despliegue reproducible y mete un tercero en la ruta crítica.

ADR-035 — El mapa de campaña se queda en SVG; el relieve es para la Ciudad

Estado: sustituida por [ADR-042](#) · 2026-08-18

Sus dos objeciones eran ciertas y ya no lo son. El mundo 2.5D dibujaba un tablero decorativo de 37 losas fijas — ahora extruye las provincias reales de `regionCells()`. Y los 96 objetivos táctiles de 44 px no cabían — una provincia se toca **entera**, por rayo sobre su superficie. Lo que hacía falta para levantar esta decisión no era insistir, era [ADR-041](#).

Contexto. El proyecto de diseño trae una vista «Mapa · Guerra» montada sobre el mismo motor 2.5D que la Ciudad (`<gdc-world scene="map">`). Al ir a implementarla aparecieron dos hechos que el mockup no podía ver:

1. **El tablero del mockup no es el mapa del juego.** El motor teje 37 losas hexagonales con el terreno fijado por anillo. El mapa real lo genera `mapgen` en **polares** ($x = \cos(\theta) \cdot r$), la adyacencia son **aristas explícitas** y no dos losas que se tocan, y el reparto de terreno sale de la bolsa del sector. Pintar el tablero decorativo bajo una partida real sería enseñar un mapa que no es el que se está jugando.
2. **El tamaño.** `SECTOR_SPEC` da **45 regiones** a 2 jugadores, **55** a 3 y **96** a 5. La vista del mockup flota tres rótulos sobre el mundo; la pantalla real necesita que *cada* región sea un objetivo tocable y enfocable. 96 objetivos de 44 px no caben en 360 px de ancho — no es una cuestión de ajustar tamaños, no hay superficie.

Decisión. El mapa de campaña sigue siendo **SVG**, con cada región como `<g role="button" tabindex="0">`, tal y como fijó [ADR-012](#). El relieve de [ADR-034](#) se queda donde sí representa lo que dibuja y donde el número de elementos es fijo y pequeño: **la Ciudad** (seis distritos).

Del diseño de la vista 05 se implementa lo que **sí** es compatible: la composición. Mapa a sangre, cabecera translúcida encima, raíl de fases y hoja de órdenes inferior con el borrador real y el botón de confirmar.

Consecuencias.

- ☒ El mapa nunca miente sobre el estado: lo que se ve son las regiones que hay, con la adyacencia que hay.
- ☒ La pantalla de campaña gana la composición del diseño sin perder ni la navegación por teclado, ni el zoom y desplazamiento por gesto, ni los objetivos táctiles.
- ☐ La partida no tiene relieve. Es deliberado, no una tarea pendiente: para tenerlo habría que construir un mundo **derivado de** `PlayerView` —losa por región en su `x/y`, aristas dibujadas— y resolver antes cómo se toca una región de 96 sin una capa de botones. Mientras eso no exista, el SVG es mejor mapa.

Descartado.

- *Poner el tablero decorativo de fondo, detrás del SVG.* Se verían hexágonos que no corresponden a los nodos de delante. Ruido que además contradice el mapa.
- *Sustituir los botones por selección con raycasting sobre el lienzo.* Devuelve el problema que [ADR-034](#) evita: sin foco, sin teclado y sin orden de lectura.
- *Nombrar las regiones como el mockup («Vado de Ceniza»).* El motor no da nombres; se usa el terreno y el identificador, que es lo que existe de verdad.

ADR-036 — Un bot no es un humano ausente, y por eso no comparten código

Estado: aceptada · 2026-08-20

Contexto. Hacen falta rivales para poder probar el juego sin reunir a cinco personas. Ya existía algo que rellenaba asientos vacíos —el **Mando Automático**— y la tentación evidente era subirle el nivel y llamarlo bot.

Es exactamente lo que **no** se puede hacer. El Mando Automático cubre a un humano que se ha ido, y su regla de diseño es *la ausencia nunca daña a un tercero*: no ataca por iniciativa propia, no rompe Sellos y no consagra. Si se le enseñara a jugar para ganar, quien se desconecta se convertiría en una ficha que mover en las negociaciones de los demás, y eso no lo ha elegido.

Decisión. Dos módulos, dos reglas:

	rules/standing.ts	rules/bot.ts
Cubre	un humano ausente	un asiento que nunca tuvo humano
Regla	la ausencia no daña a un tercero	juega para ganar
Ataca	solo para recuperar lo suyo	cuando le sale la cuenta

`standing.ts` no se toca. La resolución elige por `is_bot` **antes** de mirar los turnos perdidos, así que un humano ausente nunca cae en el código del bot.

Tres propiedades que el bot no puede perder.

1. **Decide con la misma información que un humano.** La entrada es una `PlayerView`, no el `GameState`, y evalúa los combates con `previewAttack` — la misma función que pinta la previsualización del jugador. Un rival que viera a través de la niebla no sería un rival sino un tramposo, y las pruebas de juego contra él no valdrían nada.
2. **Es determinista.** El azar sale de la semilla de la partida, nunca de `Math.random()`. Si el bot tirara un dado propio, «la partida rejugada desde (semilla, órdenes) da el mismo checksum» dejaría de ser cierto y el simulador quedaría inútil. El perfil de cada asiento también se **deriva** de la semilla: la misma campaña tiene siempre los mismos rivales.
3. **Se equivoca como una persona, no como un dado.** Un rival flojo no juega al azar: baja un escalón en su propia lista de jugadas. Jugar al azar se detecta en dos turnos y no enseña nada sobre las mecánicas.

Lo que costó entender la dificultad. El primer modelo daba a los rivales buenos un umbral de combate **alto** (atacar solo con mucho margen) y salió al revés: el temerario terminaba con 375 regiones y el implacable con 237. Ser prudente no es jugar mejor — rechazar un combate que ganarías es tan malo como entrar en uno que pierdes. El umbral se recolocó alrededor de 1,0, que es *exactamente empatar*: el malo entra por debajo, el bueno no rechaza por encima.

Y la métrica del test pasó a ser la **Ceniza**, no el territorio. Se gana consagrandolo el Núcleo y eso se paga en Ceniza; medido en regiones la escalera no sale ordenada, porque el perfil más codicioso toma menos regiones pero más ricas — que es justo lo que se le pide. Un test contra el territorio habría declarado peor al que convierte mejor.

Consecuencias.

-  Se puede jugar y medir el equilibrio en solitario, que era el objetivo.
-  El bot es código del motor, así que el simulador puede reproducir una partida con bots dentro.
-  `is_bot` **sigue siendo público**. Los rivales tienen nombre y facción para que la mesa parezca una mesa, pero no se oculta que son artificiales: que haya un bot cambia el cálculo diplomático de todos, y esconderlo contradiría esa decisión. Si algún día se quiere ocultar, es otro ADR, no un ajuste.
-  `BOT_FILL_SECONDS=0` sienta bots al instante, y con eso **dos humanos no se emparejan nunca**. Es la configuración de antes del despliegue y va en una variable de entorno justamente para poder quitarla sin migrar nada.

ADR-037 — El mapa se dibuja en hexágonos, pero no teje un panal

Estado: sustituida por [ADR-041](#) · 2026-08-21

Lo que sigue vigente: el porqué. La restricción cristalográfica es un hecho y C_5 sigue siendo la razón por la que este mapa no puede ser una retícula hexagonal. **Lo que cambia:** la conclusión de que había que dibujar las aristas. Ese ⚠ de más abajo —«dos hexágonos pueden verse próximos y no ser adyacentes»— es exactamente lo que acabó haciendo que el mapa se leyera como un árbol de investigación.

Contexto. El diseño quiere un mapa hexagonal, y con razón: es lo que dibujan los mockups y lo que permite que el mundo 2.5D represente el mapa de verdad en vez de un tablero decorativo ([ADR-035](#)).

Al ir a tejer una retícula hexagonal apareció un impedimento que no es de implementación sino de geometría. El mapa es «1 Núcleo + n sectores **idénticos por rotación C_n** », y de ahí sale la premisa del juego entero: *ganar exige pagar más Ceniza de la que da el reparto justo de un jugador*. Si el reparto no es exacto, la frase no significa nada.

Girando cada nodo de una retícula hexagonal y comprobando si cae sobre otro nodo:

orden 2: 60/60 nodos caen sobre la retícula	← posible
orden 3: 60/60	← posible
orden 4: 0/60	← imposible
orden 5: 0/60	← IMPOSIBLE
orden 6: 60/60	← posible

Una retícula periódica solo admite simetrías de orden 1, 2, 3, 4 y 6 — la restricción cristalográfica. **El orden 5 no existe en ninguna.** El juego se juega a 2, 3 y 5, así que tejer el panal costaría la equidad exacta de todas las mesas de cinco.

Hoy esa simetría sale gratis porque `mapgen` **no** usa una retícula: coloca las regiones en polares ($x = \cos(\theta) \cdot r$), y con ángulos continuos C_5 es tan fácil como C_3 .

Decisión. Las regiones se **dibujan** como hexágonos de punta arriba —la misma orientación que el mundo 2.5D— sobre las posiciones polares de siempre. La adyacencia sigue siendo la lista de aristas, no dos losas que se tocan.

Es decir: **fichas hexagonales sobre un tablero, con junta entre ellas**, no un panal continuo. La junta es visible y es el precio consciente de conservar C_5 .

Consecuencias.

- ✓ El mapa se ve como el diseño y la equidad de las mesas de cinco no se toca.
- ✓ Cada región sigue siendo un `<path>` enfocable y anunciante: [ADR-012](#) y [ADR-034](#) siguen en pie, y la conversión no costó ni un objetivo táctil.
- ✓ Abre la puerta a que el mundo 2.5D pinte el mapa **real** —una losa por región en su `x/y`— que es lo que [ADR-035](#) daba por imposible con el tablero decorativo. Eso es trabajo aparte y ese ADR sigue vigente hasta que se haga.
- ⚠ No hay vecindad implícita: dos hexágonos pueden verse próximos y no ser adyacentes. El mapa dibuja las aristas precisamente por eso, y hay que seguir dibujándolas.

Descartado.

- Retícula real, y las mesas de cinco se quedan radiales.* Dos sistemas de mapa que mantener y una partida de cinco que se ve distinta a las demás.
- Retícula real para todos, cambiando qué significa «equitativo» en cinco.* Pasar de «la misma forma girada» a «el mismo reparto» degrada una garantía **por construcción** a una comprobada por test, y encima en el modo con más jugadores.

ADR-038 — La interfaz enseña y nombra. Un icono sin rótulo no enseña: esconde

Estado: aceptada · 2026-08-23 · matiza a [ADR-027](#)

Contexto. [ADR-027](#) dice que la interfaz no explica sino que enseña, y la idea es buena: la disposición rotacional del mapa comunica «2, 3 o 5» mejor que esas cuatro palabras. Pero se llevó hasta el final y la pantalla principal acabó **sin una sola palabra**: tres glifos de simetría, tres de ritmo y un botón naranja con el emblema dentro. Sin nombre de jugador, sin facción y sin decir qué hace el botón.

El veredicto del dueño del proyecto, mirándola desplegada: «*el menú principal no tiene ni una palabra y no se entiende nada*».

Lo que falló no es la idea, es una confusión: ADR-027 prohíbe el **texto explicativo** —el párrafo que describe un botón— y se aplicó como si prohibiera **cualquier texto**, incluidos los nombres de las cosas. Un rótulo no explica; identifica. «Blitz» no es una descripción de Blitz, es cómo se llama.

Un segundo efecto, más caro: sin texto no hay nada que traducir, así que la pantalla parecía cumplir la regla de i18n cuando en realidad la esquivaba. Cero literales no es lo mismo que cero cadenas.

Decisión. Se mantiene ADR-027 para lo que decía y se le añade el límite que le faltaba:

Sigue prohibido	Sigue obligatorio	Ahora también obligatorio
El párrafo que explica un control	Que la forma comunique la regla	Que cada control se llame por su nombre
Los tutoriales y los tooltips de ayuda	El glifo junto a la cifra	Que la pantalla diga de quién es

El glifo se queda: acompaña al rótulo en vez de sustituirlo. Quien ya conoce el juego lee la forma; quien llega por primera vez lee la palabra. Las dos lecturas caben en el mismo control y ninguna estorba a la otra.

La prueba que lo caza. Un control cuyo `aria-label` es la única forma de saber qué hace está mal. El lector de pantalla lo anuncia y la pantalla no: eso no es una interfaz sobria, es una interfaz que solo funciona para quien ya no la necesita.

Consecuencias.

- ✓ La pantalla principal dice quién eres, con qué juramento, cuánta Ceniza tienes y qué hace el botón grande.
- ✓ La i18n vuelve a tener algo que cubrir, y los tests de paridad ES/EN vuelven a servir para algo en esta pantalla.
- ⚠ Más cadenas que mantener en dos idiomas, que era justo lo que ADR-027 quería evitar. Se acepta: una pantalla que no se entiende no ahorra trabajo, lo aplaza.

Descartado. *Dejarlo en iconos y añadir un tutorial.* Es cambiar un problema por otro peor y contradice ADR-027 de verdad, no en la interpretación estrecha.

ADR-039 — Se jura una vez, y hasta entonces la facción no significa nada

Estado: aceptada · 2026-08-23

Contexto. `profiles.faction_id` nace con `not null default 'vantera'` y no es escribible desde el cliente. Las dos cosas son correctas por separado y juntas dejaban un agujero: **nadie elegía facción nunca**. Toda cuenta era de Vantera sin haberlo decidido, el emblema de la Ciudad no significaba nada, y la vista de Juramento del diseño no tenía dónde escribir.

Con un `not null default` tampoco se puede distinguir «juré Vantera» de «nadie me ha preguntado»: son la misma fila.

Decisión. Entra `profiles.sworn_at`, y con él la pantalla de Juramento ([vista 02 del diseño](#)) en `/oath`. Dos pasos: juramento y nombre — el nombre es la otra cosa que no se podía configurar, y es lo que ven los demás en la mesa.

El primer juramento es **gratis y único**. Cambiar después es un Cisma, que cuesta Ceniza y renombre ([FACTIONS §5](#)), y no pasa por aquí: `swear_oath()` se niega si `sworn_at` no es nulo. La condición vive en la base de datos y no en la ruta de API, para que no dependa de que la API sea el único camino.

Sobre ADR-026. Prohíbe pantallas intermedias entre el jugador y su turno, y esto es una pantalla intermedia. No la contradice porque **no está en el camino de jugar**: está en el camino de existir, se pasa una vez en la vida de la cuenta y quien ya juró no vuelve a verla. Si algún día se pasa dos veces, es que se ha roto.

Consecuencias.

- ✓ El emblema de la Ciudad significa algo que el jugador eligió.
- ✓ El Cisma sigue costando lo que cuesta: esta puerta solo se abre una vez.
- ⚠ Una cuenta creada antes de esta migración tiene `sworn_at` nulo, así que pasará por el Juramento la próxima vez que entre. Es lo correcto: nunca eligió.

ADR-040 — El mapa es la interfaz: se ordena tocándolo, y ningún panel flota encima

Estado: aceptada · 2026-08-23

Contexto. La pantalla de campaña de v0.3 tenía el mapa a sangre y **la hoja de órdenes flotando encima**, con un degradado que a partir de media pantalla era opaco. En la práctica el resultado era el contrario del que buscaba la composición:

- El mapa dejaba de estar visible justo donde se juega.** El degradado tapaba el tercio inferior, que es donde el pulgar mira. Con la hoja de producción abierta, más de la mitad del mapa era un panel.
- Un destino resaltado podía quedar debajo de la hoja.** El mismo fallo que [ROADMAP v0.2](#), [hallazgo 3](#) ya había obligado a encuadrar el vecindario al seleccionar una fuerza: el encuadre centraba en el *viewport*, no en la parte del viewport que se ve, así que seguía ofreciendo casillas intocables.
- Las órdenes no se daban en el mapa.** Producir, ver el pronóstico y repasar el borrador ocurría en listas; el mapa solo servía para elegir destino. Una partida de conquista en la que la mayor parte de las decisiones se toman en una lista se lee como una hoja de cálculo, no como un juego.
- No se veía contra quién se juega.** La vista trae `opponents` y un `control` que es **público** (GDD §6.2), y la pantalla no enseñaba ni una cifra de ninguno de los dos: averiguar quién iba ganando exigía contar hexágonos a mano.
- Un arrastre acababa seleccionando.** No había umbral entre desplazar el mapa y tocar una región, así que navegar abría fichas sin querer.

Decisión. Cuatro cosas, y las cuatro son la misma:

a) El mapa se queda con todo el espacio que sobra, y los paneles ocupan sitio de verdad. El único cromo que flota es el que **no intercepta un gesto**: cabecera, raíl de fases y mandos de cámara. La ficha de región y la pizarra de órdenes son hermanas del mapa en la columna, no capas encima. Así el mapa nunca queda debajo de nada y el encuadre automático puede centrar en la superficie que de verdad se ve.

b) Todo se ordena tocando el mapa. Tocar una región abre su ficha y **arma el apuntado** si hay fuerza propia; tocar un destino resaltado da la orden. Mover, atacar, quedarse Firme, prestar Apoyo de Fuego, fortificar y producir son botones **con nombre** dentro de esa ficha ([ADR-038](#)), no un menú aparte. El pronóstico de combate se lee en la misma ficha del destino, sin abrir nada.

c) La pizarra enseña una casilla por fuerza, con orden o sin ella. Las casillas vacías son la pregunta que el juego le está haciendo al jugador este turno. Cuántas hay no lo decide la interfaz: son las fuerzas que el motor reconoce, y el motor no acepta más de una orden por fuerza. Tocar una casilla vacía lleva la cámara a esa fuerza.

d) Entra el reparto, que es la diplomacia que hoy se puede enseñar sin mentir. Una fila por asiento con sus regiones, sus yacimientos, si tiene el Núcleo y cuántas de sus regiones tocan las tuyas. Todo eso es **público** en el motor. Tocar una fila enciende ese territorio en el mapa y lleva la cámara a su Bastión: «¿dónde está el enemigo?» pasa a ser un tap. La Ceniza ajena no aparece porque no se ve, y **estimarla sería peor que no darla**.

Y un botón que vuelve a tu Bastión, que es el que más falta hacía: en un mapa de hasta 96 regiones, perder tu ciudad de vista y tener que buscarla arrastrando es la forma más rápida de que la partida deje de parecer un juego.

Consecuencias.

- ✓ El mapa es siempre visible y siempre tocable. No hay superficie del mapa debajo de un panel, ni por composición ni por accidente.
- ✓ Dos taps desde ver una fuerza hasta tenerla en marcha, y el pronóstico en el segundo.
- ✓ Quién va ganando se lee en una tabla, que es la premisa del juego: la diplomacia es aritmética.
- ✓ El apoyo de Fuego y la postura Firme existían en el motor desde v0.2 y **no se podían ordenar desde la interfaz**. Ahora sí.
- ✓ Las cuentas de la pantalla salen de `apps/web/lib/board.ts`, que es puro y tiene test. Antes vivían dentro de tres componentes y solo se podían comprobar mirando la pantalla.
- ⚠ La ficha de región y la pizarra se llevan hasta un 42 % del alto en 360×640. Es un techo, no una media: sin selección la pizarra ocupa lo que ocupen sus casillas.
- ⚠ **La diplomacia sigue sin existir**: no hay Sellos, ni Rupturas, ni ofertas. El reparto enseña el estado del reparto, no permite negociarlo. Eso es v0.4 y necesita motor, tablas y RLS propias; fingirlo en la interfaz sería exactamente el tipo de humo que este documento existe para evitar.

Descartado.

- *Dejar la hoja flotando y limitarle el alto.* Ya se probó en v0.2 y está en las lecciones de `CLAUDE.md`: hacer el panel más pequeño no arregla nada, siempre queda una región tocable debajo. `pointer-events: none` resuelve los taps, no la visibilidad.
- *Desplazar la cámara con un margen inferior para compensar el panel.* Funciona, pero obliga a medir el panel en píxeles y a mantener ese número sincronizado con el CSS. Que el mapa tenga su propia caja lo resuelve sin números.
- *Pasar el mapa de campaña al motor 2.5D del diseño.* Sigue vigente lo que dice **ADR-035**: su tablero son 37 losas fijas y el mapa real es un grafo en polares de 45 a 96 regiones. Lo que faltaba no era relieve, era que se pudiera jugar tocándolo.
- *Cinco casillas de orden fijas, como el mockup.* El motor admite una orden por fuerza hasta seis fuerzas. Enseñar cinco huecos con cuatro fuerzas ofrece una decisión que no existe; con seis, esconde una que sí.

ADR-041 — El mapa se tesela: territorio continuo, no un grafo dibujado

Estado: aceptada · 2026-08-24 · sustituye a **ADR-037**

Contexto. El mapa se dibujaba como lo que es por dentro: nodos hexagonales en sus posiciones polares, con junta entre ellos, y las aristas pintadas como líneas. **ADR-037** lo aceptó a conciencia y anotó el precio en sus consecuencias — «no hay vecindad implícita: dos hexágonos pueden verse próximos y no ser adyacentes, y el mapa dibuja las aristas precisamente por eso».

Ese precio resultó ser el juego entero. Jugado a cinco, con 96 regiones y el Núcleo conectado al anillo interior completo, el resultado en pantalla es una estrella de radios: fichas sueltas unidas por líneas alrededor de un centro. Es, literalmente, el dibujo de un grafo — y así se lee. La primera reacción de un jugador ante el mapa alejado fue «*parece que esto es para la investigación*». Un 4X de conquista cuyo tablero se confunde con un árbol de tecnologías ha perdido antes de empezar.

Lo que no se podía hacer era tejer un panal: la restricción cristalográfica no admite simetría de orden 5, y de esa simetría depende que el reparto entre los cinco asientos sea **idéntico por construcción**, que es la premisa del juego.

Decisión. El mapa se dibuja como una **teselación**: cada región es una provincia poligonal, y el mapa entero una superficie continua sin huecos y sin líneas de conexión.

Los polígonos son el **dual baricéntrico del grafo plano** —el esqueleto no tiene ni un cruce de aristas, comprobado—: se trazan sus caras, cada cara aporta un vértice, y la celda de una región es el polígono de las caras que la rodean. De ahí sale la propiedad que hace honesto el dibujo:

dos provincias comparten frontera si y solo si sus regiones son adyacentes.

Y con ella desaparece el motivo por el que había que pintar aristas: la frontera **es** la arista. Lo que ADR-037 tenía que compensar con líneas, aquí no hace falta compensarlo.

Vive en `packages/core/src/mapgen/layout.ts` — `regionCells(map)` — y **no** en la interfaz. La teselación define la adyacencia que el jugador ve; si la geometría viviera en el cliente y la adyacencia en el motor, el día que una de las dos cambiara el mapa ofrecería movimientos que `reduce()` rechaza. Es la misma razón por la que `previewAttack` está en el motor y no en la pantalla.

Consecuencias.

- ✓ El tablero se lee como territorio: quién tiene qué se ve por el color de las provincias, y el frente es el sitio donde dos colores se tocan.
- ✓ **La simetría C_n se conserva exactamente.** El dual de un grafo C_n -simétrico es C_n -simétrico; hay test que lo comprueba celda a celda.
- ✓ La superficie tocable de una región pasa de un hexágono de ~50 px a la provincia entera. El mínimo de 44 px deja de ser el factor que fija el zoom.
- ✓ El vértice de cada cara se pondera **por grado**. Sin eso el Núcleo —que toca el anillo interior entero— se quedaba con un cuarto del mapa: el objetivo de la partida no puede ser además la mayor provincia por accidente geométrico.
- ⚠ Un mapa de 5 sigue siendo un disco de anillos concéntricos, porque eso es lo que `mapgen` genera. La teselación lo enseña como provincias, no lo convierte en un continente. Darle forma de tierra es trabajo de `mapgen` —perturbación, prevista en v0.8— y no de la pantalla.
- ⚠ `regionCells()` es $O(V + E)$ y se recalcula al cambiar el mapa, una vez por partida. No entra en el camino de un gesto.

Descartado.

- *Un Voronoi sobre los centros de región.* Sale más orgánico y **miente**: en un mapa de 3 jugadores, 27 pares de provincias compartirían frontera sin tener arista entre ellas. El mapa estaría ofreciendo movimientos imposibles, que es peor que parecer un grafo.
- *Cambiar la adyacencia del motor para que fuera la geométrica.* Arreglaría la mentira por el otro lado, pero sube el grado medio de 3,9 a ~5, mueve el balance de movimiento y suministro entero, y rompe el invariante «ningún nodo es un callejón ni un hub» que ya tiene test. La geometría se adapta al juego, no al revés.
- *Sectores anulares perfectos, tipo diana.* Teselan igual y con menos código, pero un mapa que es exactamente una diana se lee como un diagrama. El dual da polígonos irregulares sin inventarse nada.
- *Volver a intentar el panal hexagonal.* Sigue siendo imposible a cinco, por lo mismo que explica [ADR-037](#), que en eso no ha cambiado.

ADR-042 — La campaña se juega en el mundo 3D, sobre el mapa real

Estado: aceptada · 2026-08-24 · sustituye a [ADR-035](#)

Contexto. [ADR-035](#) dejó el mapa de campaña en SVG con dos objeciones al mundo 2.5D, y las dos eran ciertas cuando se escribieron:

1. **El tablero del motor no era el mapa del juego.** `_buildBoard()` teje 37 losas hexagonales con el terreno fijado por anillo. Pintarlo bajo una partida real habría sido enseñar un mapa que no se está jugando.
2. **El tamaño.** 45 a 96 regiones, cada una un objetivo tocable de 44 px, no caben en 360 px de ancho.

Las dos han dejado de valer, y no por insistir:

- [ADR-041](#) da el **polígono real de cada provincia** con la garantía de que dos se tocan si y solo si son adyacentes. Extruir eso no es un decorado: es el mapa.
- Con provincias en vez de nodos, una región se toca **entera**. El objetivo táctil deja de ser un hexágono de 44 px y pasa a ser toda su superficie.

Decisión. La campaña se dibuja con `<gdc-world scene="campaign">`: el mismo motor de [ADR-034](#) que ya levanta la Ciudad, alimentado con `regionCells()`. Cada provincia es un prisma extruido de su polígono real, con la altura y el color de terreno de `board.ts` y el color del asiento **mezclado en la meseta** — un velo translúcido encima se lo come la luz y el mapa vuelve a ser un diorama sin frentes.

El reparto de capas de ADR-034 no se toca, y es lo que hace que esto no sea un lienzo opaco:

Capa	Qué es
Mundo	<code><gdc-world></code> , <code>aria-hidden</code> . No decide nada
Rótulos	DOM con <code>data-tile</code> : las cifras de fuerza, colocadas por proyección
Acceso	Una lista de provincias enfocable, visible al tabular
Respaldo	<code>MapFlat</code> , la vista plana completa, sin WebGL o con movimiento reducido

El motor **emite** `gdc-pick` con la provincia tocada y la pantalla decide, igual que ya hacía con la escaramuza. Tocar en relieve llama al mismo `onSelect` que tocar en plano, así que la ficha de región, el apuntado y las órdenes de [ADR-040](#) son exactamente el mismo código.

Consecuencias.

- ☒ El tablero se lee como un juego: mesetas, terreno con relieve, el Núcleo coronado y los dominios teñidos del color de cada asiento.
- ☒ Una provincia se toca entera. El mínimo táctil deja de ser lo que fija el zoom.
- ☒ El arrastre **desplaza el mapa** y el pellizco acerca, que es lo que se espera de un territorio; en la Ciudad y las vitrinas el arrastre sigue girando, que es lo que se espera de un objeto que se mira.
- ☒ La vista plana no se tira: es el respaldo, con todo lo que enseñaba.
- ☐ Sin WebGL no hay relieve. Por eso el respaldo es obligatorio y por eso `MapFlat` sigue siendo una vista completa y no un hueco gris.
- ☐ `three.js` sigue entrando por `import()` dentro de un efecto. Un import estático del motor se lleva por delante el presupuesto de 180 KB de la ruta de partida.
- ☐ El mundo se reconstruye una vez por turno, cuando cambian control y niebla. El resaltado —selección, destinos, amenazas— va por `setOverlay()` y **no** reconstruye nada: cambia materiales.

Descartado.

- *Seguir en SVG y darle sombreado.* Lo que faltaba no era sombra: era volumen. Un mapa plano con degradados sigue leyéndose como un plano.
- *Raycasting como única forma de tocar.* Es lo que ADR-034 evita: sin foco, sin teclado y sin orden de lectura. Por eso el rayo convive con la lista enfocable.
- *Poner las cifras de fuerza dentro del lienzo.* Lo prohíbe ADR-034 y con razón: si no está en el DOM, para media clase de jugadores no existe.

Un fallo que salió de aquí. `hexPrism()` gira la losa y **además** la traslada su propia altura, así que la pieza flota y todo lo que se coloca «encima» acaba medio enterrado. En la Ciudad pasaba desapercibido; aquí dejaba los resaltados **dentro** de la provincia, sin ver. La escena de campaña lee la cota superior de la geometría en vez de suponerla.

ADR-043 — El mapa se reparte por área: todas las provincias miden lo mismo

Estado: aceptada · 2026-08-24

Contexto. ADR-041 hizo que el mapa se leyera como territorio, pero al mirarlo montado seguía pareciendo roto. Medido, el motivo no era el render:

dispersión de superficie entre la provincia mayor y la menor			
2 jugadores	×2,82	Núcleo	×0,75 de la mediana
3 jugadores	×2,14	Núcleo	×0,94
5 jugadores	×1,95	Núcleo	×1,69

Casi tres veces de diferencia. Y no es solo estético: **capturar una región valía cosas distintas según dónde cayera**, en un juego cuya premisa es que el reparto es verificable.

Dos causas, las dos en `skeleton.ts`:

1. **Los anillos se separaban por un hueco constante** (`RING_GAP`), sin mirar cuántos nodos tenía cada uno. Un anillo de 15 nodos y otro de 30 recibían la misma banda, así que las provincias del primero salían al doble.
2. **El anillo exterior se estiraba hasta el borde del mundo.** `extent` añadía un margen y las celdas de fuera llegaban hasta él: un 45 % más grandes que las de dentro.

Decisión. El mapa se reparte **por área**. A cada anillo se le da exactamente la banda que necesita para que todas sus provincias midan lo mismo:

$A = \pi \cdot \text{CELL_RADIUS}^2$	← la provincia tipo
$b[r+1]^2 = b[r]^2 + \text{CELL_RADIUS}^2 \cdot n(r)$	← la banda del anillo r
$\text{radio}(r) = \sqrt{(b[r]^2 + b[r+1]^2) / 2}$	← el nodo, en la mitad por ÁREA

El nodo va en el radio que parte su banda en dos mitades de igual superficie, no en el punto medio: con anillos anchos el medio geométrico deja más área fuera que dentro y la provincia se descuelga. Y `extent` pasa a ser **la frontera exterior del último anillo**, ni un punto más: el mundo es el mapa.

El Núcleo tiene su propia cuota, `CORE_SHARE`, y su peso en la teselación se despeja en forma cerrada de la geometría en vez de salir de una heurística.

dispersión después	Núcleo
2 jugadores ×1,72	×1,25 de la mediana
3 jugadores ×1,60	×1,26
5 jugadores ×1,72	×1,31

Consecuencias.

- ☒ Una provincia vale aproximadamente lo mismo caiga donde caiga. Hay test, y fija la **intención** —dispersión < ×2— no las constantes.
- ☒ El Núcleo se nota que es el objetivo sin comerse el centro, y su tamaño ya no depende del número de jugadores.
- ☒ El tablero se lee como un reparto y no como un montón de piezas de tamaños dispares.
- ☐ `MAPGEN_VERSION` sube a **0.2.0**: la misma semilla ya no da el mismo mapa. Las partidas en curso guardan el suyo en `game_states` y siguen exactamente igual.
- ☐ La dispersión no baja de ×1,7. Lo que queda viene de que las conexiones radiales entre anillos de distinto tamaño no pueden ser todas iguales, y **eso es adyacencia**, o sea balance. La geometría se adapta al juego, no al revés ([ADR-041](#)).

Descartado.

- *Relajación de Lloyd sobre las posiciones.* Es la técnica de manual para igualar celdas y aquí **empeora**: medida, sube la dispersión de ×1,72 a ×6,77 en seis pasadas. Con el borde exterior fijo, las celdas de la costa se estiran y las de dentro se hunden. Se descarta por medida, no por intuición.
- *Ponderar el vértice de cada cara por el grado del nodo.* Parecía elegante y costaba caro: un nodo de grado 5 tiraba más que uno de grado 4 y acababa con una provincia un 50 % más pequeña, sin que eso significara nada en el juego. Peso 1 para todos menos el Núcleo.
- *Cambiar los tamaños de anillo de `SECTOR_SPEC` para que fueran proporcionales al radio.* Eso es composición de sector: cambia cuántas regiones hay, la bolsa de terrenos y el tamaño del mapa. Es diseño de juego, no un ajuste de dibujo.

ADR-044 — La cámara se mide en provincias, y el mapa enseña el frente

Estado: aceptada · 2026-08-24

Contexto. Con [ADR-043](#) las provincias ya medían todas lo mismo, y el mapa montado **según sin parecer un juego**. Mirando la pantalla y no el código, tres cosas:

1. **La cámara heredaba el encuadre de la Ciudad.** El `fudge` de 0,66 acerca la cámara para que una maqueta de 37 losas llene el cuadro; aplicado a 96 provincias amontonaba el lado lejano contra el horizonte. Las de delante enormes, las de atrás una pila de esquirlas: el efecto era el de un montón de cristales rotos, no el de un territorio.
2. **La decoración estaba dimensionada para un hexágono que se mira de cerca.** Cuatro edificios de la Ciudad puestos sobre una provincia tapaban medio territorio.
3. **No se veía dónde acababa lo tuyo.** Cada provincia iba del color de su dueño, y a cinco jugadores y bajo niebla eso hay que deducirlo comparando manchas. La queja del jugador fue literal: «*no se muestra claramente dónde están los enemigos*».

Decisión. El mapa de campaña se encuadra y se dibuja como **una carta de situación**, que es el registro visual del proyecto, y no como el diorama de la Ciudad.

- **El encuadre se mide en provincias, no en fracciones de mapa.** Se abre a `OPEN_SPAN` radios de provincia en el semieje corto —unas siete provincias de ancho en un móvil—. Una fracción fija del mapa daba provincias

del doble de tamaño en una partida de tres que en una de cinco; lo que tiene que ser constante es **cuánto mide una provincia en pantalla**, porque de eso dependen que se lea, que se toque y que sepas dónde estás.

- **El ajuste de distancia es exacto, con el término de perspectiva.** El mapa es un disco horizontal de radio R : mide R de ancho y $R \cdot \sin(\text{elev})$ de alto, y el borde cercano está $R \cdot \cos(\text{elev})$ más cerca que el centro, así que se proyecta más grande. Sin ese término el encuadre cuadra en papel y recorta el mapa en pantalla.
- **El objetivo de la cámara se recorta a lo que hay mapa.** Se puede desplazar exactamente lo que no cabe en cuadro: a encuadre completo, nada. Y no se puede alejar más allá del mapa entero.
- **Ir a tu ciudad también acerca.** Con el mapa entero en pantalla no hay adónde desplazarse, así que el botón parecía averiado justo cuando más falta hace.
- **El frente se dibuja.** Toda frontera entre dos provincias de dueño distinto —o entre una con dueño y tierra de nadie— lleva una franja clara encima. Sale de la propia teselación de [ADR-041](#): los vértices compartidos son el mismo punto, así que la pareja se encuentra por igualdad exacta.
- **La decoración es marca de terreno, no decorado.** Se encoge a la provincia que le toca, y las provincias no miden todas igual.

Consecuencias.

- ☒ Una provincia mide lo mismo en pantalla a dos, tres y cinco jugadores.
- ☒ Se ve de un vistazo dónde acaba tu territorio y con quién limita: la pregunta que abre un turno de 4X tiene respuesta sin tocar nada.
- ☒ El mapa no se puede perder de vista arrastrando, ni queda flotando en un cuadro negro.
- ☒ La decoración cuenta el terreno sin taparlo, y ya no hay una luz puntual por yacimiento —que en un mapa de 96 eran decenas y las compila todas el shader—.
- ☐ En un móvil vertical, el mapa entero deja franja negra arriba y abajo: un disco no llena un cuadro de proporción 0,47. Se acepta en el tope de alejamiento; en el encuadre de trabajo no se nota.
- ☐ Los paneles flotantes pueden tapar una ficha de fuerza que caiga bajo ellos. Los taps pasan ([ADR-040](#)) pero la cifra no se ve. Pendiente.

Descartado.

- *Cámara ortográfica.* Quita el escorzo del todo y es la solución de manual para una carta de situación, pero obliga a tocar `_place`, `_resize`, la proyección de rótulos y el rayo de selección, que hoy comparten las seis escenas. Con el término de perspectiva y una inclinación de 1,18 rad el amontonamiento desaparece igual. Si vuelve a aparecer, esto es lo siguiente que hay que probar.
- *Subir la inclinación hasta el cenital.* Mata el relieve, que es lo que distingue una Elevación de una Llanura sin leer un rótulo.
- *Pintar el frente con líneas.* `LineBasicMaterial` ignora `linewidth` en WebGL: mide un píxel y desaparece al alejarse. Por eso es una franja de geometría.

ADR-045 — El mapa de campaña usa el lenguaje visual del mockup, sin reinventarlo

Estado: aceptada · 2026-08-24

Contexto. El proyecto tiene un mockup de diseño (`world3d.js`, el `.zip` de referencia) que **ya define cómo se ve el tablero**. Al llevar la campaña al relieve no se aplicó: se inventó un lenguaje propio, y el resultado no se parecía al diseño de partida. Punto por punto, lo que hacía el mockup y lo que se hizo en su lugar:

	Mockup	Lo que se había hecho
Terreno	Color pleno (<code>TOP[kind]</code>)	Apagado hacia la pizarra
Dominio	Anillo emisor del color del asiento, en el borde	Teñir la meseta entera
Estandarte	<code>banner()</code> en cada provincia con dueño	No existía
Fuerzas	<code>unitPiece()</code> , silueta por arma, sobre la losa	Solo cifras en el DOM
Canto	Bisel de 0,045	Quitado
Junta	<code>GAP</code> 0,955 — losas separadas	Provincias pegadas
Alturas	0,14 - 0,95	Comprimidas a 0,05 - 0,40

Cada desvío tenía su razón escrita y **ninguna resistía mirar el mockup renderizado**: el terreno apagado se justificó como «que no compita con el dominio», y el problema real era teñir la meseta; las alturas se comprimieron para arreglar un amontonamiento que en verdad venía de la cámara ([ADR-044](#)).

Decisión. El mapa de campaña usa el lenguaje del mockup, y las diferencias solo se permiten cuando la geometría obliga —la provincia es un polígono irregular, no un hexágono.

- **El terreno va a color pleno.** Lo que dice de quién es la provincia es su filo.
- **Filo de dominio** del color del asiento, construido sobre el contorno real. El mockup usa un anillo del 0,6 al borde, que sobre un hexágono mirado de cerca es una corona ancha; sobre 96 provincias esa corona vuelve a ser el mosaico que se quería evitar, así que aquí llega al 0,9 — es un filo, no una mancha.
- **Estandarte** en cada provincia con dueño que no sea el Bastión, que ya tiene mástil.
- **Las fuerzas son piezas**, con silueta por arma. La cifra sigue en el DOM ([ADR-034](#)): la pieza dice que hay alguien y de quién es, el rótulo cuántos.
- **Junta, bisel y alturas del mockup**, tal cual. La junta se hace encogiéndola la celda hacia su centro con el mismo `GAP`.
- **Inclinación de cámara 0,78 rad**, cerca del 0,62 del mockup: es un diorama isométrico, no una vista cenital.

Consecuencias.

- ☒ El tablero se parece al diseño del proyecto, que era el objetivo desde el principio.
- ☒ El terreno se lee —bosque, agua, yermo— **y** el dominio también, porque ya no ocupan el mismo canal.
- ☒ Vuelve a haber piezas sobre el tablero: es un juego, no un mapa político con pegatinas.
- ☐ El filo de dominio se dibuja **por encima del velo de niebla**. Es deliberado: el control territorial es público (GDD §6.2) y esconderlo era ocultar información que el juego da.
- ☐ Sustituye a la parte de [ADR-044](#) que hablaba de apagar el terreno y de dibujar el frente como franja de ceniza. El encuadre en provincias y el recorte de cámara de esa decisión **siguen vigentes**; sin ellos, las alturas del mockup vuelven a amontonarse.
- ☐ Las luces puntuales de los yacimientos siguen apagadas en campaña: en la Ciudad son tres y aquí serían decenas.

Descartado.

- *Mantener la franja de frente de ADR-044 además del filo.* Con el filo del color del asiento en cada provincia, el frente **ya está dibujado** y con más información —se ve de quién es cada lado—. Las dos capas juntas competían.
- *Copiar el anillo del mockup al 0,6.* Medido en pantalla: cubre el 36 % de la superficie de la provincia y devuelve el mosaico de colores de asiento.
- *Teselar en hexágonos para poder copiar el mockup literalmente.* Es [ADR-041](#) otra vez: la simetría de orden 5 no admite panel, y de ella depende el reparto justo.

Plantilla para nuevas decisiones

```
## ADR-0XX – Título
**Estado:** aceptada | propuesta | sustituida por ADR-0YY · AAAA-MM-DD

**Contexto.** Qué problema obliga a decidir.

**Decisión.** Qué se decide, en una frase.

**Consecuencias.**
-  lo que ganamos
-  lo que cuesta, y cómo se mitiga

**Descartado.** Qué alternativas se consideraron y por qué no.
```